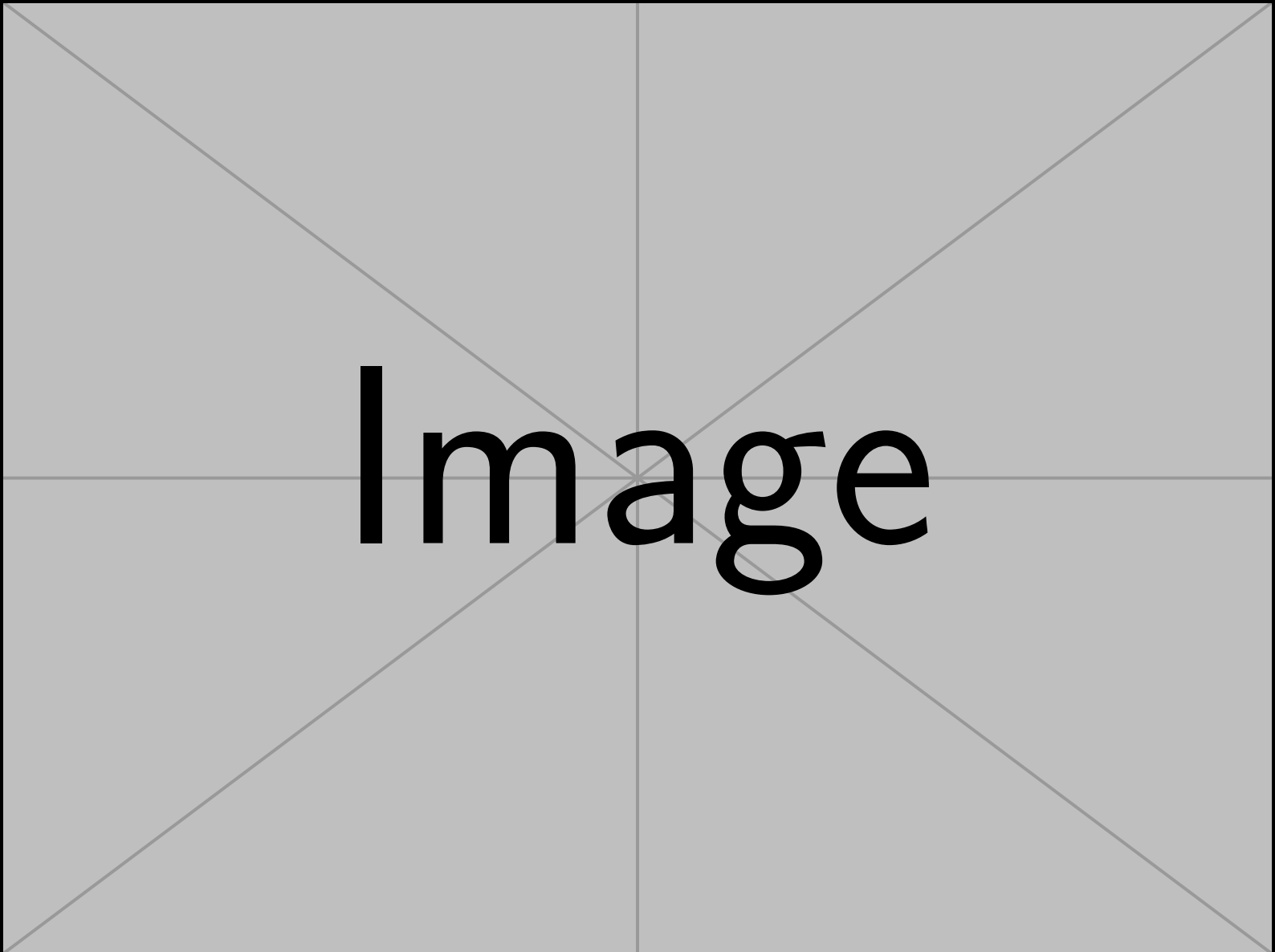




Image

气动软体机器人自研实战

从仿真到产品的完整之旅——FLOWIO-CN 全栈实录

作者：张越飞 徐颖

组织：FLOWIO-CN 项目组

时间：2026 年 10 月

版本：0.1（草稿）

开源：代码/图纸/STL 见配套仓库

开源书稿：正文 CC BY-NC-ND 4.0，代码 MIT，硬件 CERN-OHL-S。

目录

第一章 数字孪生：从概念到工程	1
1.1 概念与演化：从 PLM 构想到工业互联网基石	1
1.2 五维模型：给”孪生”一个可检查的骨架	1
1.3 使能技术栈：建模、数据、通信、服务	2
1.4 机器人数字孪生：三个近年样本	3
1.5 本书路线图：理论如何兑现为十二章	3
第二章 气动软体机器人驱动与控制	5
2.1 驱动方式谱系：为什么是气动	5
2.2 气动系统建模：三条方程与一个类化	5
2.2.1 多变气体定律：压力动力学的主方程	5
2.2.2 孔口方程：流量的两个分支	6
2.2.3 集中参数类化：RC 电路的气动镜像	6
2.3 控制器谱系：从 PID 到模型基非线性	7
2.4 交互装置脉络：PneUI → OmniFiber → FlowIO	7
2.5 康复手套前沿：便携拇指手套与织物综述	7
2.6 本书定位：在谱系坐标上	8
第三章 软体机器人与 FLOWIO-CN 生态位	9
3.1 软体驱动的前世今生：三条主线	9
3.2 平台化的呼唤：气动工具包谱系	10
3.3 MIT FlowIO：架构解剖	11
3.4 FlowIO 的边界，就是 FLOWIO-CN 的切入口	11
3.5 FLOWIO-CN 的差异化定位	12
3.6 从文献差距到产品需求	13
3.7 被引数据里的市场信号	13
3.8 康复手套的工程解剖：客户到底在买什么	14
3.9 全栈自研的产品思维	14
3.10 文献 workflow：结论是怎么来的	15
3.11 如何读这本书	15
第四章 孪生先行：物理建模与仿真	17
4.1 逻辑与物理分离：一份 pn_core，两个世界	17
4.2 从 v1 到 v2：让模型长出文献的形状	18
4.3 核心方程与 dP/dt 实现	18
4.4 参数成色：孪生模型也会”说谎”	19
4.5 v2.1 多节点端口模型：一个 bug 牵出的物理修正	20
4.6 形状对不对，断言说了算	21
4.7 传感器模型：对着手册逐条校准	21
4.8 板级电气孪生：BOARD_PARAMS 与黄徽章	21
4.9 仿真套件四电路与对拍闭环	22
4.10 孪生即测试台：API v1.2 一览	22

4.11 到货之后: <code>calibrate_pump.py</code> 三实验协议	23
第五章 原理图: 从拓扑到 13 次修正	24
5.1 设计基线: 80% 继承与总体拓扑	24
5.2 原理图即代码: <code>gen_sch.py</code> 与程序化校验	24
5.3 三个真实修正案例: 症状、根因与修法	24
5.3.1 案例一 (修正 #9): Diode-OR 的二极管——1A 器件扛 2A 电流	24
5.3.2 案例二 (修正 #11): Buck 反馈电阻——2.33V 的“3.3V 轨”	25
5.3.3 案例三 (修正 #10+#12): USB 转串口芯片三次换型——CH340N→E→K	25
5.4 13 次修正总表	25
5.5 修正从哪里来: 三层交叉检查	26
第六章 PCB 布线攻坚: 从 146 到 0	27
6.1 布局定局与层叠	27
6.2 146→17: freerouting 双轮与布线后清理	27
6.3 天线禁布区事件: 目视检查抓到的隐形事故	27
6.4 模型与工具攻坚: 17→6	27
6.5 6→0: KRT 平面修复与终版打样包	27
第七章 制造: 嘉立创全代工实战	30
7.1 下单资产包: fab/ 目录里都有什么	30
7.2 制板参数与三重验证	30
7.3 下单七步: 从 Gerber 到付款	31
7.4 via-in-pad × 8: 塞孔封帽工艺	31
7.5 BOM 的单一真值源: <code>make_bom.py</code> 与 LCSC 编码	32
7.6 标准版是硬约束, 不是省钱选项	33
7.7 板 5 装 2 与费用区间	33
7.8 CPL 坐标与人工核对	34
7.9 收货验收: 从目检到上电	34
7.10 制造交付的完成定义	35
第八章 结构: FreeCAD 参数化外壳	36
8.1 从 OpenSCAD 到 FreeCAD: 一次工具链变更	36
8.2 参数区: 外壳的全部体质	36
8.3 尺寸传递链: 从板到壳的一条路	37
8.4 侧开孔清单: 每个孔对着一个连接器	38
8.5 公差链: 间隙、配合与自攻	38
8.6 HD 移位: 一个安装孔的 (77,62)→(68,65)	38
8.7 铜柱、自攻与装配动线	39
8.8 天线净空与通风栅	39
8.9 孪生网格: <code>make_meshes.py</code> 与坐标映射的证伪	40
8.10 3D 打印下单与水密自检	40
第九章 固件: ESP-IDF 分层架构	43
9.1 四层架构与两个“没有”	43
9.2 主机 33 测试: 逻辑层的证据链	43
9.3 动作层: 阀位组合与状态字	44

9.4 CLI: 十三条命令的调试与验收两用	44
9.5 TCA9548A: 函数注入保住的纯逻辑	45
9.6 pn_cmd_feed: 串口与 BLE 的同一入口	46
9.7 10ms 控制节拍: 七件事与一盏灯	47
9.8 QEMU 冒烟: 没有射频与外设的全固件测试	48
第十章 BLE 无线: 契约先行	49
10.1 架构: 命令不新开解析器	49
10.2 UUID 命名空间与四服务	49
10.3 载荷一: 0xA5 帧骑在 write 上	50
10.4 载荷二: 20 字节 state 帧与小端纪律	50
10.5 线程模型: 回调不阻塞协议栈	52
10.6 真机验证三步与无 BT 降级	53
第十一章 上位机与孪生前端 v2: 产品即主角	54
11.1 布局倒转: 从仪表盘到舞台	54
11.2 流拓扑同源: make_flows.py 与 flows.json	55
11.3 电流辉光与气流粒子: 数据怎么“长”上去	55
11.4 板级孪生: 解析式 RL 与一次续流 τ 审查	56
11.5 双源轮询与安静降级	57
11.6 仿真实验室: 四电路重算浮层	58
第十二章 Python SDK: 把协议钉死	59
12.1 0x8C 事故未遂: 计划草案不是真源	59
12.2 crc8 与 encode: 全函数与两枚锚点	59
12.3 vectors.json: 与固件同向量的回归网	60
12.4 client.py: 高层 API 与传输接缝	61
12.5 hello_glove: 一个来回的冒烟示例	61
12.6 接口面: SDK 的边界即产品的边界	62
第十三章 验收实录: 数字孪生与实物	64
13.1 上篇: 数字孪生验收 (已完成)	64
13.1.1 A101 装配叙事与爆炸跟随	64
13.1.2 A102 开阀电流亮起、真空反向与静默差分	64
13.1.3 A103 仿真实验室四电路重算	66
13.1.4 A104 SDK 协议向量双向 + hello_glove 冒烟	67
13.1.5 A105 固件 QEMU 冒烟 + 主机 33 测试	67
13.1.6 A106 BLE 契约载荷向量 (向量先验)	67
13.1.7 test_webapp.js: 44 断言矩阵	67
13.1.8 26 项目视审计: 从截图到像素证据	69
13.1.9 验收工单机制: 一张工单, 两份产物	70
13.2 下篇: 实物验收 (工单驱动, 待回板)	71
13.2.1 回板后 workflow: 每张工单 15-30 分钟	71
13.2.2 硬件 24 张 + 标定 6 张工单总表	72
13.2.3 首电安全三测预告 (A201): 万用表的唯一保留环节	73
第十四章 产品化: 标定、精度与路线	74

14.1 Phase CAL 六项对拍	74
14.2 数字孪生精度报告	74
14.3 路线图	74
参考文献	75

第一章 数字孪生：从概念到工程

本书的孪生先行章（第 4 章）会让固件在真板回来之前就跑在一套气动微分方程上；孪生前端章（第 11 章）的 Web 页面会以”孪生/真机”两种徽章渲染同一份遥测。这些工程实践并不是凭空发明——它们站在一个已有二十余年积累的方法论之上：**数字孪生**（digital twin, DT）。本章把这条理论线索讲清楚：数字孪生从哪里来、它的五维模型是什么、靠哪些技术栈落地、在机器人领域已经走到哪一步。理论篇的使命只有一个——让后面十章的每一条孪生决策都能向上追溯到一个被反复引用过的框架，而不是”别人都这么做”。

1.1 概念与演化：从 PLM 构想到工业互联网基石

数字孪生的思想源头通常追溯到 2003 年前后 Grieves 在密歇根大学产品全生命周期管理（PLM）课程中提出的”镜像空间模型”：一个产品系统应当同时包含**物理空间**的实体、**虚拟空间**的数字化模型，以及连接两者的**信息接口**——三要素构成的闭环，是此后一切数字孪生定义的雏形。2010 年代初，NASA 在飞行器健康管理的技术路线图中采用 digital twin 一词，用它描述”与单个飞行器全生命周期对应的集成多物理、多尺度、概率性仿真模型”，概念由此从课堂构想进入工程语境；工业互联网与工业 4.0 浪潮随后把它推成了智能制造的核心关键词之一。

对这段历史与工业落地现状最系统的整理，是陶飞团队 2019 年发表于 IEEE TII 的综述 *Digital Twin in Industry: State-of-the-Art* [10]。这篇综述在 2026 年 10 月经 AMiner 检索已被引用逾 4600 次，是中文世界被引最高的数字孪生文献之一；它把概念提出到成文的时间差概括为”约十五年”，并把工业应用归纳为产品设计、生产制造、故障预测与健康管理（PHM）等若干大类，同时列出数据、建模、标准三个层面的挑战。对本书而言，这篇综述的价值有三：其一，它确认了”虚拟-物理无缝集成”是数字孪生的定义性特征，而非附加属性；其二，它把 PHM（预测性健康管理）列为一级应用方向——本书验收篇（第 13 章）的泄漏诊断与孪生对拍精度报告，本质上就是驱动板这一”产品”的 PHM 缩影；其三，它指出的挑战（模型可信度、数据质量、标准缺位）恰好对应本书第 4 章”参数成色四档”与第 14 章”精度报告”要解决的问题——理论界的挑战清单，就是工程书的目录。

概念演化见表 1.1。

表 1.1: 数字孪生概念演化脉络（据 [10] 综述整理）

时期	事件	对本书的含义
2003 前后	Grieves 于 PLM 课程提出镜像空间模型：物理/虚拟/连接三要素	孪生的最小定义，即”模型 + 实体 + 闭环”
2010 年代初	NASA 路线图采用 digital twin 描述飞行器健康管理	PHM 成为一级应用，对应验收篇对拍与泄漏诊断
2015–2019	工业 4.0/工业互联网把 DT 推为智能制造使能技术	B2B 客户对”孪生”有认知，数据报表有市场
2019 至今	工业综述 [10]、建模方法论 [11]、使能技术盘点 [3] 相继成体系	本书孪生架构的理论锚点（本章 §1.2–§1.3）

1.2 五维模型：给”孪生”一个可检查的骨架

概念会通胀，结构不会。数字孪生领域最经得起引用的结构化表述，是陶飞学派提出的**五维模型**：物理实体（physical entity, PE）、虚拟实体（virtual entity, VE）、服务（services, Ss）、孪生数据（twin data, DD）与连接（connection, CN）五者互联 [3]。它把早期三要素扩展为工程可检查的清单——一个声称”有数字孪生”的系统，应当能逐维回答”你的 PE 是什么、VE 在哪、数据流是什么、提供什么服务、靠什么连接”。

五维各自的内涵，在陶飞团队 2022 年的 *Digital Twin Modeling* [11] 中得到系统化：该文从应用领域、层级、学科、维度、普适性、功能性六个视角剖析现有孪生模型，再按建模理论体系归纳六个建模方面，并盘点配套的

使能技术与工具。对建模者最重要的论断是：**孪生建模是数字孪生准确刻画物理实体的核心**，孪生的功能服务能否兑现，取决于模型成色 [11]。这句话在本书将被反复验证——第 4 章的物理 v2 重构（每条规律给出文献出处）、参数成色四档（实测 > 手册 > 仿真 > 假设），正是“模型成色决定服务成色”在一张 90×75 mm 驱动板上的落地。

把五维模型当作检查表套在 FLOWIO-CN 上，得到表 1.2——它也是理解全书结构的另一张地图。

表 1.2: 五维模型 → FLOWIO-CN 孪生逐维映射

维度	FLOWIO-CN 中的实现（章）
PE 物理实体	P1 驱动板 + 五路执行器气路 + 泵阀传感（第 5、6、7 章）
VE 虚拟实体	twin_api.c 气动物理 + board_model.py 电气 + 3D 装配模型（第 4、8、11 章）
Ss 服务	闭环节拍、对拍精度报告、验收工单、数据报表（第 9、13、14 章）
DD 孪生数据	50 ms 遥测流、600 s 环形历史、CSV 对拍数据集（第 4、11 章）
CN 连接	串口 0xA5 协议、BLE GATT 四服务、HTTP API（第 10、12 章）

这张映射表还暴露了一个诚实的细节：五维的“成色”并不均一。PE 维是实测的（板在手），CN 维是契约化的（协议有测试），但 VE 维的若干参数（阀线圈电阻/电感、泵死点）在 BOM 锁定前仍是手册值或典型值——第 4 章用徽章把它们显式标黄，而不是藏在演示数字后面。把未标定的维度标成黄色，是五维模型对单人项目最有用的一条工程化改造：它把“模型可信度”（[10] 列出的领域级挑战）压缩成了一张人人可查的表。

1.3 使能技术栈：建模、数据、通信、服务

五维是骨架，使能技术是血肉。齐庆林（Qi）等 2021 年发表于 *Journal of Manufacturing Systems* 的 *Enabling technologies and tools for digital twin* [3] 从五维视角盘点了高频使用的使能技术与工具，是引用逾千次的领域工具书式综述。按“建模/数据/通信/服务”四类归纳，技术栈与本书的对应关系见表 1.3。

表 1.3: 数字孪生使能技术四类与本书选型（框架据 [3]）

类别	领域常用技术（综述概括）	FLOWIO-CN 选型
建模与仿真	几何/物理/行为/规则多层建模，多物理场仿真	C 物理积分（多变定律 + 孔口）+ Python 电气模型 + STL 装配
感知与数据	多源传感、数据融合、实时数据库	6 路 XGZP6897D 并行采样、50 ms 遥测、环形历史
通信与连接	总线、无线、网络协议	I2C/TCA9548A、UART 0xA5、BLE GATT、HTTP
服务与应用	监控、仿真、预测、优化四类服务 [11]	遥测面板、时间缩放回放、泄漏诊断、闭环整定

值得强调的是综述里的一条隐含纪律：**技术选型应从维度需求出发，而不是从流行度出发**。五维中 DD 维的“实时性”决定连接技术的采样率与协议；Ss 维的“预测”才引入学习类方法。本书没有引入任何学习型组件——因为它的服务清单（闭环、对拍、报表）用物理模型加统计即可兑现；这条“按服务倒推技术”的路线，正是 [3] 工具盘点方法论的逆序使用。

1.4 机器人数字孪生：三个近年样本

数字孪生的工业叙事以飞机、产线、车间为主角；机器人是其中更年轻的分支。三个近年样本足以勾勒这条支线的现状（表 1.4）。

样本一：持续学习抓取检测。北航 Ren 等 2024 年发表于 IEEE TIE 的 *Digital Twin Robotic System With Continuous Learning for Grasp Detection in Variable Scenes* [4] 构建了“数字孪生 + 持续学习”的机器人系统：孪生侧生成由工业零件构成的合成抓取检测数据集，物理侧在变化场景中抓取，新场景数据回流触发模型再训练——用孪生数据补真实数据的稀缺，用持续学习吸收场景漂移。这是“孪生数据（DD 维）直接喂养服务（Ss 维）”的完整示范。

样本二：手术模拟器。Cai 等 2023 年在 *Bioengineering* 上实现的虚拟现实数字孪生微创手术模拟器 [1]，把孪生用于技能训练而非监控：物理机器人执行操作，孪生侧渲染 VR 场景并评估操作质量。它提示孪生的 Ss 维不必是“看板”，也可以是**训练与评估环境**——本书孪生前端 v2 的“仿真实验室”浮层（第 11 章）与此同宗：把危险或昂贵的物理试验搬进可重放的数字副本。

样本三：物理-虚拟机器人手。Singh 与 Ray 2024 年在 IJIDeM 上提出物理-虚拟数字孪生机器人手 [8]，让物理手与虚拟手双向映射、协同演示。它与本书的关系最近：手的分指动作在虚拟侧复现，正是康复手套数据报表的产品形态预演——白牌厂商的客户（康复科）最终看到的“疗程数据”，就是这类物理-虚拟映射的消费端。

表 1.4: 机器人数字孪生三个样本与本书的对照

工作	孪生的主要用途	与 FLOWIO-CN 的同构	差异
Ren 2024 [4]	合成数据 + 持续学习	DD 维反哺服务	学习型组件本书暂不引入
Cai 2023 [1]	VR 训练评估环境	仿真实验室浮层同宗	本书面向产品验收而非技能训练
Singh 2024 [8]	物理-虚拟双向映射	分指遥测的报表预演	本书映射对象是驱动板与气路

三个样本的共同点值得写成一句判词：机器人数字孪生的价值不在“复制出一模一样的几何”，而在**让数字副本承担物理系统做不起的试验**——数据稀缺时造数据（Ren），试验昂贵时当沙盘（Cai），展示不便时做镜像（Singh）。本书孪生承担的正是第三种之外的全部：回板前的固件沙盘（第 9 章 QEMU 冒烟）、协议调试的假硬件（第 10、12 章）、出厂筛选的泄漏工况注入（第 4 章）。

1.5 本书路线图：理论如何兑现为十二章

本章的理论至此收束为三条可执行的原则，它们将在实践篇被逐条兑现：

- **五维检查表：**任何自称孪生的模块，必须能回答 PE/VE/Ss/DD/CN 五问（表 1.2）。第 4 章的孪生架构、第 11 章的前端信息架构，都按这张表自检。
- **模型成色分级：**孪生建模是核心，模型成色决定服务成色 [11]；实测 > 手册 > 仿真 > 假设的四档徽章（第 4 章）是这条原则的操作化。
- **按服务倒推技术：**使能技术的取舍从 Ss 维服务清单出发 [3]，不从流行度出发——这就是本书没有机器学习组件、却有 36 张验收工单的原因（第 13 章）。

第 3 章（生态位）回答“为什么做”，本章回答“用什么理论做”，第 4 章开始回答“具体怎么做”。第 II 部的十章实践，本质上就是把表 1.2 的五行逐行施工；第 III 部的验收与产品化，则是对 Ss 维（精度报告、数据报表）的最终交付。理论到此让位。

本章小结

数字孪生从 Grieves 的三要素构想演化到工业互联网的基石 [10]，其最有工程价值的结晶是五维模型 [3] 与“建模为核心”的方法论体系 [11]；机器人分支的三个样本 [1, 4, 8] 展示了孪生在造数据、当沙盘、做镜像上的价值。本书把五维模型用作检查表、把模型成色用作徽章、把服务清单用作技术选型的倒推依据——理论不是装饰，是第 4 章之后每一张图的坐标轴。

第二章 气动软体机器人驱动与控制

第 4 章的孪生物理引擎 v2 里跑着三条方程：多变气体定律、ANSI 孔口流量、RC 集中参数类化。它们不是本书的发明，而是气动软体机器人社区近十年的公共财产——大部分来自 Xavier、Fleming 与 Yong 的系列工作与 Facebook Reality Labs 的气路响应时间模型。本章把这些方程的来历、适用条件与推荐参数讲透；第 4 章只是它们的工程化读者，本章是文献意义上的原始出处课。同样重要的一半是**装置史**：从 PneUI 到 FlowIO 的交互装置脉络与从拇指手套到织物综述的康复前沿，共同构成 FLOWIO-CN”分指压力闭环 + B2B 白牌”定位的参照系。

2.1 驱动方式谱系：为什么是气动

软体机器人的”软”来自执行器，驱动执行器的能量形式却只有有限的几条路线。Xavier 等在其非线性控制器综述的引言里列举了软机器人常用的驱动族：介电弹性体、气动、液压、形状记忆合金（SMA）等 [12]。把工程约束（响应速度、功率密度、驱动电压、可微制造性、成本）压上去，谱系就收窄了，见表 2.1。

表 2.1: 软体执行器主流驱动方式谱系

驱动方式	代表特征	主要瓶颈	在本书场景的裁决
气动（流体）	响应快、力量/重量比高、执行器即气囊	需泵阀气路与传感闭环	选定 ：康复手套事实标准，白牌厂商工艺成熟
SMA 形状记忆	结构极简、可直接电驱动 [5]	热惯性慢、迟滞大、效率低	放弃：分指节拍（秒级热循环）撑不住训练密度
介电弹性体	应变大、速度快	千伏级驱动电压	放弃：人体穿戴安全与绝缘成本
绳驱/腱鞘	传动成熟、电机生态好	摩擦/迟滞、非软体本体	部分竞品采用；与织物气囊工艺不亲

SMA 一支值得多说一句：Sansone 等 2022 年在 *Actuators* 上完成了 SMA 扭转执行器从概念设计到初步设计的完整方法学 [5]——这类工作证明了替代路线的成熟度，也反衬了它的边界：热驱动的高能量密度换不来快响应与低迟滞，而康复训练恰恰要求高频、可重复、可计量的节拍。气动的”短板”（需要一套泵阀传感系统）在本书语境里不是缺点，而是**产品本身**——驱动系统做对了，气动路线的全部优点都被执行器侧继承。

2.2 气动系统建模：三条方程与一个类化

2.2.1 多变气体定律：压力动力学的主方程

Xavier 等 2020 年 AIM 论文 *Modelling and Simulation of Pneumatic Sources for Soft Robotic Applications* [13] 给出了软体机器人气源建模的热力学起点：理想气体假设下，控制体内压力的一般变化由能量方程导出，对多变过程（polytropic，指数 γ 从等温 1.0 到绝热 1.4）可写成

$$\Delta P = \frac{\gamma RT}{V} (\dot{m}_{\text{in}} - \dot{m}_{\text{out}}) - \frac{\gamma P}{V} \Delta V, \quad (2.1)$$

即压力变化由**净质量流入**与**边界变形**两项竞争 [13]。若以体积流量 Q （而非质量流量）表出、腔体容积暂视作常数，就得到本书孪生每 50 ms tick 积分一次的主方程（Xavier 2022 前述综述 Eq.2/Eq.4 的形式 [12]）：

$$\frac{dP_A}{dt} = \frac{\gamma P_A}{V_A} Q, \quad Q = Q_c - Q_d. \quad (2.2)$$

多变指数怎么取？[13] 的建议是 $\gamma = 1.2$ ：充气过程偏绝热、放气偏等温，实验与仿真落在两者之间；且多数软体机器人工作在 100 kPa、10 SLM 以下，过程“更靠近等温”。这条经验值被本书孪生原样继承（第 4 章参数表 $\gamma=1.2$ ），[13] 同时报告：计入壁面换热只带来约 10% 的差异——对工程孪生，经典多变模型已是足够的精度。

2.2.2 孔口方程：流量的两个分支

压力动力学里的 Q 从哪来？对阀控系统，Xavier 2022 采用 ANSI/(NFPA) T3.21.3 阀流量模型 [12]：体积流量（L/s）随占空比线性标度、随上下游压差平方根增长：

$$Q = 114.5 u C_v \sqrt{\frac{P_{\text{low}} \Delta P}{T_{\text{high}}}}, \quad \Delta P = P_{\text{high}} - P_{\text{low}}, \quad (2.3)$$

其中压力取绝对压（bar）、温度取开尔文， C_v 为流导系数， $u \in [0, 1]$ 为 PWM 占空比——“泵/阀 PWM 线性标度流量”这条被 FlowIO 工程化 [6] 的经验律，在孔口模型里就是 u 的线性因子。把充气（上游为汇流管/储气罐 P_R ）与排气（下游为大气 P_{atm} ）两支代入式 (2.3)，整理即得孪生 v2 的充气渐近死点行为： $Q_c \propto \sqrt{P_A(P_R - P_A)}$ ，当腔压逼近气源压力时流量自然趋零 [12]。

式 (2.3) 是亚音速分支。当上下游绝对压比超过空气临界压比（约 1.893）时，孔口流速钳制在音速，流量不再随 $\sqrt{\Delta P}$ 增长——即 choked flow（阻塞流）。这正是本书孪生对 $f = P_{\text{low}} \Delta P$ 项做上下钳位的物理依据（第 4 章），其可观测签名是泄放初期降压速率近似恒定、之后才转入平方根衰减尾。选型层面，阀的流通能力以 ISO 6358 声导 C （L/(s·bar)）给出，可换算为 ANSI C_v [12]——[12] 给出的完整选型流程（目标压力/上升时间 → 储气罐容积 > 10 倍腔体 → 声导下限 → 泵流量下限）是 FLOWIO-CN 泵阀选型的模板。

2.2.3 集中参数类化：RC 电路的气动镜像

第三条方程是类化而非方程本体：压力 ↔ 电压、体积流量 ↔ 电流，气路元件即可写成 [13]

$$R = \frac{128 \mu l}{\pi d_h^4}, \quad C = \frac{V}{\beta}, \quad L = \frac{\rho l}{A}, \quad (2.4)$$

气阻（管道沿程损耗， μ 黏度、 d_h 水力直径）、气容（ V 容积、 β 体积模量；多变过程下 $\beta = \gamma P$ ）、气感（管内流体惯性）。以此类化，[13] 把“理想泵直充”“泵 + 泄漏”“泵 + 管路”“泵 + 储气罐 + 执行器”“带压力控制的全系统”五种工况逐一写成常微分方程——例如含泄漏的直充工况为 $dP_a/dt = (\gamma P_a/V_a)Q_c - (\gamma P_a^2)/(V_a R_c)$ ，泄漏项以 P^2 增长，最终压力稳定在源流量与泄漏平衡处。

同一类化的实验验证来自 Stanley 等 2021 年（2020 年在线）的 ASME 论文 [9]：他们对 100 μm –1 mm 的流阻、5–200 kPa 气源、0.5–16 mL 容积的组合系统标定了 RC 响应时间模型，63.2%（ $1 - 1/e$ ）充放气时间的预测与实测的决定系数达 $r^2 = 0.983$ ，响应时间从毫秒级到秒级全覆盖。对本书的直接意义：汇流管 + 端口气腔的每个通道都是一阶 RC 系统，闭环带宽、阀 PWM 频率、孪生积分步长的量级估算都以此为尺。三条方程汇总见表 2.2。

表 2.2: 本书孪生物理 v2 的三条方程与文献出处（详细对应见第 4 章表）

规律	采用形式	出处
压力动力学	式 (2.2), $\gamma=1.2$	[12, 13]
阀孔流量	式 (2.3) + 音速阻塞钳位	[12]
气路类化	式 (2.4), RC 响应时间 $r^2=0.983$	[9, 13]

2.3 控制器谱系：从 PID 到模型基非线性

有了模型，控制器才能“看菜下饭”。Xavier 等 2022 年在 *Frontiers in Robotics and AI* 的模型基非线性控制器研究 [12] 把单阀系统的压力动力学整理为控制仿射形式 $\dot{x} = f(x) + g(x)u$ (x 为腔压、 u 为 40 Hz PWM 占空比)，并系统比较了谱系上的四档策略（表 2.3）：实验整定的 PID/PI；状态依赖 Riccati 方程（SDRE）控制；滑模控制（SMC）与反馈线性化（FL）两支模型基非线性控制，以及为抗模型不确定性给二者附加的积分作用；最后是模型前馈 + 抗饱和 PI 的组合——用稳态占空比的前馈解 u_{ff} （令 $\dot{x} = 0$ 反解）抵消大部分非线性，PI 只收残差 [12]。

表 2.3: 气动软体执行器压力控制器谱系（据 [12] 整理）

档位	策略	特征	本书采用
经典	PID/PI + 抗饱和	免模型、整定靠实验	采用：pn_core 分指闭环（第 9 章）
前馈增强	$u_{ff} + \text{PI}$	模型抵扣非线性，PI 收残差	候选特性（泵 PID 连续流量同类思路）
模型基	SDRE / SMC / FL	需要可信模型与全状态	暂不引入：参数成色不足
学习型	持续学习/自适应	数据驱动，样本昂贵	不引入（同第 1 章裁决）

本书的选择是谱系的“保守端”：在阀线圈与泵参数仍是手册值（黄徽章）的阶段，模型基控制器的精度承诺兑现不了，PI + 抗饱和反而是成色最诚实的选择——这与第 1 章“按服务倒推技术”的原则一致：等孪生对拍把参数洗成实测值，前馈增强才有立足之地。

2.4 交互装置脉络：PneUI → OmniFiber → FlowIO

理论之外，本书的另一条血脉是 MIT 媒体实验室的交互装置谱系。2013 年的 PneUI [15] 把“气动驱动 + 多层软复合材料”引入界面：复合材料同时集成输入传感与形变输出，会变形的手机、可变高度的实体图标（phicon）、可变身平板套与变形台灯四个应用，确立了“压力 → 形变”作为可编程物质的交互原语 [15]。执行器形态此后持续演化：2021 年的 OmniFiber [2] 把流体驱动做进细径纤维——外径毫米级的薄壁纤维内嵌流体通道，可编织、可缠绕，钢琴教学辅具、呼吸感知胸衣、动力首饰等应用展示了“驱动器即材料”的可能性；其驱动硬件正是 FlowIO（合作者含 Shtarbanov）[2]。

执行器越千变万化，驱动器越需要标准化——这是 FlowIO 的诞生逻辑。Shtarbanov 2021 年的 CHI 论文 [6] 把它做成五端口、模块化（S/M/L 泵模块磁吸互换）、全集成（泵阀传感电池一体，主模块 77 g）、带完整软件栈（Arduino/JavaScript/Web GUI）的可穿戴气动开发平台，“软体机器人的树莓派”；其 2025 年博士论文 [7] 补齐了各泵模块的实测锚点（Small 模块 $-38 \sim +61$ kPa、 ± 0.3 L/min）——这组数字后来成为本书孪生参数表的初值（第 4 章）。FlowIO 的动作语义（充气/抽真空/泄放/保压/测压/流量可变）与 PWM 调流量经验律 [6]，被 FLOWIO-CN 的 CLI 动词与孔口乘子逐条继承（第 3 章对照表）。

2.5 康复手套前沿：便携拇指手套与织物综述

本书目标市场的两条最新文献边界，勾勒出“学术前沿走到哪、空白在哪”。Xie 等 2026 年发表于 *Advanced Science* 的便携式软体手套 [14] 是拇指全功能辅助的代表作：连续分段双腔折纸（origami）拇指执行器覆盖腕掌外展/内收、屈伸与指间伸展全部功能范围，指尖力 18 N（约为克服典型中风痉挛所需的两倍）；系统集成传感化的便携驱动与“按需辅助”（assist-as-needed）控制器，三名慢性期卒中受试者完成捏取、侧捏、绕环、滑屏、拧

盖与拇指对掌全部任务，侧捏力提升 62.9%、拇指活动范围扩大九倍 [14]。它验证了三件事：**拇指通道必须全功能**（与本书分指/分腔路线互证）、**驱动必须便携且带传感**（开环不可接受）、国产压力传感器（XGZP 同族）已进入顶刊方案。

Yilmaz 等 2026 年在 *Advanced Materials Technologies* 的织物外骨骼手套综述 [16] 则盘点行业现状：制造家族、驱动机制、传感集成、控制架构与能源五个维度的进展与短板，并直指两个空白——**开环控制对康复手套根本不够**，以及**行业缺乏统一测试标准** [16]。前者是本书“分指压力闭环一等公民”的文献判词，后者是“李生对拍精度报告 + 36 张验收工单”的市场机会（第 14、13 章）。

2.6 本书定位：在谱系坐标上

把本章三段谱系放进同一张表（表 2.4），FLOWIO-CN 的位置一目了然：它不做执行器（织物与折纸气囊留给白牌厂商的传统强项），不做新材料驱动（SMA/纤维/DE 均非本书战场），而是把气动路线的**驱动与控制层**做成 B2B 产品——以本章三条方程为李生物理，以谱系保守端的 PI 闭环为分指节拍，以 FlowIO 的语义为兼容接口，以两篇 2026 综述指出的空白（闭环、标准）为差异化卖点。

表 2.4: 本书与本章文献谱系的定位对照

工作	定位	与 FLOWIO-CN 的关系
PneUI [15] / OmniFiber [2]	交互原语 / 驱动材料	需求来源：执行器谱系越丰富，标准驱动器越稀缺
FlowIO [6, 7]	可穿戴开发平台	对标本体：语义与参数锚点的直接来源
Xavier 系列 [12, 13] / Stanley [9]	建模与控制方法	李生物理 v2 的方程供应链
Xie 2026 [14]	顶刊拇指手套	路线互证：分指/传感/便携三项一致
Yilmaz 2026 [16]	行业综述	空白判词：闭环与测试标准即卖点
本书 (FLOWIO-CN)	分指压力闭环 B2B 驱动套件	以上文献的工程收口

本章小结

气动在软体驱动谱系中以“快响应、高力量重量比、执行器即气囊”胜出，其代价——一套泵阀传感系统——正是本书的产品本体 [5, 12]。系统建模有三条公共方程：多变定律式 (2.2) ($\gamma=1.2$)、ANSI 孔口式 (2.3)（含音速阻塞分支）与 RC 类化式 (2.4) [9, 12, 13]；控制器谱系从实验整定 PI 到模型基非线性依次升高，本书按参数成色选保守端 [12]。装置脉络 PneUI→OmniFiber→FlowIO 证明驱动器标准化是执行器多样性的必然反题 [2, 6, 15]；康复前沿的拇指手套与织物综述则把“分指闭环 + 测试标准”标成行业空白 [14, 16]。下一章开始，这些坐标全部变成工程图纸。

第三章 软体机器人与 FLOWIO-CN 生态位

如果把一台气动康复手套拆开，你会看到三层东西：最里面是硅胶或织物制成的**软体执行器**——气囊、气腔、编织层；中间是**气路**——气管、快插接头、汇流管；最外面才是本书的主角——**驱动系统**：泵、阀、传感器、主控板、固件与上位机。三层里最不起眼的是最后一层，但文献与产业报告反复指出同一个事实：它消耗了软体机器人项目最多的工程时间。MIT 媒体实验室的 FlowIO 论文开篇即写道，气动项目的研究者“通常必须从零开始自建驱动系统，有时甚至要把大部分时间花在气动、电子与软件上，而不是开发新颖的软体执行器、应用或用户体验”。

本书是一本“自研驱动系统”的完整实录。我们以 MIT FlowIO 为对标，面向国内康复手套白牌厂商设计了 FLOWIO-CN——一套定价 300–500 元的 B2B 套件（主板 + 固件 + 上位机 SDK），并把从原理图、PCB、制造下单、参数化外壳，到固件、数字孪生、BLE、验收工单的全过程写成十二章。本章先回答两个问题：软体机器人到底需要一台什么样的驱动设备；FLOWIO-CN 凭什么在已有开源平台的世界里再做一个。

3.1 软体驱动的前世今生：三条主线

气动软体机器人不是一个领域，而是三条需求差异极大的主线在共用同一套物理（压缩气体 × 可变形腔体）。

主线一：人机界面与可编程材料（HCI）。2013 年 MIT 的 PneuUI（UIST’13，被引数以百计）把“气动驱动 + 软复合材料”引入界面设计：多层复合材料集成传感与形变输出，高度可变的实体图标（phicon）、会变形的手机、可变身平板套、变形台灯四个应用展示了“压力 → 形变”的连续映射。这条线后来长出了 aeroMorph、MorphIO、Bubble、AuxeticBreath 等一系列工作——它们的共同点是：执行器千变万化，驱动器最好是一台小巧、可穿戴、能编程的“气源 + 阀 + 传感”黑盒。FlowIO 正是 Bubble 项目（2018–2019）找不到这样一台五端口微型驱动器，逼出来的产物——作者 Shtarbanov 为此做了两年、超过 20 轮原型迭代。同一条线上还有两个值得一提的近亲：OmniFiber（UIST’21，Shtarbanov 参与合作）把流体驱动做进了绕在轴上的纤维，PneuKnit（MIT SMarChS 2022）用气压让针织物自成形——执行器的形态在从“气囊”向“材料本身”演化，驱动器反而越来越需要标准化。

主线二的旁支：教育套件。PneuBots（TEI’22，哈佛 GSD）把模块化充气执行器做成可拼插的玩具形态，用于软体机器人的公众教育。教育场景对驱动器的要求与创客几乎相同：便宜、安全、图形化编程。FLOWIO-CN 的文献台账把 PneuBots 标注为“教育定位直接对标”——虽然我们的主战场是 B2B 康复，但同一套硬件天然覆盖高校实验室与创客教育的第二市场，这也影响了开放 API 的设计优先级。

主线二：抓取与通用软体机械臂（机器人学）。气动欠驱动夹爪利用软体材料的顺应性抓取异形易碎物体，研究重心在执行器的材料、腔体拓扑与建模（集中参数电路、RC 类比、非线性控制器）。这条线对驱动系统的要求是**压力包线与流量**：更高压力、更大流量、更快的动态响应。我们在文献库里精读的 Xavier 2022（Frontiers in Robotics and AI）与 Stanley & Amini 2020（ASME 集中参数气路）都出自这条线——它们后来成为 FLOWIO-CN 数字孪生物理引擎的方程来源（第 4 章）。

主线三：康复与助力手套（可穿戴医疗）。这是 FLOWIO-CN 的目标市场，也是近年文献最密集的方向。几篇关键文献勾勒出技术底线：

- **压力量级**：手部康复手套的工作压强大约在数十 kPa——Chen 2026（arXiv，Sant’Anna）的 CSP 手套对患者测试区间为 50–70 kPa；我们的孪生参数表也以此校准量程。
- **拇指通道数**：Wang 2022（Applied Sciences，千叶大学）用对照实验裁决了“拇指要不要独立通道”——三腔独立驱动（3C-ACT）组 Kapandji 拇指功能评分满分，模块化耦合驱动（M-ACT）组仅 0–5 分；最优组合是“3C 拇指 + 模块化四指”。结论：**分指/分腔独立控制不是锦上添花，而是功能性的分水岭。**
- **传感方案**：Xie 2026（Advanced Science，港中大汤启宇组）的便携拇指助力手套用了 XGZP6857 系列压力传感器——与 FLOWIO-CN BOM 中的 XGZP6897D 同族；这说明国产传感器方案已进入顶刊工作。
- **行业现状**：Yilmaz 2026 织物手套综述（Advanced Materials Technologies）盘点五大制造家族后直言“开环控制对康复手套根本不够”，且行业缺乏统一测试标准——前者正是我们的卖点（分指压力闭环），后者是

数据报表的机会。

- **免费竞品：**郑州大学专利 CN108392375A（气动康复手套）公开了“双泵 + 双作用 + 12 传感器 + 迭代学习控制”的完整方案——一份免费拆机对象，其架构（多阀多传感 + 闭环）印证了 B2B 客户的真实需求。

三条主线对驱动系统的交集可以概括成一张“最小系统”清单：**气源（泵）+ 方向控制（阀）+ 反馈（压力传感）+ 大脑（MCU/SoC）+ 通信（USB/BLE）+ 外壳**。差异只在量级与侧重——HCI 要小要美，机器人学要压力流量，康复要闭环与分指。表 3.1 总结三者的取向。

表 3.1: 气动软体机器人三条主线对驱动系统的不同要求

主线	代表工作	对驱动系统的核心要求	典型压力
HCI 界面	PneUI、Bubble、FlowIO 应用生态	微型、可穿戴、低功耗、易编程（GUI/API）	<30 psi 级
抓取/机械臂	Xavier 2020/2022、Stanley 2020	压力包线、流量、动态建模与非线性控制	数百 kPa 级
康复手套	Wang 2022、Xie 2026、Chen 2026、CN108392375A	分指闭环、多传感、数据记录、成本	50–70 kPa

3.2 平台化的呼唤：气动工具包谱系

电子原型有 Arduino，便携计算有树莓派，编程入门有 Scratch——平台把一个个“高门槛领域”变成了周末项目。软体机器人在 2020 年前后仍在等自己的平台。FlowIO 官网关于页梳理了此前三次尝试（表 3.2）：Soft Robotics Toolkit（SRT）是散件 DIY 方案；Pneuduino 有模块化硬件与 Arduino 库，但仍需外接泵与电源；Programmable Air 把泵集成进设备、用料更便宜，但基础版只有单个气动端口，仍依赖外部大电流电源，泵也不易更换。三者共同的问题：体积大、传感能力有限、缺 GUI、只兼容 Arduino。

表 3.2: 气动软体工具包谱系（依据 FlowIO 官网与 CHI'21 论文整理）

平台	形态	主要短板	FLOWIO 的回应
SRT	散件 DIY	体积大、外部气源/电源	全集成微型整机
Pneuduino Programmable Air	模块化硬件 + Arduino 库 集成泵、低价	外接泵与电源 单端口、外部大电流电源	泵模块内置 5 端口、可换泵模块
FlowIO（CHI'21）	主模块 + 泵模块（磁吸）	见 §3.4	——（对标本体）

FlowIO 对这些短板的回应构成了它的设计基因：**模块化**（主模块与 S/M/L 三档泵模块磁吸互换，几秒钟按压力/流量/尺寸需求重组系统）、**全集成**（泵、阀、传感、电池、充电全在机上）、**软件栈**（这是作者自认“最重要的特征”——网页 GUI 经 Chrome Web Bluetooth 零安装控制、JavaScript API、串口命令、Arduino API 多层抽象）。FlowIO 主模块仅重 77 g，平台包线 $-26 \sim +30$ psi（约 $-179 \sim +207$ kPa）、流量至 3.2 L/min。它是“软体机器人的树莓派”这个比喻的出处，也是我们一切对标的锚点。

平台技术的另一半生命在社区。CHI 2022 上 Shtarbanov 与 RWTH、康奈尔、KTH 等机构的合作者组织了“软体机器人 for HCI”工作坊——一天议程，三十个名额，线上下混合；议题从交互、界面、制造工艺一路排到包容性设计与商业化机会，现场备有多台 FlowIO 与软体机器人入门套件供参与者上手。工作坊收录的短文阵容（气动纹理、模块化充气玩具 PneuBots、织物到软体机器的实践路径……）勾勒出这个社区的用户画像：他们要的不是更强的执行器，而是**更低的驱动门槛**。FLOWIO-CN 选择兼容这个社区的语言（表 3.3），本质上是在为同一群人的国内版本做准备。

3.3 MIT FlowIO: 架构解剖

对标必须对到细节。按 CHI'21 论文的硬件构成一节，FlowIO 主模块（控制器）的气动-电子架构如下：

- **阀拓扑**：7 只常闭电磁阀组成汇流管（manifold）构型——2 只分别作进气与排气，5 只对应 5 个气动 I/O 端口。每端口支持充气、抽真空、泄放至大气、保压、测压与流量可变六种动作。常闭阀的默认态即“保压”：断电时各腔体各自密封，这一语义我们原样继承进 FLOWIO-CN 固件。
- **传感**：单只压力传感器直连汇流管，通过阀的时分复用（time-multiplexing）轮流测任一通道压力——想测哪个口，先开哪个口的阀。
- **主控**：Adafruit Feather Sense nRF52840（BLE + IMU/气压/光感等板载传感），500 mAh 锂电藏于阀阵之下，USB 充电。
- **泵模块**：S/M/L 三档，区别在泵与电池选型；两泵 + 驱动板 + 电池，磁吸四针连接主模块。Shtarbanov 2025 年的博士论文（303 页）给出了各档的实测锚点——Small 模块 $-38 \sim +61$ kPa、 ± 0.3 L/min（论文 Table 2/3），这组数字后来直接成为我们孪生参数表的初值来源（第 4 章）。
- **配置生态**：五种标准气动配置（各端口/泵/阀的组合接法）覆盖充气、真空、双作用等典型工况；论文还收录了 12 个部署案例，从可穿戴到桌面应用——一个平台自我证明的方式，是别人拿它做了多少种东西。
- **调速**：泵 PWM 调流量、阀 PWM 调流量（论文 §4），占空比 $u \in [0, 1]$ 线性标度流量——这条经验律后来被我们孪生的孔口方程继承为流导系数的乘子。

软件栈自底向上：固件层（稳定）、Arduino 库与 API（基本完备）、JavaScript API（演进中）、Web GUI（Chrome）。控制方式四种：网页 GUI、JS API over BLE、USB 串口命令、自写固件。这套“一份硬件、多层软件入口”的分层思想，直接映射为 FLOWIO-CN 的固件（pn_core）/串口协议/上位机 SDK/Web GUI 四层。

FLOWIO-CN 继承的不只是分层，还有**动作语义**。FlowIO 为每个端口定义了六种动作：充气、抽真空、泄放、保压、测压、流量可变——这组动词是五年用户教育沉淀下来的公共词汇，白牌厂商的工程师大概率已经在 FlowIO 生态里见过它们。P1 的 CLI 命令动词与之逐一对应（表 3.3），客户从 FlowIO 迁移的认知成本接近零；命令文法与 BLE 载荷共用同一份协议定义（第 10 章），这是“开放 API”差异化的具体形态。

表 3.3: FlowIO 动作语义与 FLOWIO-CN P1 的 CLI 动词对应

FlowIO 语义	P1 CLI	说明
inflation	I < 口 > < 占空比 >	如 I 1 255: 1 号端口全速充气
vacuum	V	抽真空（泵 + 排气阀组合）
release	R / S	R 释放但不清占空比；S < 口 > 确定性关阀（隔离密封）
pressure hold	H < 口 >	泵侧密封 + 端口保持通——保压/泄漏诊断标准工况
pressure sense	传感遥测	6 只传感器并行回读（/api/state、BLE notify）
流量可变	占空比参数	泵/阀 PWM，0 ~ 255 线性标度

3.4 FlowIO 的边界，就是 FLOWIO-CN 的切入口

FlowIO 官网的 Limitations 页坦率列出了平台仍未解决的问题。对一家想拿它做产品的 B2B 厂商来说，这张清单比论文更有价值——每一条都是差异化的机会：

- **分时测压**：单传感器时分复用意味着五口压力不可同时测量，且测压必须开阀（开阀本身就是一次扰动）。官方的补丁是另购 Sensors++ 模块（每端口一只专用传感器）。——FLOWIO-CN 把“每口一只传感器”直接做进主板：汇流管 1 只 + 端口侧 5 只 XGZP6897D，经 TCA9548A 五通道 I2C 复用器接入，恰是“Sensors++ 内置化”。

- **同刻只能同种动作**：可以同时多口充气，但不能口 1 充气同时口 2 抽真空。官方解法是买两台设备同步控制。——分指康复恰恰需要异口异动作；FLOWIO-CN 的 8 路阀（7 阀 + 1 泵）由 8 只独立 AO3400 MOSFET 直驱，阀位图与泵向自由组合。
- **低层反馈闭环未完成**：官方明言“跑在 MCU 上的反馈控制环”属于待实现特性（部分实验性功能仅在付费 premium 内容中开放）。——FLOWIO-CN 以分指压力闭环为一等公民（第 9 章）。
- **可穿戴优先的权衡**：wearable-first 意味着压力/流量让位于体积与功耗；且 FlowIO 定位是“演进中的开源项目”而非产品。——B2B 客户需要的是交付形态：可代工制造、可验收、带精度报告的整机方案。

3.5 FLOWIO-CN 的差异化定位

产品定位在项目规格书 §1 以一张表锁定（表 3.4）。客户是**康复手套白牌厂商**：他们有执行器与织物工艺、有渠道，缺的是一颗可靠的“大脑”与配套软件；售价 300–500 元/套（主板 + 固件 + 上位机 SDK），对标 MIT FlowIO，差异化四件套为分指压力闭环、数据报表、国产 BOM、开放 API。

表 3.4: FLOWIO-CN P1 产品定位（规格书 §1 原表）

项	值
名称	FLOWIO-CN P1「升级大脑」
客户	康复手套白牌厂商（B2B）
售价	300–500 元/套（含此板 + 固件 + 上位机 SDK）
对标	MIT FlowIO（7 常闭阀、传感、开源 API）
差异化	分指压力闭环 + 数据报表 + 国产 BOM + 开放 API

P1 板（顶/底面渲染见图 6.1 与图 6.2）把定位落成硬件：ESP32-S3-WROOM-1-N16R8 主控（BLE + WiFi，国产渠道现货）；8 路 AO3400A 直驱 7 阀 + 1 泵；TCA9548A 挂 6 只 XGZP6897D（汇流管 1 + 端口 5）实现全通道并行测压；双输入电源（DC 5V/3A 主供电 + USB-C 逻辑供电，SS34 二极管或门）；90×75 mm 四层板。表 3.5 逐项对比两代平台。

表 3.5: FlowIO 主模块与 FLOWIO-CN P1 对比

维度	MIT FlowIO（CHI'21）	FLOWIO-CN P1
定位	开发平台（演进中开源项目）	B2B 产品套件
主控	nRF52840（Feather Sense）	ESP32-S3-WROOM-1
阀	7 常闭（2 进/排 + 5 端口）	7 常闭 + 泵直驱，共 8 路 MOSFET
测压	单传感器分时（Sensors++ 另购）	6 传感器并行（TCA9548A 五通道）
异口异动作	不支持	支持（阀位图自由组合）
闭环	低层反馈环待实现	分指压力闭环一等公民
供电	锂电 + USB（可穿戴）	DC 5V/3A + USB-C 双输入
软件栈	Web GUI / JS / 串口 / Arduino	固件 pn_core / 串口 0xA5 / BLE GATT / Python SDK / Web 孪生
BOM	海外渠道	国产 BOM，全代工可造

差异化不是口号，每一条都有文献或工程依据：分指闭环对应 Wang 2022 的三腔独立裁决与 Yilmaz 综述“开环不够”的判词；每口传感对应 FlowIO 单传感器分时的官方短板；数据报表对应行业无统一测试标准的空白（孪生对拍精度报告，第 14 章）；开放 API 对应白牌厂商二次开发的真实工作流（第 12 章）。国产 BOM 则是供应链的现实选择——也正是它把整板物料成本压进了 300–500 元的售价区间（第 7 章的 5 片打样物料仅约 150–300 元）。

3.6 从文献差距到产品需求

差异化定位不能停在口号，要能追溯到”哪篇文献的哪个结论、落实为板上哪颗料”。表 3.6 是这条追溯链的全貌——它也是项目规格书的生成方式：每条硬件决策都能向左回答”为什么”、向右回答”怎么做”。

表 3.6: 文献结论 → 产品需求 → P1 硬件落点

文献结论	产品需求	P1 落点
Wang 2022: 三腔独立 Kapandji 满分，耦合组 0-5 分	分指独立控制	8 路 AO3400A 直驱
Yilmaz 2026: ”开环对康复手套根本不够”	板载压力闭环	阀位图自由组合 pn_core 闭环节拍 + 章)
FlowIO 官方: 单传感器分时测压的短板	并行测压	TCA9548A 五通道 XGZP6897D
Chen 2026 / Xie 2026: 患者工作压 50-70 kPa; XGZP 同族传感进顶刊	量程与传感选型	±100 kPa 差压量程， 道
Shtarbanov 论文 Table 2: Small 泵 +61 kPa 死点 Yilmaz 2026: 行业无统一测试标准	孪生参数锚点 评估工具即卖点	充气渐近 61 kPa (第 36 张验收工单 + 孪 报告 (第 13、14 章 多阀多传感 + 闭环 一致
专利 CN108392375A: 双泵 + 12 传感 + 迭代学习	竞品架构印证	

这张表还回答了一个潜在质疑: 为什么不直接买 FlowIO? 除了海外渠道与价格，根本原因是表中第二列——FlowIO 的硬件形态 (单传感器分时、同刻同动作) 由它的可穿戴优先定位决定，改不动；而这些恰是康复分指闭环的硬前提。复刻一个对标物的短板没有意义，继承它的语义、修掉它的约束，才是”对标”的完整含义。

3.7 被引数据里的市场信号

”国产替代”是不是一厢情愿？项目用数据回答了这个问题：经 Semantic Scholar API 拉取 FlowIO (CHI’21) 的全部 83 篇引用论文，再以拼音姓氏启发式初筛、AMiner 逐篇核查作者机构——结果是 42 篇 (51%) 含中国作者，且逐年引用量 (2021-2026: 3、15、16、15、18、11 篇进行中) 持续增长无衰减。中国是 FlowIO 最大的用户群体，这个判断被引文数据坐实；海外渠道的价格与交期，恰好是这个用户群最敏感的痛点。

机构核查还产出了一份具体的国内客户地图：浙江大学圈 (气动形变界面与 STEAM 教育，四篇引用，其中 KiPneu 被项目标注为”教育版 FlowIO”)、清华大学三个组 (柔性液压皮肤、医疗压力源、静音阀)、南方科技大学与深圳技术大学 (iSoRD 可穿戴驱动)、深圳大学 (EtherCAT 组网混合阀) 等。这份清单的价值不在名单本身，而在它揭示的需求结构：国内最活跃的气动 HCI 团队与医疗压力源团队，要的是 FlowIO 形态的设备——而他们现在只能海外采购。

同一轮调研还抓出了直接对标的国产竞品规格锚点 (表 3.7)。值得注意的是南科大 iSoRD: 单泵 + 四位二通阀的架构、-53 ~ +83 kPa 的包线、PID 连续流量——与 FLOWIO-CN 同赛道，其”连续流量靠泵 PID”的思路后来也被记为 P1 固件的候选特性。竞品不是坏消息：它证明赛道真实存在。

表 3.7: 直接对标产品规格锚点 (学术地图调研, 2026-09)

产品	关键规格	对 P1 的启示
FlowIO 本尊	-179 ~ +207 kPa (模块包线) / 3.2 L/min	平台上限; P1 定位数十 kPa 康复档差异化
iSoRD (南科大, RA-L’25)	-53 ~ +83 kPa; PID 连续流量 15 mL/s; 2.95 W/通道	同赛道印证; 泵 PID 思路入固件候选
清华蠕动泵压力源 (RA-L’24)	掌上、±50 kPa、MRAC 控制误差 <0.85 kPa	医疗级精度标杆, 闭环对拍目标

3.8 康复手套的工程解剖：客户到底在买什么

理解 B2B 定位，最好把白牌厂商的整机拆开看。一只气压康复手套从内到外大致四层，每层的成熟度与利润结构都不同：

- **执行器层**：硅胶气囊或织物气腔，热封或模压成形。Yilmaz 2026 综述把这一层的制造归为五大家族——这是白牌厂商真正的传统强项，也是他们不需要我们的部分。Chen 2026 甚至展示了用改装 FDM 打印机做 CNC 热封的桌面级方案，整只执行器手环仅 90 g。
- **传感层**：腔压传感（XGZP 系列）、关节测角（Xie 2026 用 IMU 测拇指角度）、肌电/肌腱刺激触发（Chen 2026）。这一层决定“知不知道手在干什么”。
- **驱动层（本书）**：泵 + 阀 + 压力闭环 + 主控 + 无线。Xie 2026 与 Chen 2026 都把“控制箱”列为便携化的主要瓶颈——Chen 的方案把 ESP32、12 只电磁阀、双作用泵和浏览器 GUI 塞进一只手腕盒，架构与 FLOWIO-CN 同构（ESP32 + 浏览器 GUI + 多电磁阀），这在文献上印证了我们的技术路线不是孤例。
- **软件层**：处方设定、疗程记录、数据报表。学术工作止步于控制算法（Xie 的贝叶斯 AAN 控制、专利 CN108392375A 的迭代学习控制），但把算法变成产线可复制的商品需要的是工程化——这正是“大脑 + 固件 + SDK”三件套的含义。

从这个解剖看定价就清楚了。套件 300–500 元覆盖“板 + 固件 + 上位机 SDK”。以第 7 章的打样数据粗算：5 片打样物料 150–300 元，单板物料约 30–60 元；标准版贴装的工程费、钢网、上料（300–600 元）是一次性投入，随批量摊薄。也就是说，批量单板的硬件成本远低于套件定价——差额买的是固件、SDK、验收体系与精度报告这些“软件密度”高的部分。这与 Yilmaz 综述点出的行业空白（无统一测试标准 → 评估能力即卖点）是同一件事的两面：硬件趋同的行业里，可验证的软件与数据能力才是利润所在。

3.9 全栈自研的产品思维

FLOWIO-CN 的第二个差异化在方法论：**全栈联合设计**。传统白牌流程里，电路外包给方案公司、结构外包给模具厂、软件外包给团队——三者以“甲方需求文档”为界面，扯皮发生在界面上。本书的流水线把四条腿收进同一个仓库、同一套规格书体系：

- **电路（KiCad）**：原理图与 PCB 由 `gen_sch.py/gen_pcb.py` 代码生成，选型修正以编号决策记录（第 5 章）；
- **结构（FreeCAD）**：外壳参数直接读取 PCB 板框与连接器坐标，改板不必重画壳（第 8 章）；
- **软件（ESP-IDF / Python / Web）**：固件逻辑层与数字孪生共用同一份 `pn_core` 代码（第 4、9 章）；
- **调试**：测试点 TP1–TP12 上板、仿真四电路先行、逻辑分析仪/示波器动线写进验收工单（第 13 章）。

这套流程的地基是一份 2026-09-22 制定的项目宪法（宪法.md）——全部条文由项目实践沉淀，每条注明出处事故。与本书各章关系最密的几条：**孪生物理必须真实**（禁止硬编码轨迹或查表模拟，物理模型必须来自文献加实验校准——第 4 章的立法依据）；**先读后写**（任何规格书制定前必须先读文献原文，引用必须落到本地 PDF）；**双宿主单一逻辑源**（`pn_core` 是唯一控制逻辑，Windows 孪生 DLL 与 ESP32 固件共用同一份代码，任何改动两侧同验）；**验收即证据**（宣称“完成”必须附带刚运行的证据——测试输出、编译退出码、冒烟日志，不允许凭记忆宣称）。

宪法同样规定了**不做什么**，且与卖点同等具体（表 3.8）。“四不做”背后是同一条逻辑：本项目由独立开发者全职启动、后转业余推进，财务与人力都经不起战线扩张——每一条“不做”都对应一种真实的诱惑与它的代价。其中“69 元整机价格战”一行值得展开：康复手套市场确有超低价整机在走量，但那是一条**执行器成本 + 注册成本 + 渠道成本**三重挤压的路，与“卖大脑赚软件密度”的定位正交；主动放弃它，才守得住 300–500 元的部件定价与毛利结构。

表 3.9 是全书路线图：十二章对应这条流水线的十二站，每站有 git 提交与可复现产物。

表 3.8: 宪法”四不做”清单（宪法.md 第一章）

不做	理由
69 元整机价格战	与”部件供应 + 软件密度”的毛利结构正交；执行器/注册/渠道三重成本挤压
二类医疗器械注册（现阶段）	注册周期与成本吞没独立开发者；白牌厂商自有注册路径
通用平台叙事	FlowIO 的定位空档恰是专用闭环；通用 = 与本尊正面竞争
BCI（脑机接口）	技术栈与康复手套无重叠；纯粹的战线扩张

表 3.9: 本书十二章地图

章	主题	核心产物
1	生态位与产品定位（本章）	差异化论证
2	数字孪生物理建模	孪生引擎 + 校准矩阵
3	原理图	13 次编号修正的教训
4	PCB 布线	146→0 未连接攻坚
5	制造（嘉立创全代工）	可直接下单的 fab/ 资产包
6	参数化外壳	FCStd/STEP/STL + 孪生网格
7	固件	pn_core + QEMU 冒烟
8	BLE	自研 GATT 四服务
9	上位机与孪生前端	Web GUI v2
10	SDK	flowio_sdk (Python)
11	验收	36 张工单机制
12	产品化	标定、精度报告、路线图

3.10 文献工作流：结论是怎么来的

本书引用的每篇文献都经过了同一套入库流程——它本身值得记录，因为”结论可信”的前提是”文献管理不出错”。项目文献库（`literature/`）的结构是两层：根目录 15 篇精读核心（平台与物理 9 篇 + 精细动作控制 6 篇），`citers/` 子目录 81 篇 FlowIO 被引文献（Semantic Scholar 元数据对账而来）。

入库的验证流程四步定型：`pypdf` 抽页数与首页文本 → 标题/作者/DOI 与检索清单比对 → 统一重命名 `YYYY-作者姓-venue-slug` → 全文提取 `.txt` 伴生文件（图片扫描件除外）。四步挡住的都是真实发生过的坑：远端静默截断的 PDF（无 EOF 标记需续传）、数据库 DOI 记错（Xavier 2022 的正确 DOI 是 10.3389/frobt.2022.818187）、以及下载到不相关的论文——文献库里至今留着一篇误下载的 SMA 扭转执行器论文（本想下拇指三腔对比），标注”与产品无关”存档而不是删除。错误留档是这套流程的底色：它让每一次勘误都可追溯，也让后来者不会重复同一个坑。

这套流程的产出直接服务于工程：第 4 章孪生物理的每条方程都能在文献库指出对应的 PDF 与章节；泵参数锚点（+61 kPa）标注着”博士论文 Table 2”的出处。文献库不是装饰，是孪生模型的参数供应链。

3.11 如何读这本书

本书十二章按开发流水线排列，但三类读者可以各取捷径。创客读者关心”我能不能照着做一台”：直接从第 5 章的原理图与第 7 章的下单包入手——fab/ 目录的资产包可以原样上传嘉立创，绕过电路设计的学习曲线先把板拿到手。嵌入式工程师关心固件与协议：第 4（孪生物理）、9（pn_core）、10（GATT 契约）、12（上位机）四章是一条完整的”逻辑层单源、多端复用”参考实现。产品/厂商读者关心的是风险与交付：第 13 章的 36 张验收工单与第 14 章的精度报告路线图，回答”这东西怎么证明它能用”。所有章节共享同一套纪律：决策有编号、修正有记录、每个数字有出处——这也是全书唯一要求读者带走的”方法论装备”。

本章小结

软体机器人三条主线汇流到同一张驱动系统最小清单；FlowIO 用模块化与软件栈把它做成了”软体机器人的树莓派”，但其分时测压、同刻同动作、闭环缺位与”项目而非产品”的形态，恰是 B2B 康复市场的切入点。FLOWIO-CN 以分指压力闭环、每口传感、数据报表与开放 API 立位，用国产 BOM 把价格压进 300-500 元；每条差异化都能追溯到具体的文献结论，并落实为板上具体的料。接下来十一章，是这套论证的工程兑现。

第四章 孪生先行：物理建模与仿真

PCB 下单到回板之间有两三周的空窗。传统流程里这段时间只能等；FLOWIO-CN 的流程里，这段时间被数字孪生填满：firmware/twin/ 目录下的板级物理模型（twin_api.c 气动 + board_model.py 电气）让固件命令、遥测面板、BLE 契约、上位机 SDK 在真实硬件到来之前就全链路跑通。回板那天，孪生不是被抛弃的脚手架，而是与真机对拍的标尺——两者的偏差本身就是产品交付物（第 14 章的”数字孪生精度报告”）。

本章讲三件事：孪生的架构为什么必须”逻辑与物理分离”；气动物理如何从工程近似（v1）升级为文献形式（v2/v2.1）；以及如何诚实地管理模型参数的成色——”理论值”与”实测值”用徽章区分，不让演示数字冒充实测。

4.1 逻辑与物理分离：一份 pn_core，两个世界

孪生的第一条架构原则：逻辑层与物理层彻底分离（图 4.1）。逻辑层是 pn_core——命令分发、状态机、闭环控制、调度器，它与烧进 ESP32-S3 的固件是同一份代码；物理层是 mock_hal 之后的仿真世界：阀位图不再驱动真实线圈，而是驱动一套气动微分方程，压力传感器的读数不再来自 XGZP6897D，而是方程的积分值。

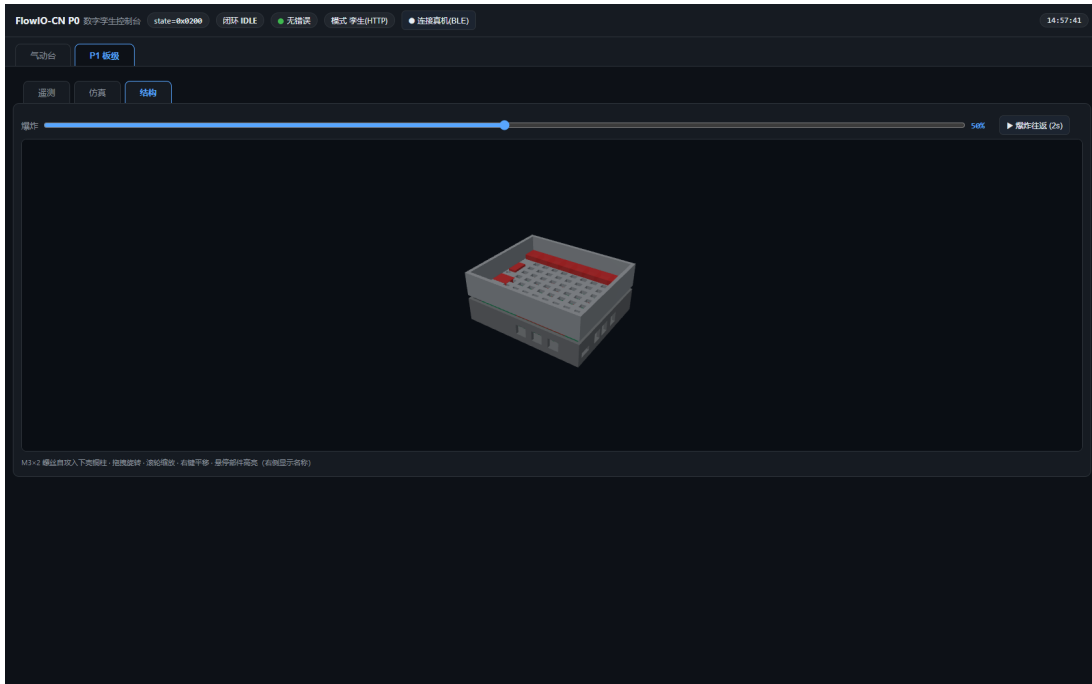


图 4.1: 孪生控制台的 3D 装配视图：五部件（上壳/器件阵/PCB/下壳）可整体查看亦可分解——孪生不仅算得出压力，也看得见结构

这样做的收益是结构性的。固件在 QEMU 或主机上冒烟通过，意味着回板后烧进去的逻辑大概率没有低级 bug；孪生编译成 pn_twin.dll (C ABI) 交给上位机 server.py 加载，前端遥测面板、Web GUI、SDK 的每个接口在回板前就有”会动的假硬件”可调；而闭环算法的调参，在孪生上先跑通一轮，真机只做最后确认。twin_api.c 因此只是一层薄壳：初始化、命令注入、50ms 节拍推进、传感器读出：

Listing 4.1: twin_api.c 的公共接口：逻辑世界与物理世界的全部通道（摘自源码）

```
PN_TWIN_API void pn_twin_init(void); /* 虚拟上电: pn_core 复位 + 泄漏/端口节点清零 */
PN_TWIN_API void pn_twin_command(const char *line); /* CLI 命令，与真机串口同一文法 */
PN_TWIN_API int pn_twin_tick(void); /* 推进 50ms: 物理积分 + 闭环节拍 */
PN_TWIN_API float pn_twin_sensor(uint8_t idx); /* 汇流管压力 kPa */
```

```
PN_TWIN_API float pn_twin_port_pressure(uint8_t idx); /* 端口侧独立节点压力 (v2.1) */
PN_TWIN_API void pn_twin_set_leak(uint8_t idx, float k); /* 泄漏注入 (出厂筛选工况) */
```

4.2 从 v1 到 v2: 让模型长出文献的形状

李生的第一版物理 (v1) 是工程一阶近似: 充气压头线性衰减 ($1 - p/P_{\max}$)、抽真空对称、被动排气按时间常数指数衰减。它的优点是直觉、可调; 缺点是形状不对——真实气动系统的高压差段是恒流 (音速阻塞), 充气曲线是 $\sqrt{\quad}$ 压头而不是线性渐近, 开阀数量会改变升降压速率。演示够用, 对拍不够。

v2 重构 (PHYSICS-SPEC.md) 的原则是每条规律给出文献出处, 表 4.1 是逐条对应关系。

表 4.1: 李生物理 v2 的文献依据 (PHYSICS-SPEC §1)

物理规律	文献出处	采用形式
压力动力学	Xavier 2022 §2.1 Eq.1–4	多变气体定律 $PV^\gamma = \text{const}$; $dP_A/dt = \gamma P_A Q/V_A$ ($\gamma=1.2$)
阀孔流量	Xavier 2022 §2.1 (ANSI/NFPA T3.21.3)	$Q = 114.5 u C_v \sqrt{\Delta P \cdot P_{\text{low}}}/\sqrt{T}$; 充气 $Q_c \propto \sqrt{P_A(P_R - P_A)}$
音速阻塞	ANSI 标准隐含 (临界压比 1.893)	高压差段流量与 $\sqrt{\Delta P}$ 解耦 (恒质量流)——v1 完全缺失
占空比标度	Xavier 2022 (40 Hz PWM 阀); FlowIO CHI'21 §4 (泵 PWM)	$u \in [0, 1]$ 线性标度流导
常闭保压/被动释放 单传感器分时	FlowIO CHI'21 §4 FlowIO CHI'21 §4	沿用 v1 已有实现 S0 定位汇流管; S1 端口侧 (Sensors++ 锥形)
规格包线	FlowIO CHI'21: -26 ~ +30 psi、3.2 L/min	UI 曲线按 ± 100 kPa 传感器量程展示

表 4.2 列出 v1→v2 的逐环节替换。最有教学意味的是“容积”一行: v1 里腔体容积是常数, v2 里 $V(n) = V_m + n \cdot V_p$ 随开阀端口数变化——同样的泵功率, 开 3 个口充压自然比开 1 个口慢。这不是高级点缀, 而是 Xavier 2022 Eq.4 中 $1/V$ 因子的直接后果, 也是后来验收断言“容积耦合”的物理来源。

表 4.2: v1 (工程一阶) 与 v2 (文献形式) 逐环节对比

环节	v1	v2
泵充压	线性压头 ($1 - p/P_{\max}$)	孔口形式 $\sqrt{p(P_R - p)}$ + choked 段
抽真空	对称线性	孔口形式 $\sqrt{p(p - P_{\min})}$ + choked 段
被动排气	指数衰减 $\tau=1.6$ s	ANSI 孔口 (choked 恒流段 → $\sqrt{\quad}$ 衰减尾)
容积	固定	$V(n) = V_m + nV_p$ + 开阀瞬时等温混合
微漏	比例 1%/s	不变 (经验值, 标注待标定)

4.3 核心方程与 dP/dt 实现

v2 的每个 50 ms tick 做四件事: 容积合成、孔口流量求和、多变定律积分、传感器钳位。核心代码 (twin_api.c, 注释为源码原有):

Listing 4.2: twin_api.c 物理积分核心: 孔口流量、choked 钳位与多变定律

```

float V = V_MANIFOLD_L + popcount(ports) * V_PORT_L; /* V = V_m + n * V_p (L) */
float P = p + P_ATM_KPA; /* 表压 -> 绝对压 */
float net_q = 0.f; /* 净流量 L/s, 充为正 */

if (pump_on && inlet_on) { /* 充气：孔口形式渐近死点 */
    float PR = PUMP_P_MAX_KPA + P_ATM_KPA;
    net_q += PUMP_C * duty * sq_pos(P * (PR - P));
} else if (pump_on && vent_on) { /* 抽真空：对称形式渐近极限 */
    float Pm = (PUMP_P_MIN_KPA > VALVE_P_MIN_KPA ?
        PUMP_P_MIN_KPA : VALVE_P_MIN_KPA) + P_ATM_KPA;
    net_q -= PUMP_C * duty * sq_pos(P * (P - Pm));
}

if (vent_on && !pump_on) { /* 被动排气/负压回充 (choked 钳位) */
    float d = P - P_ATM_KPA;
    float f = P_ATM_KPA * d; /* f = P_low * ΔP (带符号) */
    float f_choked = P_ATM_KPA * (CHOKED_RATIO - 1.f) * P_ATM_KPA;
    if (f > f_choked) f = f_choked; /* 高压差恒流：与 √ΔP 解耦 */
    if (f < -f_choked) f = -f_choked;
    net_q -= VENT_C * sqrtf(fabsf(f));
}

/* .....泄漏项：进气/排气阀密封面 + 端口侧泄漏 (阀开才对汇流管可见) ..... */
p += GAMMA * P / V * net_q * dt; /* 多变气体定律：dp = γ * P/V * Q * dt (γ=1.2) */
if (p > SENSOR_SPAN_KPA) p = SENSOR_SPAN_KPA; /* XGZP6897D ±100kPa 饱和 */
if (p < -SENSOR_SPAN_KPA) p = -SENSOR_SPAN_KPA;

```

三处细节值得放大。其一，**choked 段**：当上游与下游绝对压比超过 1.893（空气临界压比），孔口流速钳制在音速，流量不再随 $\sqrt{\Delta P}$ 增长——对应代码里对 $f = P_{\text{low}} \Delta P$ 的上下钳位。真机上它的可观测签名是：泄放初期降压速率近似恒定，之后才转入 $\sqrt{}$ 衰减尾。v1 完全没有这个现象，对拍时一眼就能识破。其二，**开阀瞬时等温混合**：端口阀 0→1 的瞬间，汇流管气体与新接通的端口容积（初压取端口节点当前值）均压， $p \leftarrow \frac{P_1 V_1 + P_2 V_2}{V_1 + V_2} - P_{\text{atm}}$ ——密封 60 kPa 时开一个新端口，压力阶跃下降且幅度按 $V_m/(V_m + V_p)$ 可预测，这是验收用例之一。其三， $\gamma=1.2$ ：多变指数介于等温 1.0 与绝热 1.4 之间，取自 Xavier 2022 §4 的实验值——快速充放气介于两者之间，这是文献给出的折中。

4.4 参数成色：孪生模型也会“说谎”

方程形状对了，参数仍可能错。PHYSICS-SPEC §3 为每个参数标注来源与成色（表 4.3），其中真空渐近值的推导过程最能体现“规格书交叉验证”的思维：泵规格书极限是 -58 kPa（370 迷你真空泵，DFRobot FIT0801），但 0520D/F 阀规格书的动作压力下限是 -53.3 kPa——阀在这个深度以下不能可靠通电切换，闭环“关阀保压”会失败。系统可用真空由链路最薄弱环节决定： $\max(-58, -53.3) = -53.3$ kPa。最初的 -38 kPa 是从 FlowIO 借来的值，-58 是泵单独极限，均已被这条推导取代。

注意 C_{vent} 的成色标注：它是全表最“虚”的数——裸阀 C_v 0.05（0520D）/0.10（0520F）全开流导比它高约两个数量级，实物排气会显著更快。SPEC 不掩盖这一点，而是写明“到货重标”。参数表还诚实处理了时间尺度：370 泵空载流量 2.2 L/min 直驱 7 mL 汇流管理论充压远快于孪生当前时标——孪生时标锚定的是 FlowIO 文档级充气曲线（秒级，真实执行器容积下成立）。对不上实物的原因被写进文档，而不是藏进代码。

参数表之外还挂着一**条如实建模的已知缺陷**：固件的超压保护阈值定在 120 kPa，而传感器量程上限是 100 kPa——压力永远到不了 120 才会被钳在 100，真机上这个保护永远无法触发，是一段客观存在的盲区。孪生没有“顺手修好”它，而是原样建模（钳位在 ± 100 ，保护阈值 120 原样传入 `pn_check_overpressure`），并在文件头注

表 4.3: 孪生气动参数表（PHYSICS-SPEC §3 浓缩版；成色 = 数值可信度）

参数	值	来源与成色
$P_R - P_{\text{atm}}$ （充气死点）	+61 kPa	FlowIO Small 模块实测死点（Shtarbanov 论文 Table 2）；到货必测
$P_{\text{min}} - P_{\text{atm}}$ （真空渐近）	-53.3 kPa	max(泵 -58, 阀动作窗 -53.3)——阀是最薄弱环节（0520D/F 规格书）
C_{pump}	9.0×10^{-8}	按 FlowIO 实测充压曲线拟合；到货
C_{vent}	1.0×10^{-5}	calibrate_pump.py 重标
密封微漏率	0.01/s	演示时间尺度假设（40 kPa→0 约 4 s，无实物依据）
V_m / V_p	5 / 2 mL	好品假设 $\approx 3 \text{ mL/min@40 kPa}$ ；手册上限（阀内漏 $\leq 30 \text{ mL/min} \approx 0.073/\text{s}$ ）——接近上限的阀该被判泄漏
γ	1.2	结构设计值，待实测
阀响应	$\leq 20/30 \text{ ms}$	Xavier 2022 §4
		0520D/F 规格书， $\ll 50 \text{ ms tick}$ ，瞬时假设成立

释里标注”待用户决策阈值”。理由与黄徽章一脉相承：孪生的职责是忠实复现当前设计——包括它的缺陷——让缺陷在对拍与验收里暴露，而不是在仿真里被无意识地”优化”掉。孪生若悄悄比真机更安全，它就失去了作为测试台的可信度。阈值本身最终裁定建议降到 90 kPa（量程内留裕量），列入回板前的待决策清单。

4.5 v2.1 多节点端口模型：一个 bug 牵出的物理修正

v2 交付后，用户实测报了一个 bug：G 闭环运行时汇流管压力缓缓下降，而界面上的”端口 1 压力”纹丝不动。排查结论出乎意料——不是显示 bug，是物理模型的 bug：物理层只有汇流管单一气压节点，所谓”端口压力”只是前端变量（阀开时拷贝汇流管值、阀关时冻结），根本没有仿真状态。更深一层：端口泄漏被无条件作用于汇流管节点，而物理上阀关之后，端口侧的泄漏对汇流管传感器不可见——用这样的模型生成的训练数据，含有物理上不成立的签名。

v2.1 的修正（twin_api.c 中 s_port_p[5] 数组）：

1. 每端口独立节点：关阀瞬间快照汇流管压力，此后独立演化（自身密封微漏 + 端口泄漏孔口流）；开阀时与汇流管同节点；
2. 开阀混合的端口初压改用端口节点真值（原假设大气压）；
3. 端口泄漏移到端口侧：阀开对汇流管可见、阀关不可见；
4. 基线微漏条件修正为”泵侧密封即适用”（含 H 诊断保压的端口全通态）；
5. 新增 CLI H <ports> 命令（泵侧密封 + 端口保持通）——泄漏诊断的标准工况。

修正同时暴露了一个采样率声明错误：每 tick 一个采样点实际是 20 Hz，原标注 50 Hz，斜率特征因此虚高 2.5 倍。两处修正一并落地后，孪生行为与”阀是隔离边界”的物理直觉终于一致：关阀后端口压力沿自身泄漏曲线漂移，汇流管读数不再被下游污染。pn_twin_port_pressure() 把端口节点导出到 /api/state.ports_p，前端端口卡片直接显示真值，原先的”冻结 hack”删除。

v2.1 的涟漪还波及了当时搭载在孪生上的 TinyML 泄漏检测：训练工况改为 H 语义、标签改为物理可观测的 Δ 分级、采样率修正并入、类平衡采样——终值准确率 95.73%、normal 召回 98.6%、major 召回 95.3%（零漏报为 normal）、模型 18 KB。但这个功能最终在 2026-10-02 随用户裁定整体下线：对标 FlowIO，官方平台没有出厂级泄漏诊断，白牌客户也未必愿意为此付钱；后端、接口、前端三层拆除归档，CLI L 命令退化为应答 leak=off。这段插曲值得记录的不是模型指标，而是方法论：孪生的价值不随搭载功能的下线而消失——泄漏注入接口（pn_twin_set_leak）保留至今，用于出厂筛选工况的复现与孪生自测；被裁掉的是产品功能，留下的是物理能力。

4.6 形状对不对，断言说了算

v2 换方程之后，验证用例也随之换血——旧断言（渐近线性）对新形状（ $\sqrt{p}$ 压头）必然失效，TEST-CASES §I 重写为一组规律形状断言（表 4.4）。这些断言测的不是数值精度（那是回板对拍的事），而是曲线的函数形状——形状错了，参数再准也没用。

表 4.4: 孪生物理 v2 验证用例（TEST-CASES §I 要点）

用例	断言要点
充压曲线	按 $\sqrt{p(P_R - p)}$ 形式做非线性回归拟合（替代 v1 的渐近线性断言）
choked 恒流	高压差段（表压 $> 53 \text{ kPa}$ ，即 $P/P_{\text{atm}} > 1.893$ 边界之上）释放初期降压速率近似恒定
容积耦合	同 PWM 下开 1 口 vs 开 3 口充压，后者速率显著低（ $\Delta V/V$ 断言）
开阀混合	密封 60 kPa 时开新端口 $\rightarrow$ 压力阶跃下降，幅度按 $V_m/(V_m + V_p)$ 可预测
真空/回充	渐近 -53.3 kPa ；负压开排气阀自然回充至大气压
微漏/量程饱和 自测重算	密封态按微漏率缓降；注入超量程值按 $\pm 100 \text{ kPa}$ 钳位 <code>twin_selftest.c</code> 期望值按 v2 全部重算， <code>build_twin.sh</code> 自测通过

4.7 传感器模型：对着手册逐条校准

压力传感器 XGZP6897D-C 的孪生模型不是拍脑袋的 $\pm 100 \text{ kPa}$ 钳位就完了。对照 CFSensor 官方手册 V1.1 逐条核对，发现原实现有三处到货必炸的驱动级错误（表 4.5）：I2C 地址、读法与换算全错。这正是“孪生先行”的另一种价值——在真机到来之前，靠文档把驱动层的雷排掉。

表 4.5: XGZP6897D-C 驱动级三处修正（对照手册 V1.1，2026-09-23）

项	原实现（错）	手册
I2C 地址	0x6D/0x5D 可配	固定 0x58 （-C 型选型码即地址） $\Rightarrow$ 双传感器必须 TCA9548A
读法	写 0x30 $\leftarrow$ 0x0A 启动 $\rightarrow$ 轮询 $\rightarrow$ 读 0x06	从 0x04 连读 5 字节（ASIC 自动刷新，无启动命令）
换算	$K = 2^{24}/\text{span}$	$P = \text{补码和}/2^{21} \times (\text{span})$ （ $\pm 100 \text{ kPa}$ 型 $\text{span}=200\,000$ ，原差 8 倍）

传感器的其余手册参数进孪生的取舍同样有据：量程饱和 $\pm 100 \text{ kPa}$ 已建模；精度 $\pm 2.5\% \text{FS}$ 属系统性分量，经上电零点自校准消除（装配指南 bring-up 步骤）；零点漂移 $\pm 1\% \text{FS}$ 按校准后残差 $N(0, 0.5)$ 建模；响应 $2.5 \text{ ms}@ \text{OSR}1024$ 与上电 24 ms 均 $\ll 50 \text{ ms tick}$ ，忽略；量化步约 0.1 Pa ，忽略。

4.8 板级电气孪生：BOARD_PARAMS 与黄徽章

气动孪生之外，回板前还建了电气孪生（`board_model.py`）：8 路阀线圈的 RL 电流、5V/3V3 双轨负载、一阶热惯性、600 s 环形历史。阀电流用解析式而非数值积分：开通 $\tau_{\text{on}} = L/(r_{\text{coil}} + r_{\text{ds}}) \approx 1.78 \text{ ms}$ ，关断经 SS14 续流回路 $\tau_{\text{off}} = L/r_{\text{coil}} \approx 1.79 \text{ ms}$ ，电流过零被二极管反向截止钳位——与仿真套件 `sim_engine` 的续流 ODE 是同一物理量，防“双源漂移”。

Listing 4.3: board_model.py 的 BOARD_PARAMS: 每个参数带成色注释 (摘自源码)

```
BOARD_PARAMS = {
    "r_coil": 14.0, # [假设-行业典型] 阀线圈电阻 14Ω (5V/14Ω 0.356A 稳态)
    "l_coil": 25e-3, # [假设] 阀线圈电感 25mH
    "rds": 0.040, # [手册] A03400 @ VGS=3.3V 导通电阻 ~40mΩ
    "vbus": 5.0, # [标称] USB 5V 总线
    "vf_ss34": 0.31, # [手册] SS34 肖特基正向压降 (5V 轨串入)
    "vout3v3": 3.269, # [实测] buck 标称输出
    "load_reg": 0.012, # [仿真] buck 负载调整率 (V/A)
    "i_pump": 0.35, # [假设] 泵电机平均电流
    "theta": {"cpu": 35.0, "buck": 130.0, "mos": 350.0}, # [手册] 热阻 °C/W
}
BUCK_EFF = 0.876 # [仿真] buck 效率
```

模型的其余两个状态量同样一行一个物理假设:逻辑负载电流 0.43 A 由三部分估算——ESP32 峰值约 0.24 A、TCA9548A 与 CH340 等外围合计补足 (成色: 估算, 回板电流钳实测); 板级热惯性按一阶模型推进, 时间常数 30s, 三个热节点 (CPU/buck/MOS) 各挂手册热阻 35/130/350 °C/W。温升的意义在验收侧: A215 工单要用红外热像对照修正这三条热阻, MOS 那 350 °C/W 是全板最孤注一掷的数字——SOT-23 封装的天然弱势, 设计上靠”8 路阀不同时满占空比”的使用假设兜底。

电气参数的成色管理与气动侧同一套思想, 汇总成物理校准状态矩阵 (设计规格书 §10): 每个模型参数区分两件事——**结构正确性** (方程是不是物理规律, 如多变定律、RL 解析式) 与 **参数校准度** (数值是不是本机实测), 不混谈。成色分四档: 实测 > 手册 > 仿真 > 假设。全项目电气参数中最硬的一档在仿真引擎里: buck 分压比 R4/R5 就是板上实阻 (3.27 V 输出的来源, 见第 5 章修正 #11); 最软的一档是阀线圈 R/L——BOM 未锁传感器与阀型号前只能用行业典型值。

矩阵的呈现约束写进了前端: 遥测面板角落固定显示**黄色徽章**”模型参数: 理论值 (未本机标定)” ; 回板后跑完六项电气校准 (A211–A215: 阀线圈万用表 +LCR 实测、RDS 示波器反推、ESR 纹波反推、效率四线法、 θ_{ja} 红外对照、ESP32 电流钳), 改 BOARD_PARAMS 常量, 徽章转绿。不让演示数字冒充实测——这是孪生的诚信底线, 也是它作为产品文档的可信度来源。

4.9 仿真套件四电路与对拍闭环

与板级模型配套的还有四个电路的行为级仿真 (sim_engine.py, 无 ngspice 环境下的自研状态机 + SVG 输出): 电源效率、输出纹波、温升、I2C 时序。四电路的达标判据与一次 I2C 告警案例的复盘见第 13 章 A103 工单; 参数重算接口 (POST /api/board/sim) 允许前端改元件值即时重跑。

4.10 孪生即测试台: API v1.2 一览

孪生对上位机的价值, 取决于它暴露的接口与真机的距离。API v1.2 之后, 孪生服务 (server.py 加载 pn_twin.dll) 的板级接口与真实遥测同构 (表 4.6)——前端与 SDK 写一次, 两种模式 (HTTP 孪生 / BLE 真机) 切换运行, 仅以徽章区分, 防混淆。

其中 /api/time 与 record/replay 是孪生独有的两种能力: 时间可以暂停、可以慢放——真机上 20ms 的阀电流建立过程在孪生里可以拉到肉眼可见; 场景可以录制、可以逐帧回放——一个触发条件可疑的 bug, 录制一次就能反复重演。board_model 侧还有两处工程细节: 遥测快照与推进互斥 (tick 线程持锁写入, HTTP 线程同步重算读出, 杜绝”半个状态”); 服务失联时置 stale 标志, 前端立即降级显示而非渲染过期值。600s 的环形历史 (6000 点) 覆盖最长的一次充放气循环, 配合 CSV 导出即得对拍数据集。

表 4.6: 孪生服务板级 API v1.2（设计规格书 §3 增量契约）

接口	用途
GET /api/board/state	板级实时遥测：阀电流、双轨电压、结温、历史
POST /api/board/sim	参数重算：改元件值即时重跑四电路仿真
GET /api/board/sim/presets	预设参数集（BOM 实值 / 手册 / 假设三档标签）
GET /api/board/assembly	结构件清单（3D 爆炸视图数据源，第 8 章）
GET/POST /api/time	孪生时间控制（暂停/倍速——真机没有的调试自由度）
POST /api/record+replay	场景录制回放：一次复现，永远复现
GET /api/board/history/export	遥测 CSV 导出（对拍数据就地落盘）

4.11 到货之后：calibrate_pump.py 三实验协议

孪生参数表里所有“借值”与“假设”都指向同一个收尾动作：到货实测。calibrate_pump.py 把这个动作固化成三个实验的协议（P0 装配完成、共地确认后执行）：

1. 实验 A，充气死点：堵死充气口（或全密封），I 31 255 连续 30s，每秒记录汇流管压力——末端渐近值即 P_R 实测；
2. 实验 B，真空死点：堵死气路，V 31 255 连续 30s，每秒记录——末端渐近即 $P_{\min}$ 实测（验证 -53.3 的阀门假设）；
3. 实验 C，升压曲线：接实际执行器容积，I 1 255 从 0 充到死点，每 0.5s 记录——用于拟合 C_{pump} 。

记录格式统一为无表头两列 CSV（time_s, pressure_kPa）；脚本读入后做渐近估计（末端中位数）并检查收敛（末端抖动 < 0.5 kPa 才算到达死点，否则提示加长实验时长），输出直接可抄的 #define 修改建议——含建议值与数据文件名注释，对照 twin_api.c 改两行即完成一次参数升级。实验 C 的 1–4s 平均升压率与 FlowIO 标定参照值（2s→19 / 4s→35 kPa，约 8–9 kPa/s）对照，明显更快则说明 C_{pump} 需上调。改完常数后的回归门槛写在脚本尾注里：build_twin.sh 与 run_tests.sh 全绿。三个实验、一个脚本、两行宏——这就是“黄徽章转绿”在气动侧的全部操作量；它对应的验收工单是 A216–A218（第 13 章）。

电气侧的校准动线同构但更手工：六项实测（表 4.7）改的是 BOARD_PARAMS 常量块，无需新脚本——校准工具的价值在于协议与判定门槛，不在于代码量。

表 4.7: 电气孪生六项校准（设计规格书 §10.2；对应工单 A211–A215）

参数	当前成色	回板校准法
阀线圈 $R=14\ \Omega$ / $L=25\ \text{mH}$	假设（行业典型）	万用表 + LCR 实测
AO3400 $R_{\text{DS}}=40\ \text{m}\Omega$	手册	示波器 V_{ds} 波形反推
buck ESR / C_{out}	手册/仿真	示波器纹波实测反推 ESR
负载调整率 / 效率 87.6%	仿真	四线法效率实测
θ_{ja} （MOS 350 / buck 130 / CPU 35 °C/W）	手册	红外/热电偶对照修正
ESP32 逻辑电流 0.43 A	估算	电流钳实测（WiFi 活动系数）

从仿真到验收的闭环由 Phase CAL 六张工单（A301–A306）完成：改 BOARD_PARAMS 与 PHYSICS 常数后，孪生曲线与实测曲线叠图，六个维度（阀电流、电源效率、温升、充气升压、真空死点、闭环精度）各出一张对比图，汇总成“数字孪生精度报告”——详见第 14 章。

本章小结

孪生先行的本质是把“等板子”变成“先跑起来”：逻辑层与物理层分离让固件、前端、SDK 在回板前全链路演练；气动物理以文献形式（多变定律 + ANSI 孔口 + choked + 容积耦合）重写，再用多节点端口模型修正了“冻结端口压力”这类物理不成立的行为；电气侧以 BOARD_PARAMS 成色注释与黄徽章机制管理参数可信度。回板之后，孪生与真机对拍——偏差本身就是交付物。

第五章 原理图：从拓扑到 13 次修正

本章记录 FLOWIO-CN P1 主控板原理图的诞生过程。原理图不是一次画成的：它经历了 13 次编号修正（总表见 5.4 节），其中三次——电源或门二极管、Buck 反馈电阻、USB 转串口芯片——任何一次漏掉都会让整板无法工作或无法烧录。本章将这三个案例按“症状 → 根因 → 修法”完整复盘，并给出可 grep 的源码锚点。全部修正的权威记录见规格书 docs/superpowers/specs/2026-10-01-flowio-p1-pcb-design.md §9。

5.1 设计基线：80% 继承与总体拓扑

原理图以 ESP32-S3-DevKitC-1 官方原理图为 baseline：模组连接、EN 复位电路（RC+ 按钮）、strapping 偏置、自动下载（SS8050x2）、USB 差分对与 ESD 全部继承，约占 80%；FLOWIO-CN 特有的 8 路 AO3400 阀驱动、TCA9548A 五通道 I2C、五路 XH2.54 传感器座、WS2812B 状态灯与 12 个测试焊盘为新增。

电源拓扑（修正后）如下：DC-005 座（5V/3A）与 USB-C VBUS 各经一颗肖特基二极管做 Diode-OR 汇入 5V 共用轨；5V 轨经 TPS54331DR Buck 降至 3.3V/3A，供 ESP32-S3、TCA9548A、CH340K、WS2812B 与五路传感器；8 路 AO3400A 直接驱动 7 阀 + 1 泵。阀 GPIO 分配为 4/5/6/7/10/11/12/21，I2C 主总线在 GPIO8/9（各 4.7kΩ 上拉），WS2812B 在 GPIO48。

5.2 原理图即代码：gen_sch.py 与程序化校验

整张原理图由 hardware/flowio-p1/tools/gen_sch.py 程序化生成：135 个器件、101 个网络、344 个连接全部写成数据结构，经程序化校验后导出 KiCad 工程与 PDF，ERC 零错误（commit 086e789）。这带来一个关键性质：**修正是可 diff 的**。每颗器件的 LCSC 编码、封装与引脚-网络映射都是一行代码，本章三个案例都能用一行 grep 定位到修正后的现场：

- Diode-OR: `grep "SS34_C8678" gen_sch.py` → L215–L228（D1/D2）与 L245–L246（Buck 续流 D3）；
- Buck 反馈: `grep "R324K" gen_sch.py` → L249（R5 下分压电阻）；
- CH340K: `grep "CH340K" gen_sch.py` → L263–L266（U4 引脚-网络映射）。

5.3 三个真实修正案例：症状、根因与修法

5.3.1 案例一（修正 #9）：Diode-OR 的二极管——1A 器件扛 2A 电流

症状。初版双输入或门（DC 座与 USB-C 各一颗 1N5819W，1A/40V 肖特基）在纸面上成立：两路 5V 各自隔离、互不倒灌。但做电流预算复核时发现，5V 共用轨的下游是 8 路执行器——泵与多阀同时工作的峰值电流超过 2A，而无论从哪一路输入，全部电流都要流经那颗 1A 额定的二极管。若未修正，回板后的表现将是二极管压降异常增大、严重发热，长时间满载运行有烧毁风险——一块“原理上能跑”的板子藏在一条超载的电源路径。

根因。选型凭习惯拿了信号级的 1N5819W（1A，SOD-123），没有对电源路径做峰值电流预算。这是典型的“拓扑正确、参数错误”：Diode-OR 结构本身没有问题，问题是流过它的电流。

修法。或门两颗二极管升级为 SS34（3A/40V，SMA 封装，C8678），与 Buck 续流管（D3）统一物料；阀路续流仍用 SS14（1A）——修正 #8 已把两类续流管的电流等级区分开。修法落在 gen_sch.py L215–L228：D1（L219）接 DC_IN → +5V，D2（L222）接 USB_VBUS → +5V，均标注 SS34_C8678。

5.3.2 案例二（修正 #11）：Buck 反馈电阻——2.33V 的“3.3V 轨”

症状。TPS54331 的输出电压由反馈分压决定： $V_{\text{out}} = V_{\text{ref}} \times (1 + R_{\text{上}}/R_{\text{下}})$ ，其中 $V_{\text{ref}} = 0.8 \text{ V}$ 。初版取 $R_{\text{上}} = 10 \text{ k}\Omega$ 、 $R_{\text{下}} = 5.23 \text{ k}\Omega$ （E96 顺手取值），代入得 $V_{\text{out}} = 0.8 \times (1 + 10/5.23) \approx 2.33 \text{ V}$ ——一颗电阻选错，3.3V 轨实际只有 2.33V。ESP32-S3 的供电范围为 3.0–3.6 V：回板后模组根本不会启动，表现是“电源灯亮、串口无横幅”，属于最难盲查的一类故障（一切看起来都通电了）。

根因。下分压电阻按 E96 序列顺手取值，取完没有代回分压方程验算。电源芯片的外围参数不能靠“阻值常见”来选，必须算。

修法。 $R_{\text{下}}$ 改 $3.24 \text{ k}\Omega$ 、1% 精度： $V_{\text{out}} = 0.8 \times (1 + 10/3.24) \approx 3.27 \text{ V}$ ，落在 $3.3\text{V} \pm 3\%$ 的窗口内。修法落在 gen_sch.py L248–L249：R4（ $10 \text{ k}\Omega$ ，+3V3 → FB）与 R5（ $R324\text{K}$ ，FB → GND）构成分压。这个案例后来直接进入了回板验收工单 A201“首电三测”的期望值： $5\text{V} \pm 5\%$ 、 $3.3\text{V} \pm 3\%$ 。

5.3.3 案例三（修正 #10+#12）：USB 转串口芯片三次换型——CH340N→E→K

症状。自动下载电路需要转串口芯片同时引出 DTR# 与 RTS# 两根握手线（经 $1 \text{ k}\Omega$ 驱动两颗 SS8050，分别拉 EN 与 GPIO0，这是 esptool default_reset 进下载模式的前提）。初选 CH340N：封装里没有 DTR# 引脚，自动下载电路无从谈起——每次烧录都要手按 BOOT。于是修正 #10 换成 CH340E：内置振荡器还省掉了 12 MHz 晶振与两颗负载电容。但核对官方文档 CH340DS1 v3B 的引脚表后发现：**CH340E 同样没有 DTR#**。若到此为止，回板后将得到一块“能打印日志、但不能自动烧录”的板子。

根因。两次选型都依赖社区记忆而非官方引脚表——“CH340 系列应该有 DTR”是一个想当然。引脚级的契约（哪个封装的哪个脚是 DTR#/RTS#）必须以厂商文档为准。

修法。换 CH340K（ESSOP-10，C968586）：官方引脚表确认 pin4=DTR#、pin6=RTS#，且内置振荡器。修法落在 gen_sch.py L255–L284：U4 引脚-网络映射（L263–L266）中“4”：“DTR”，“6”：“RTS”，经 R19/R20（ $1 \text{ k}\Omega$ ）与 R21/R22（ $10 \text{ k}\Omega$ 上拉）驱动 Q11/Q12 两颗 SS8050（L276–L283）完成自动下载。另按修正 #2，CH340K 从 3.3V 轨供电且 V3 短接 VCC（3.3V 模式），从根上消除 USB 5V 向 3.3V 轨的倒灌路径。

5.4 13 次修正总表

表 5.1: 原理图阶段 13 次编号修正（spec §9 修正记录摘编）

#	修正	原因
1	电源从双轨改单轨	过度复杂；消除 CH340N 倒灌路径
2	CH340N 从 3.3V 供电	手册 p9 倒灌防护；简化
3	新增 I2C 上拉 $\times 12$	51 核心板交叉检查发现遗漏
4	新增 strapping $\times 4$	51 核心板交叉检查发现遗漏
5	新增 SS8050 外围 $\times 4$	深度审查发现遗漏
6	新增 WS2812B 外围 $\times 2$	深度审查发现遗漏
7	电源开关改为无开关	SS-3235S 的 50mA 额定不够 3A
8	SS14 与 SS34 区分使用	阀续流 1A 与 Buck 续流 3A 电流等级不同
9	Diode-OR 改 SS34	5V 轨承载泵 + 阀峰值大于 2A，1N5819W 的 1A 额定不足
10	CH340N→CH340E	N 无 DTR# 无法自动下载；E 内置振荡器省晶振
11	反馈电阻 $5.23\text{k} \rightarrow 3.24\text{k}$	5.23k 只得 2.33V； 3.24k 得 3.27V
12	CH340E→CH340K	官方引脚表核对：E 无 DTR#，K 有 DTR#(4)+RTS#(6)
13	板尺寸 $80 \times 70 \rightarrow 90 \times 75$	17 个连接器在 80×70 饱和；8 端子单排需 $\geq 88 \text{ mm}$

5.5 修正从哪里来：三层交叉检查

复盘 13 次修正的来源，可以归纳出三层互补的检查网：其一，**对照检查**——拿一块已知可用的核心板（51 板）逐网络比对，补出 I2C 上拉与 strapping（#3/#4）；其二，**参数验算**——对每条电源路径做电流预算、对每个分压/补偿网络代回公式（#7/#9/#11）；其三，**官方文档核对**——引脚级契约以数据手册为准（#10/#12，CH340DS1 v3B）。三层缺一不可：第一层抓“漏画”，第二层抓“画错”，第三层抓“想当然”。

第六章 PCB 布线攻坚：从 146 到 0

布局定稿时，四层板上还留着 146 个未连接；Gerber 交付时，这个数字是 0。本章按时间线复盘这条下降曲线——146→17→6→0——的四个阶段：自动布线器的两轮冲锋、一次靠目视检查才抓到的隐形事故、两个几何模型的修正，以及自研工具链的收官。每一阶段对应一次 git 提交，数字与细节均可回溯。

6.1 布局定局与层叠

骨架定稿（commit 41e697e）：90×75 mm 四层板，层叠为 L1 信号 + 元件 / L2 GND 整片 / L3 电源分区（5V+3.3V）/ L4 信号；电源过孔 54 个、GND 缝合过孔 36 个。板尺寸从 80×70 放宽到 90×75 是布局实践的结论：8 个螺丝端子单排就需要 ≥88 mm，17 个连接器在原尺寸下已经饱和（修正 #13）。布局收敛后，自动布线器（自研 A* 双层版）已建立但尚不完善，剩余 146 个未连接决定交给 Freerouting 与后续攻坚。

6.2 146→17：freerouting 双轮与布线后清理

commit c3337a3：采用 freerouting v2.4.1（headless CLI，DSN 导出、SES 回灌）跑两轮 rip-up&reroute 与 BatchOptimizer，未连接从 146 降到 17，短路为零。布线后的清理同样关键：重建电源平面（GND 全实连）、把 0.25 mm 线宽回退到 0.2 mm 消除约 130 处间距违规、zones_intersect 与 starved_thermal 清零。

6.3 天线禁布区事件：目视检查抓到的隐形事故

这一阶段最有教学价值的不是布通率，而是一次目视检查的战果。用户强制要求对顶/底面 2400×2000 渲染图逐区人工检查，配合程序化核实，结果发现：布线脚本 route_pcb.py 在重建电源平面时，把 ESP32 模组天线下方的禁布区删掉了——GND 与 +3V3 填充直接铺进了天线正下方（HitTestFilledArea 四层命中）。这类事故不会报任何 DRC 错误：铜皮完全合法，只是 WiFi 从此没有辐射窗口。修复是补写 fix_keepout.py：重新写入 ANT_KEEPOUT 规则区域（x 19.5–36.5 / y 0.2–6.4 mm，四个铜层禁填充/走线/过孔），重灌后 6 个探测点零命中，DRC 无新增。安装孔四处全净空、走线无乱麻。教训只有一句话：自动化的每一步“重建”，都可能静默丢弃它不认识的约束——这就是本书坚持“渲染图逐区目检 + 程序化复核”双轨的原因。

6.4 模型与工具攻坚：17→6

接下来是三个提交的连续攻坚。第一步修模型（commit d549f4a）：GND 过孔避让 C1.1 时发现旧的碰撞模型假设“矩形焊盘四角伸出孔半径一半之外按圆近似”，几何上不成立——改为点对旋转矩形的精确距离计算，孔距规则随之从 3 收紧到 2。第二步修布局（commit cd47167）：删坏线、补 5V 过孔、重连 D10.1；把 HD 从 (77,62) 移到 (68,65) 给 D10 让出走线走廊，孔类违规 6→1、悬空过孔 2→1。第三步上自研工具链（commit a38172a）：boardgeom 提供全障碍感知的落点库（焊盘含孔、走线、过孔、NPTH、板边、禁布区），netdoctor 做填充岛探测（PIP）、岛间跳线、焊盘逃逸与 In1–In2–B.Cu 多层 BFS 路由；配合 freerouting 三轮，未连接 17→6，其余 error 类全零。遗留的 6 个未连接（R13/R29/C3–U3.2 口袋簇与 GND/3V3 zone 对）原本预计留给 GUI 手工收尾。

6.5 6→0：KRT 平面修复与终版打样包

收官来自 commit d0eb1b2：KRT repair_planes 做全层修复——GND 三层平面与 3V3（12 区 11 路）、5V（6 区 5 路）的 In2 断区全部接通，oracle_reconnect 连带解决了 R13/R29/C3–U3.2 口袋簇，未连接 6→0，error 类全

零,超出预期。工艺侧同步对齐: `min_hole_clearance` 25→20 (JLC 经济档合规), 8 处 `via-in-pad` 标注需 IPC-4761 Type VII 填孔电镀。终版打样包 (commit fa2f138) 经三重验证交付: KiCad DRC、KRT `check_connected` (75 网全通、`refill` 交叉核对)、禁布区探测零命中; 顶/底渲染目视复查中发现的视觉疑点, 经程序化裁决确认为 GND 填充纹理的误读。剩余的 111 条 DRC 记录全部是外观类 (`silk_over_copper` 54、`courtyards_overlap` 44 等), 已列入豁免清单, 嘉立创可制。

成品渲染如图 6.1 (顶面: 模组、端子排与天线净空区) 与图 6.2 (底面: 慢信号与 GND 缝合)。

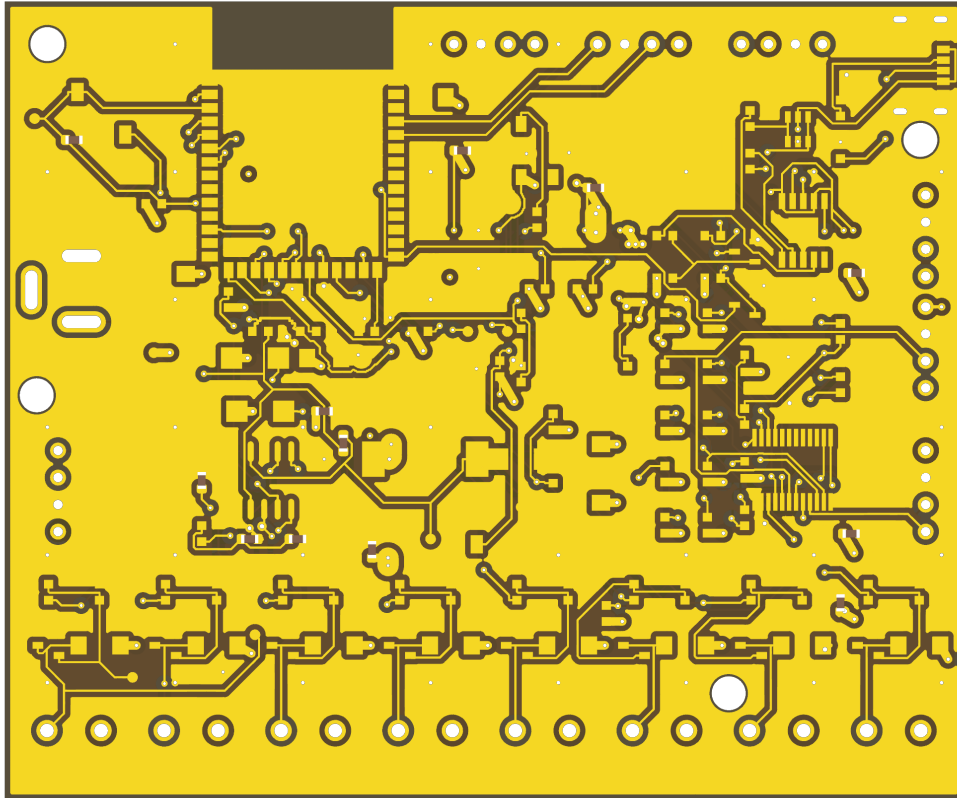


图 6.1: P1 顶面渲染: ESP32-S3 模组天线端朝板边, 端子排单列布局

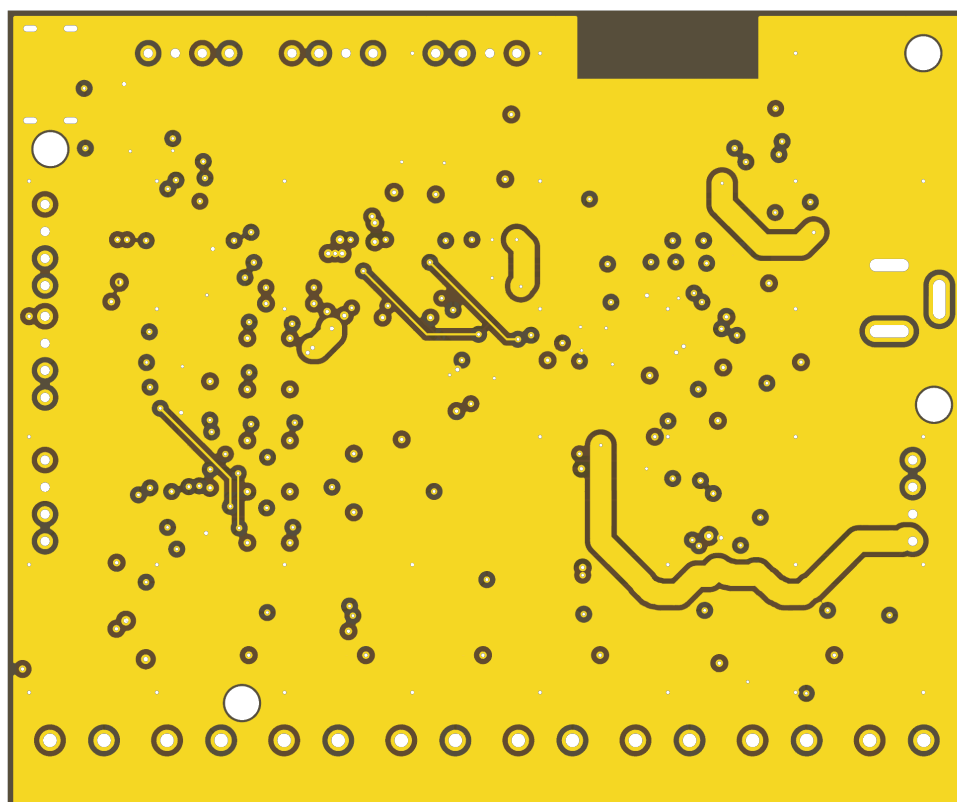


图 6.2: P1 底面渲染: L4 慢信号层与 GND 缝合过孔

第七章 制造：嘉立创全代工实战

第 6 章结束时，P1 板的 DRC 清零、Gerber 交付。但”可制造”与”已制造”之间还隔着一段路：BOM 要带商城编码、贴装坐标要被 SMT 产线解析、工艺备注要让工人看得懂、费用要能过财务。本章是这段路的完整实录——目标交付形态在 README-jlc-order.md 第一行就写死了：用户零手工焊。嘉立创（JLC）完成 PCB 制造 + 107 件 SMT 贴焊 + 16 件 THT 代插代焊，上下外壳 3D 代打；白牌厂商收货后只需拧 M3 螺丝、插线即用。对 B2B 客户而言，”零焊接门槛”不是便利，是采购决策的一部分——他们的产线上没有回流焊。

为什么坚持全代工而不是”贴片代工 + 插件自焊”的半代工？算三笔账。**能力账**：16 只插件里有 DC 座、USB-C 座、8 只螺丝端子与 5 只 XH 座，手工焊接的一致性完全依赖工人，而端子排的机械强度直接吃焊接质量——半代工把整板最受力的一批焊点留给了随机性。**责任账**：单一供应商完成制造 + 贴装 + 插件，出问题时责任边界清晰；半工工的虚焊到底是”板的问题”还是”焊的问题”，扯皮成本远超省下的代插费。**规模账**：打样 2 片与小批量数百片走同一条标准版工艺路径，转产不需要换产线、换工艺文档——对以”起量”为目标的 B2B 项目，这比单价更重要。

7.1 下单资产包：fab/ 目录里都有什么

制造输出的全部资产集中在 hardware/flowio-p1/fab/（表 7.1）。每个文件都有明确的上传去向，下单流程里不会出现”临时导出一个文件”的环节——所有产物在布线清零当日一次性生成、打包、核验。

表 7.1: fab/ 下单资产清单（README-jlc-order §0）

文件	用途
flowio-p1-jlc.zip	PCB 制造 Gerber 包（下单第一步上传）
flowio-p1-bom-jlc.csv	PCBA 物料清单：38 行 / 123 件，逐行带 LCSC 编码
flowio-p1-pos-jlc.csv	贴装坐标（CPL）：首 4 行 # 注释为核对说明
flowio-p1-pos.csv	原版无注释 CPL（上传者报错时的备用，内容与上行一致）
assembly-top/bottom.pdf	上/下贴位置人工核对图
../enclosure/case-top.stl 等	3D 打印上传件（第 8 章）

7.2 制板参数与三重验证

制板参数（表 7.2）按嘉立创经济档能力逐项对齐：0.2/0.2 mm 最小线宽/间距、0.3 mm 孔/0.6 mm 盘的最小过孔（电源孔个别放宽到 0.45/0.9）。两个容易被忽略的选项各有一句理由：**阻焊过孔覆盖**（Gerber 已做 subtract-soldermask 处理）让过孔不被锡珠桥接，也为后文的 via-in-pad 塞孔打底；**表面处理选 ENIG**（沉金）而不是喷锡，是为了 XH 插件与端子的可焊性一致性——这是打样包里少数”贵但值”的选项。

表 7.2: 嘉立创打样参数（README-fab）

项	值
层数/尺寸	4 层 90×75 mm 矩形
板厚	1.6 mm
最小线宽/间距	0.2/0.2 mm
最小过孔	0.3 mm 孔 / 0.6 mm 盘（个别 0.45/0.9 电源孔）
阻焊	过孔覆盖（Gerber 已 subtract-soldermask）
表面处理	ENIG（XH 插件可焊性）
丝印	白色
工艺边	无需

布线终态的 DRC 记录一并写入 README 供下单前回看：未连接 0、error 类 0，仅剩外观类豁免（丝印压铜/库封装 courtyard 等，详见第 6 章豁免清单）。交付不是“DRC 过了”一句话，而是三重连通性验证：KiCad DRC + KRT 铜级连图（75 个网络全通，refill 交叉核对一致）+ 天线禁布区 8 点探测零命中。三道验证互为独立实现，任何一道单独放行都不作数——这是本书反复出现的“双轨复核”原则在制造环节的落地。

Gerber 包本身的构成也有核验口径：11 个层文件（F/In1/In2/B.Cu 四铜层 + F/B.Paste + F/B.Silk + F/B.Mask + Edge.Cuts）加 .gbrjob，钻孔 ExCELLon 格式（PTH/NPTH 合并、单位 mm、附 map 图核对）；打包 flowio-p1-jlc.zip 后的三条自检——非 txt 文件数 = 层数文件数 + 钻孔、无 0 字节文件、zip 不超 20 MB——写在收尾规格书 §4，属于“上传前 30 秒能做完”的检查，但省掉任何一条都可能换来一轮退单。规格书同时记录了一个显式的“不做”：不导出 STEP——多数封装无 3D 模型（渲染已证实），导出无意义；外壳核对走尺寸图（第 8 章）。把“不做事”和“做的事”一起写进决策记录，是防止后人好心补漏反而引入垃圾产物的办法。

7.3 下单七步：从 Gerber 到付款

PCBA 下单本身是一个七步流程（jlc.com → PCB 制造 → 勾选“PCB 组装”），每一步的核对点见表 7.3。它看起来琐碎，但每一步都对应一类真实的翻车：参数错档、料不上贴、坐标翻转、工艺未声明、贴装面搞反——每一类都有人付过学费。

表 7.3: 嘉立创 PCBA 下单七步与核对点（README-jlc-order §1）

步	操作与核对点
1	上传 flowio-p1-jlc.zip，参数照表 7.2（4 层 / 90×75 / 1.6 / 0.2/0.2 / 过孔覆盖 / 白油 / 无工艺边 / ENIG）
2	组装类型选 标准版 （硬约束，见 §7.6）
3	数量按打样策略填写（§7.7）
4	上传 BOM，确认解析：38 行全部匹配物料、无“无物料/缺码”行，全部器件勾选上贴
5	上传 CPL，确认解析：123 个位号、全部 top 面；PosY 为负正常（§7.8）
6	留言栏抄录工艺备注：via-in-pad 塞孔封帽 + 孔距 0.20 + ENIG（§7.4）
7	付款前看预览拼版图：顶面 107 SMT + 16 插件全部有位号标注

7.4 via-in-pad × 8：塞孔封帽工艺

全板有 8 处盘内孔（via-in-pad）：TP1.1、C12.1、LED2.1、U5.2、R24、Q11.2、D10.1、C16.1。它们的成因各不相同（表 7.4）——去耦电容与测试点的焊盘下借孔换层、USB 串口区的器件密集、+5V 肖特基附近的走线拥挤——但风险是同一个：普通过孔不塞孔，回流焊时焊膏会顺着孔桶流走，焊盘虚焊、器件立碑，而且缺陷藏在器件底下，目检看不见。

表 7.4: 8 处盘内孔清单与器件角色

焊盘	器件	角色
TP1.1	测试点	+3V3 首电测量点（A201 工单）
C12.1 / C16.1	0603 电容	100 nF 去耦（电源就近）
LED2.1	LED	电源指示灯（PWR）
U5.2	USBLC6	USB ESD 保护
R24	1 kΩ	用户指示灯限流电阻
Q11.2	SS8050	自动下载三极管（GND 脚）
D10.1	SS14	+5V 肖特基

对策是 IPC-4761 标准的**树脂塞孔 + 电镀封帽**工艺：孔先用树脂填实，表面镀铜盖平，焊盘恢复成平整平面。打包参数页（README-fab）标注为 Type VII，下单留言栏按 Type VIII（封帽之上再加表面覆盖）抄录——两者在生产端指向同一条“塞孔 + 封帽”产线，名称差异不影响执行，但必须在下单时**显式声明**，否则产线默认不塞孔，焊膏流入孔桶的虚焊就在所难免。下单留言原文如下，可直接抄录：

本板 8 处 via-in-pad（TP1.1 / C12.1 / LED2.1 / U5.2 / R24 / Q11.2 / D10.1 / C16.1），需 IPC-4761 Type VIII（树脂塞孔 + 电镀封帽），否则焊膏流入孔桶造成虚焊；最小孔间距 0.20mm（经济档标准支持，如需回到 0.25 请联系改布）；表面处理 ENIG（XH 插件可焊性）。

另一条工艺备注同样来自布线终态：`min_hole_clearance` 从 0.25 收到 0.20mm（经济档标准支持）。README 里特意写了“若需回到 0.25 请告知重布”——把可回退的余地留给产线，比硬顶参数下单后被退单更省一轮时间。

7.5 BOM 的单一真值源：make_bom.py 与 LCSC 编码

BOM 不是从原理图工具里“导出”的，而是从代码生成的**真值源**里“再生成”的。P1 的原理图本身由 `gen_sch.py` 生成（第 5 章），其中 PARTS 列表是全板元件的唯一事实：每颗料以 `P(ref, sym, val, fp, lcsc, x, y, nets)` 七元组喂入——`lcsc` 字段在选型时就回填好商城编码，BOM 生成时零查询、零手工。`make_bom.py` 的工作是把这份真值源变成 JLC 上传格式，全程纯标准库：

Listing 7.1: `make_bom.py` 核心：受限解析、分组合行、覆盖率与交叉验证（摘编）

```
# 受限 ast 解释器：只允许 P() 调用 + 字面量/名/下标/算术/f-string,
# 覆盖 gen_sch.py 里 PARTS 的三种喂料方式：顶层 += 列表 / for-append / for-+=
# (禁 exec —— 生成下单文件时不执行任何原理图代码)

placed = [p for p in PARTS if p["lcsc"]] # 有码 -> 可上贴
# 分组键：(lcsc, footprint, value) —— 同码同封装同值合一行
for p in placed:
    groups.setdefault((p["lcsc"], p["fp"], p["val"]), []).append(p["ref"])

def mpn_of(pt):
    """sym 字段清洗成 MPN：去掉 JLC 库命名尾缀 Cxxxxxx/ 悬空下划线"""
    return re.sub(r"_C\d+$", "", pt["sym"] or "").rstrip("_")

rows.sort(key=lambda r: (r["tht"], ref_key(r["refs"][0]))) # SMT 在前，ref 自然序
# ..... 写出 flowio-p1-bom-jlc.csv (Comment, Designator, Footprint, LCSC Part #) .....

print(f"SMT_LCSC 码覆盖率: {sum(1 for p in placed if not is_tht(p['fp']))}/{n_smt} = 100%"
      if all(p['lcsc'] for p in placed if not is_tht(p['fp'])) else "SMT 存在缺码!")
# 交叉验证: BOM 位号集合 == CPL 位号集合 (TP/HA-HD 等无器件孔除外)
if not only_pos and not only_bom:
    print(f"交叉验证: BOM_{len(bom_refs)}_refs == CPL_{len(pos_refs)}_refs, 一致")
```

四个设计点值得展开。其一，**受限 ast 解释器**：解析 PARTS 时只允许 `P()` 调用与字面量运算（禁 `exec`），与 `gen_pcb.py` 同一模式——生成下单文件的动作不执行任何原理图代码，供应链工具链不做代码执行。解释器覆盖 PARTS 的三种喂料写法：顶层 `+=` 列表、`for` 循环 `append`、`for` 循环增量累加——真值源怎么组织是原理图的自由，解析器全接得住；遇到不支持的语法节点立即报错退出，绝不静默漏料。其二，**分组合行**：以 (LCSC 码, 封装, 值) 为键合并同类料，123 件收敛为 38 行，行序 SMT 在前、同组位号自然数排序 (C1, C2, ..., C16 而不是字典序的 C1, C10, C11)；表 7.5 是成品 BOM 的头部几行。其三，**LCSC 编码回填**：编码不是 BOM 阶段补的，而

是选型阶段随 P() 调用逐颗回填进真值源，sym 字段里的 JLC 库命名后缀（_C8678 之类）在 mpn_of 里清洗成干净的 MPN——”设计即含供应链信息”，这是全代工艺流程能全自动的原因。其四，自检兜底：SMT 与 THT 的 LCSC 覆盖率分别断言 100%，BOM 与 CPL 位号集合交叉验证必须相等（pos 有而 BOM 无、BOM 有而 pos 无，两个方向的差集都必须为空）；无码器件（测试点 TP1-TP12 是裸铜焊盘、安装孔 HA-HD 是板件孔）明确打印”不入 BOM”，产线不会看到来路不明的空行。

表 7.5: 成品 BOM 头部 (flowio-p1-bom-jlc.csv, 节选)

Comment	Designator	Footprint	LCSC
CL10B104KB8NNNC	C2,C5,C6,C10-C13,C15,C16	C_0603	C1591
SS34	D1,D2,D3	SMA	C8678
SS14	D4-D11	SMA	C2480
TYPE-C_6P	J2	TYPE-C-SMD	C456012
ESP32-S3-WROOM-1	U1	SMD 18x25.5	C2913202

价格口径顺带一提：全板最贵单件 U1 (ESP32-S3-WROOM-1-N16R8) 在规格书记录为 5.14 元、下单页实时价一度约 26 元——LCSC 编码锁定的是物料身份，价格以下单页实时价为准，预算区间里已留了余量。

7.6 标准版是硬约束，不是省钱选项

PCBA 下单第二步要选组装类型：经济版或标准版。对这个决定，README 的措辞罕见地斩钉截铁——”不是省钱问题，是硬约束”，理由有二：

- **U1 卡死经济版**：ESP32-S3-WROOM-1-N16R8 (C2913202) 在 JLC 元件库的属性是 standard-only——经济版产线无法上贴这颗料。这不是传闻，是实测查询确认的库属性。
- **THT 只有标准版代插代焊**：J1 (DC 座) + J5-J9/J18/J19/J10-J17 共 16 只插件，只有标准版提供代插代焊。这 16 个孔位器件正是”零手工焊”的关键——经济版下单意味着 16 只连接器全部手工焊接，白牌客户的产线门槛立刻回来了。

页面若出现”经济型 + 代插件”之类的组合选项也不行——U1 那一条就先卡死。所以标准版不是在两档价格里挑了贵的一档，而是**唯一的可行档**。这个结论写进下单指南的目的，是防止后来者在下单页看到价差时”优化”掉这个决定。

顺带厘清一对容易混淆的概念：**经济档与标准版说的是两件事**。”经济档”指制板工艺档位（最小线宽/间距 0.2/0.2 mm、过孔 0.3/0.6 mm 这些能力边界，P1 的设计规则全程对齐它，个别 0.15 mm 细线只在布线攻坚时作为最后手段用于 6 条 <10 mm 补线）；”标准版”指贴装服务档位（哪些料能上贴、能否代插 THT）。P1 的选择是”制板按经济档能力设计 + 贴装必须标准版”——两头各取所需，互不矛盾。

表 7.6: 经济版 vs 标准版 (P1 视角)

维度	经济版	标准版 (P1 选用)
元件库范围	基础库 + 扩展库 (经济档)	全库，含 standard-only 料
U1 (C2913202)	不可上贴	可上贴
THT 代插代焊	无 (或受限组合)	16 只插件全代焊
费用	低	高 (工程费 + 钢网 + 上料)
对 P1 的意义	仅制板工艺档可用	”零手工焊”的唯一路径

7.7 板 5 装 2 与费用区间

打样数量的最终策略是**板 5 装 2**：PCB 制造 5 片（制板起订最低量），贴装只做 2 片（主力 + 备用）。取舍逻辑很直白——打样阶段真正的风险集中在首块贴装板上，多贴三片救不了系统性缺陷，却把物料费翻倍；多出的

3 片裸板留着作补焊与改制实验的板材。费用区间按两个口径记录（表 7.7）：全贴 5 片的完整估算（README 记录，约 550–1150 元），与板 5 装 2 策略的执行口径（约 360–720 元，HANDOFF 记录值）——差额主要来自按片计费的贴装与物料。

表 7.7: 打样费用区间（元，不含运费；上行 README 全贴 5 片估算，末行 HANDOFF 板 5 装 2 记录）

项	区间
PCB（4 层 90×75×5，ENIG）	~50–100
标准版贴装（工程费 + 钢网 + 上料 + THT 代插）	~300–600
元器件物料	~150–300（5 套）
3D 打印（两外壳）	~50–150
合计（全贴 5 片）	~550–1150
合计（板 5 装 2）	~360–720

对照第 3 章的售价定位：单套打样全成本落在数百元区间，与“套件售价 300–500 元 + 白牌厂商的执行器/织物成本”的 B2B 定价结构是自洽的——制造端没有结构性亏损点。

自购清单同样为零：BOM 内 123 件全部由 JLC 供料上贴或代插，不存在“自购不上贴”项。需要自备的只是使用耗材——XH2.54-4P 对线、USB-C 数据线、DC 5V/3A 适配器、气路快插接头与气管、装壳的 M3×6 螺丝 4 颗。

7.8 CPL 坐标与人工核对

贴装坐标文件（CPL）是产线的“放料地图”，出错即批量错位。P1 的 CPL 有三个实操细节。其一，PosY 为负值——这是 KiCad 向下 y 轴的导出方向，JLC 直接接受，无需翻转；与其花时间在导出器里翻轴，不如把这个事实写进注释让核对者安心。其二，-j1c 版在文件头加了 4 行 # 注释说明核对方法，若上传者报表头错误，删注释行再传——原版 flowio-p1-pos.csv 内容完全一致，作为无注释备份保留在同级目录。一份数据、两个文件版本，是用“冗余”换“上传者兼容性”的廉价保险。其三，也是最重要的——**机器解析通过不等于坐标正确**：上传者只校验格式，不校验语义；必须人工抽检，任取 3 行对照 assembly-top.pdf，核对法为 U1 应在板左上、J10–J17 应在板底边一排、J2 应在板右上角。三个锚点分别覆盖“大件 BGA 类”“多件端子排”“小件连接器”三种易错形态。位号共 123 个、全部 top 面；付款前再看一眼预览拼版图：顶面 107 SMT + 16 插件全部有位号标注，即可下单。

7.9 收货验收：从目检到上电

收货后的验收三步走，写在下单指南末尾、与回板工单衔接（A201 首电三测等，见第 13 章）：

- 目检**：U1 模组四侧焊脚、J10–J17 端子无歪斜/浮高、XH 座无损脚——插件工序的缺陷在这一步最容易被肉眼发现；
- 上电**（DC 座或 USB 供电）：TP2 测 +5 V、TP1 测 +3.3 V、LED2（PWR）亮——三个测试点就是第 6 章布线时特意保留的裸铜焊盘；
- 外壳试装**：PCB 入下壳、四角 M3 孔对位（HA–HD）、上壳按键对准 SW1/SW2/SW3——结构对位的最后确认（第 8 章）。

三步全过，制造闭环完成；任何一步失败，证据（照片/万用表读数）按工单机制回流到书稿（第 13 章的“踩坑实录”专栏——fail 的工单同样入书）。

7.10 制造交付的完成定义

“可以下单了”不是一个感觉，而是一张可勾选的门槛表。收尾规格书 §1 为制造交付定义了六条验收门槛（表 7.8），其中“目视检查”一行值得强调：它被用户定为**强制要求**——修复后重新渲染顶/底两视图逐区人工检查。第 6 章天线禁布区那场“自动布线器静默删约束”的事故，正是这条铁律的直接理由。

表 7.8: 制造交付验收门槛（收尾规格书 §1）

验收项	门槛
DRC error 类	0（未连接/间距/孔间距全清零）
悬空过孔	0
外观类	豁免清单记录在案（JLC 可制，见第 6 章）
制造输出	Gerber + 钻孔 + 坐标 + BOM + 装配 PDF, JLC 参数核对通过
外壳	参数化源文件 + 渲染 STL, 四铜柱位与最终 HD 孔位一致
目视检查	顶/底两视图渲染逐区人工检查（用户强制要求）

六条门槛的共同点：每一条都有**机器可复查的产物**——DRC 报告、豁免清单、下单 zip、外壳脚本与渲染图。后人接手时不必信任任何人的记忆，重跑一遍工具链即可验证。这就是“制造输出”与“制造完成”的区别：前者是文件，后者是文件加证据。

把本章散落各节的翻车点汇成一张速查表（表 7.9）——每一行都来自本项目真实踩过或真实核实过的风险，不是想象出来的清单。

表 7.9: 下单失败模式速查（症状 → 根因 → 预防）

症状	根因	预防
BOM 有行“无物料/缺码”	选型时未回填 LCSC 码	覆盖率 100% 断言：缺码即中止生成
部分器件未上贴	BOM 解析后未勾选	逐项确认“经济 + 免经济”全部勾选
CPL 上传者报表头错误	# 注释行不被解析	删注释行重传；无注释备份件常备
坐标镜像/翻转	误改 PosY 符号	三锚点人工抽检（U1/J10–J17/J2）
via-in-pad 虚焊	未声明塞孔工艺	留言栏显式抄录 Type VIII 要求
U1 无法上贴	误选经济版	标准版是硬约束（§7.6）
外壳支撑过剩	打印方向被自动重排	“摆件需人工确认”+ 备注原始方向

本章小结

制造环节的全部输出都是**可复现的文件**而非人工操作：BOM 从 gen_sch.py 的 PARTS 真值源再生成，LCSC 编码在选型时逐颗回填，受限解析器保证生成过程不执行原理图代码；8 处 via-in-pad 的塞孔封帽与 0.20 mm 孔距作为工艺备注写进下单留言；标准版组装是 U1 库属性与 16 只 THT 共同决定的硬约束；板 5 装 2 把打样成本压到约 360–720 元。收货三步验收之后，“可制造”终于变成“已制造”——接下来的问题是给它一个家。

第八章 结构：FreeCAD 参数化外壳

主板造出来只是一块裸板：端子裸露、按键悬空、天线区的净空一碰就没。第 7 章末尾的收货验收第三步“外壳试装”，指向的就是本章的产物——`hardware/flowio-p1/enclosure/` 下的参数化外壳（FCStd/STEP/STL 三格式）与它的孪生网格。外壳设计只有一条核心原则：**关键尺寸全部由 PCB 决定**——板框、铜柱位、连接器坐标、端子节距。板改了，改几个参数重跑脚本，壳就跟着变；反过来，壳上的任何一个孔，都能在板子上指出对应的器件。这条“参数化联动”原则让结构设计不再是画图，而是把装配关系写成代码。

8.1 从 OpenSCAD 到 FreeCAD：一次工具链变更

项目最初选的是 OpenSCAD（收尾规格书 §6 还留着它的决策表），执行中变更为 FreeCAD 1.1（OCCT 内核，变更记录 C7），理由只有一条但足够硬：**规避 OpenSCAD 与 FreeCAD 之间 STL 互操作的丢拓扑/丢参数问题**。数字孪生需要外壳网格 (§8.9)，生产需要 STEP（BREP 精确边界表示），存档需要可参数化重开的工程文件——三种用途、三种格式，从同一个 OCCT 建模内核出发一次产出，不在网格与实体之间来回转换。

`make_case.py` 以 FreeCADCmd 无头运行（`E:/FreeCAD/bin/FreeCADCmd.exe make_case.py`），输出 `flowio-p1-case.FC`（参数化工程）+ `case-bottom.step`/`case-top.step`（生产）+ 两只 STL（打印与孪生）。三种格式不是冗余，而是三种下游的母语（表 8.1）：工程文件记得住“参数”，边界表示记得住“曲面”，网格只记得住“三角形”——下游要什么，就交什么，谁也不必迁就谁。坐标系约定在文件头写死：板左下角为原点， $x \rightarrow$ 右， $y \rightarrow$ 上（板宽 75）， $z \rightarrow$ 高出板面。

表 8.1: 外壳的三格式输出与各自下游

格式	信息保真	下游
FCStd（参数化工程）	全部建模历史与参数	改板后重开改参重跑
STEP（BREP 边界表示）	精确曲面与实体拓扑	生产/检查：实体数、体积、有效性
STL（三角形网格）	离散面片（水密即可）	3D 打印切片、孪生 3D 渲染

8.2 参数区：外壳的全部体质

整个外壳的几何由 20 来个参数完全决定（代码列表 8.1）。壁厚 2.4 mm 与板壁间隙 0.5 mm 是基础量；内腔高 13.5 mm 不是拍的——端子高 11 mm + 裕量，端子（J10-J17）是板上最高的器件，它定了腔高，腔高加板厚加底壁就是壳体总高。铜柱位 ST 四元组直接来自 PCB 的四个安装孔，其中第四个 (68, 65) 是布线攻坚后移动过的 HD 实测位 (§8.6)——外壳脚本不认原理图里的旧默认值，只认修正后的真值。

Listing 8.1: `make_case.py` 参数区（摘自源码）

```
WALL, CLR = 2.4, 0.5          # 壁厚 / 板-壁间隙
BW, BH, PCB_T = 90.0, 75.0, 1.6
INNER_H = 13.5                # 端子高11 + 裕量
TOTAL_H = INNER_H + PCB_T + 1.5
OX = WALL + CLR               # 板原点 -> 壳内壁偏移 (=2.9)
OW, OH = BW + 2*OX, BH + 2*OX # 壳体外廓 95.8 x 80.8
# 铜柱（板坐标，HD 为 PCB 修复后实测位）
ST = [(4, 4), (3, 37), (86, 13), (68, 65)]
PD, PH = 4.2, 5.0             # M3 自攻底孔 / 柱高
SCREW_D = 3.4                 # 顶盖过孔
```

```
# 侧开孔 [面, 板坐标位置, 宽, 高]
CUTS = [("L", 27.0, 10.0, 9.0), ("L", 46.0, 8.0, 8.5),
        ("R", 6.0, 10.0, 4.0), ("R", 22.0, 8.0, 8.5),
        ("R", 32.5, 8.0, 8.5), ("R", 46.0, 8.0, 8.5),
        ("T", 46.0, 8.5, 8.0), ("T", 59.5, 8.5, 8.0), ("T", 73.0, 8.5, 8.0)]
TERM_X = [6.5 + 11*i for i in range(8)] # 底边 8 端子, 节距 11
TY = 68.5
```

壳体本身是三步布尔运算：外盒减内腔得壳体；壳体再减全部侧开孔（`side_cut` 生成穿越壁厚的切割盒，孔中心统一落在离壳底 3 mm 起的安装带上）；最后熔合四根铜柱——外圆柱半径底孔/2 + 1.75 mm，中心钻 4.2 mm 的 M3 自攻底孔，柱高 5 mm。`b2c(x,y)` 这个两行函数是全文档的枢纽：板坐标加 0X 偏移即壳坐标，所有开孔、铜柱、螺丝孔的位置都以板坐标书写——看外壳脚本就能读出器件布局，这是联动的具体形态。

顶盖是壳体的镜像减法再加两个特征：内腔四边各收 0.4 mm（盖与体的配合面——这 0.4 mm 让顶盖内壁恰好扣住底壳外壁，形成无卡扣的过盈配合），顶面开 8×6 通风栅格（每个 5×4 mm，网格原点 0X+12, 0X+18, 节距 9×8 mm），四角开 3.4 mm 螺丝过孔对准铜柱。每个布尔阶段的产物都过一遍 `removeSplitter()`——OCCT 会把 `fuse/cut` 留下的重合面与冗余边清理掉，STEP 里就是一个干净的单实体；这一步省掉的话，下游打印机切片器与孪生加载器都可能被碎面绊倒。

表 8.2 把外壳的完整设计决策汇总成一张表——左三列是收尾规格书 §6 的初版决策（当时还是 OpenSCAD），右侧注明执行中的两处变更（C7 工具链、C9 铜柱位）。决策表与参数区代码互相印证：表里的每个数字都能在代码列表 8.1 里找到同名常量。

表 8.2: 外壳设计决策表（收尾规格书 §6 + 执行变更记录）

项	值	备注
形式	底壳 + 顶盖, M3 自攻铜柱 ×4	盖体 0.4 mm 过盈配合
壁厚/间隙	2.4 mm / 0.5 mm	板到壁间隙
内腔高	13.5 mm (端子 11 + 裕量) + 板 1.6	总高约 18.5 mm
外廓	95.8 × 80.8 × 19 mm	板 90 × 75 + 双壁
铜柱位	HA(4,4) · HB(3,37) · HC(86,13) · HD(68,65)	C9: HD 从 (77,62) 移位
侧开孔	9 孔 + 8 端子带	表 8.4
顶盖	螺丝孔 ×4 + 通风栅 8 × 6	天线区上方不开金属
工具	FreeCAD 1.1 (OCCT)	C7: 由 OpenSCAD 变更
输出	FCStd + STEP + STL	三格式同源

8.3 尺寸传递链：从板到壳的一条路

把“关键尺寸由 PCB 决定”落到实处，就是一条显式的传递链（表 8.3）：板上的每个特征，穿过某个公式，长成壳上的某个参数。这条链上没有一处需要人脑记忆——改板时跑一遍脚本，链自动重算。

表 8.3: PCB 特征 → 外壳参数的传递链（代码常量名见 `make_case.py`）

PCB 特征	外壳参数	代码常量
板框 90 × 75	外廓 95.8 × 80.8 (双壁 + 间隙)	OW, OH
板厚 1.6 mm	腔底垫高、总高 18.5	PCB_T, TOTAL_H
最高器件: 端子高 11 mm	内腔高 13.5 (含裕量)	INNER_H
安装孔 HA-HD 坐标	铜柱位 (HD 取移位后真值)	ST
端子节距 11 mm × 8	底边开孔带 6.5 + 11i	TERM_X
连接器坐标与体量	九个侧开孔的位/宽/高	CUTS
天线净空区	顶盖该区完整塑料面	(约束, 非孔)

传递链的每一环都有一个真实的源头故事。外廓 95.8 来自“ $90 + 2 \times (2.4 + 0.5)$ ”——壁厚与间隙是仅有的两个自由量，其余全部被板锁定。内腔高 13.5 的“端子高 11 + 裕量”里，11 是 BOM 里 CONN-TH_2P 端子的实测体高（孪生网格字典同值），裕量给的是装配公差与器件高度偏差——而器件高度可信的前提，又回到第 7 章的 via-in-pad 塞孔工艺。天线净空一环最特殊：它传递的不是尺寸而是约束——“顶盖在那里”“什么都不做”，这件事必须显式记录，否则下一次改壳的人很可能顺手把通风栅挪过去“利用空间”。

8.4 侧开孔清单：每个孔对着一个连接器

九个侧开孔加八个端子孔，每一个都对应板上一只连接器（表 8.4）。左壁两个：DC-005 电源座（10×9）与一只 XH 座；右壁四个：USB-C（10×4，扁平孔贴合 Type-C 的低矮轮廓）、三只 XH；顶壁三个：J9/J18/J19 的 XH 座；底边则是 8 只螺丝端子的连贯开孔带—— $TERM_X = 6.5 + 11i$ 与板上端子节距 11 mm 严格一致，单孔 9.6×9，让每只端子的螺丝压接面都能从壳外操作。

表 8.4: 侧开孔与端子孔清单（板坐标；对应连接器据收尾规格书 §6）

壁面	板坐标位置	开孔（宽×高 mm）	对应器件
左	27.0	10×9	DC-005 电源座（J1）
左	46.0	8×8.5	XH 座（J8）
右	6.0	10×4	USB-C（J2）
右	22.0 / 32.5 / 46.0	各 8×8.5	XH 座（J5/J6/J7）
顶	46.0 / 59.5 / 73.0	各 8.5×8	XH 座（J9/J18/J19）
底	$6.5 + 11i, i = 0..7$	各 9.6×9	螺丝端子排（J10-J17）

开孔带的高度统一从壳底 3 mm 起量（ $cz = Z0 + 3.0 + hh/2$ ），保证所有线缆插座都在同一水平带内——装配时不用翻转壳体对孔，这对 B2B 客户产线上的装配节拍是有实际意义的细节。

底边端子带还有一个记录在案的取舍：收尾规格书给出过两个方案——每端子一个 11×9 单孔，或整条 82×9 长槽。最终实现取的是单孔方案（9.6×9、节距对齐端子）。长槽的优势是开的程序少、对位要求低；代价是底边整条被切断，壳体底部变成两条悬臂梁——抗弯与抗扭刚度都受损，跌落工况最吃亏。单孔之间保留的塑料梁让底边保持连续结构，代价只是布尔切割的循环次数。对一个要长期挂在康复设备上的产品，结构刚度优先于建模省事，这个优先级不难判断，但把备选方案记进决策文档才是让它可复核的关键。

8.5 公差链：间隙、配合与自攻

外壳的可用性最终取决于几个毫米级的数字（表 8.5）。它们构成一条从“板能放进壳”到“螺丝能拧紧”的公差链——任何一环吃紧，装配体验都会崩坏。

自攻连接值得多说一句，因为它是 3D 打印外壳最常被低估的环节。M3 自攻螺丝切入 4.2 mm 底孔时，塑料被挤压成内螺纹——连接强度完全取决于孔周塑料的周向完整性：孔太小，切入阻力过大直接滑丝；孔太大，牙型不饱满，拧两轮就秃。4.2 mm 是 ABS/PLA 上 M3 的标准底孔径，柱外径 7.7 mm 保证牙区外圈还有约 1.75 mm 厚的实心塑料受力。柱高 5 mm 对板厚 1.6 mm 的咬合长度约 3.4 mm——足够 M3×6 螺丝建立稳定预紧。这些数字单看都不起眼，但任何一条被“顺手更改”，装壳那天的返工成本都是整只外壳。

8.6 HD 移位：一个安装孔的 (77,62)→(68,65)

铜柱位 (68, 65) 的第四个坐标背后是布线攻坚（第 6 章）里最干净利落的一次修复。故事从一处坏过孔开始：route_pcb 阶段残留的手工 +5V 过孔 @ (78.36, 60.5) 紧贴 D10（+5V 肖特基）与 HD 安装孔，一口气命中 4 条孔间距、1 条悬空、1 条未连接违规。删掉坏过孔后，剩余冲突定位到：D10.1 焊盘孔与 HD 安装孔 (77, 62) Ø3.2 的边距为 0.000 mm——两孔相切，且 HD 堵死了 D10 的出线走廊。

表 8.5: 外壳公差链（数值均出自 make_case.py 参数区）

特征	值	作用
板壁间隙 CLR	0.5 mm	板入壳的滑动配合；吸收板边毛刺与打印偏差
壁厚 WALL	2.4 mm	强度主体； $\geq 1.2\text{ mm}$ 打印底线的两倍裕量
盖体配合	0.4 mm 收边	顶盖内壁扣住底壳外壁，无卡扣过盈配合
自攻底孔 PD	4.2 mm	M3 自攻标准底孔；柱体被螺丝牙切入
柱外径	约 7.7 mm	底孔半径 + 1.75 mm 壁——保证牙区周向塑料完整
顶盖过孔 SCREW_D 开孔中心带	3.4 mm 离壳底 3 mm	螺丝穿过不咬合；拆盖不松动主板 所有线缆插座同一水平带

此时有两个选项：挪 D10（焊盘有网络有走线，牵一发动全身）或挪 HD。裁决是挪 HD，理由是排障规格书里那句朴素的风险评估——**安装孔无网络无走线，移动零风险**。move_hd.py 用与过孔落点相同的全障碍感知搜索（焊盘含孔、走线、过孔、NPTH、板边、禁布区全部纳入）做全区扫描，(68, 65) 以净空 5.6 mm ($\varnothing 3.2$ 孔 + 1.2 mm 铜净空) 成为全场最优点，同时避开了端子排区 ($y \geq 66$) 与 D10。

这次移位暴露了一个结构性盲区：**DRC 不会替你保护安装孔**。孔间距规则查的是“孔与孔的几何距离”，但 HD 作为无网络器件，它的“合理位置”（四角对称、避让器件、给铜柱留塑料壁厚）不在任何电气规则的定义域里——它在机械设计者的脑子里。所以移位后的真值必须显式传递：standoffs.txt 记录四柱终位（HA 4,4 / HB 3,37 / HC 86,13 / HD 68,65），make_case.py 的 ST 数组与之一致。任何一侧单独改动都会造成“板上孔、壳上柱”对不上的装配事故——参数化联动的纪律在这里比任何地方都重要。

8.7 铜柱、自攻与装配动线

装配动线四步（收货验收第三步的展开，对应验收工单 A220）：PCB 入下壳 → 四角 M3 孔对位（HA-HD）→ 上壳按键对准 SW1/SW2/SW3 → 四颗 M3×6 螺丝自攻入铜柱。三个几何参数各司其职：铜柱底孔 4.2 mm 是 M3 自攻螺丝的标准底孔（塑料柱体被螺丝牙切入形成螺纹）；柱高 5 mm 保证板厚 1.6 mm 之后仍有足够咬合长度；顶盖过孔 3.4 mm 让螺丝穿过而不咬合顶盖——拆卸顶盖不松动主板。

这里有一个跨章的呼应，也是“全栈联合设计”的注脚：第 7 章的 8 处 via-in-pad 塞孔封帽工艺，与外壳的内腔高度裕量直接相关。内腔高按“端子 11 mm + 裕量”设定，这个设定成立的前提是器件以设计高度稳定焊接——盘内孔若不塞孔，焊膏流失造成器件沉降或立碑，最高的器件高度就不再可信，裕量设计随之失效。工艺决定焊接质量，焊接质量决定器件高度，器件高度决定壳体尺寸——链条跨过 PCB 工艺、贴装与结构三个专业，这正是把三者放进同一个仓库设计的原因。

8.8 天线净空与通风栅

顶盖有两处“刻意不做”的设计。其一，ESP32 模组天线区（板坐标 $x 19.5\text{--}36.5 / y < 6.4$ ，即第 6 章的禁布区）正上方的顶盖不开金属、允许塑料——塑料对 2.4 GHz 电磁波近乎透明，而 PCB 侧已保证天线正下方四层无铜；顶盖的唯一责任是不在这里加金属件。其二，通风栅格的位置刻意远离该区：栅格起于壳坐标 $y = 20.9$ （板坐标 $y \approx 18$ ），与天线区之间留出完整塑料面。8×6 个 5×4 mm 的栅格孔承担泵与阀长时间工作的散热——8 路阀线圈满负荷时每只约 0.36 A 电流（5V/14Ω），腔体积热不可忽视。

8.9 孪生网格：make_mesher.py 与坐标映射的证伪

外壳 STL 的第二去向是数字孪生的 3D 装配视图（第 4 章图 4.1，五部件爆炸见图 8.1）。make_mesher.py 把 PCB、器件阵与外壳组装成同一坐标系下的网格包，输出的 assembly.json 驱动前端爆炸视图。

这份脚本最有教学价值的是坐标映射的证伪过程。KiCad 导出的 pos.csv 用向下 y 轴，板坐标用向上 y 轴，两者之间最可疑的变换有两种： $y = -\text{PosY}$ 或 $y = -\text{PosY} + 75$ （75 是板宽）。脚本没有选一个“看起来对”的，而是用三只位置已知的器件做锚点校验——J10 端子应在 (6.5, 68.5)、J2 USB-C 应在 (86, 6)、J1 应在 (5.5, 27)：

Listing 8.2: make_mesher.py 的坐标映射校验（摘自源码）

```
def verify_mapping(parts):
    """已知器件校验 pos.csv->板坐标映射, 定稿后供建模使用."""
    refs = {p["ref"]: p for p in parts}
    checks = [("J10", 6.5, 68.5), ("J2", 86.0, 6.0), ("J1", 5.5, 27.0)]
    for ref, ex, ey in checks:
        p = refs[ref]
        x, y = p["x"], -p["y"]      # 候选映射: x=PosX, y=-PosY
        ok = abs(x - ex) < 0.01 and abs(y - ey) < 0.01
        assert ok, "Ymapping verification failed for %s" % ref
    print("[map] mapping FIXED: x=PosX, y=-PosY"
          "'-PosY+75' rejected by J10/J2 checks")
```

+75 翻转假设在 J10/J2 两个锚点上立刻露馅（会给出 143.5/81 这类板外坐标），被断言拒绝。三锚点、0.01 mm 容差、失败即中止——这与第 6 章“目视检查抓出天线禁布区被删”是同一个信条：凡是人推定的几何关系，都要有程序化的证伪手段。

器件几何按封装关键字查表（H 字典覆盖 CONN-TH、TYPE-C、WROOM、XH、SOP、CDRH 等 17 类，外加 L1 电感与 SOIC 两个别名），90° 旋转交换宽深，top 面器件自板面向上生长、bottom 面自板底向下生长；全部器件合成一个复合体再统一网格化（LinearDeflection 0.4 / AngularDeflection 0.5），逐一断言 isSolid。BOM 无底面器件时，parts_b.stl 写入 84 字节的空二进制占位——下游加载器不需要特判。assembly.json 登记五部件（上壳/器件阵-顶/PCB/器件阵-底/下壳）、整机包围盒 95.8×80.8×19 mm 与装配备注（M3 螺丝自攻入下壳铜柱）。五部件各自的爆炸位移也登记在同一文件里：上壳 +28 mm、顶面器件阵 +12 mm、PCB 不动、底面器件阵 -8 mm、下壳 -16 mm——前端爆炸滑杆从 0 拉到 100%，装配体按这条轴线分解，装配顺序（下壳 → 板 → 器件 → 上壳）即拆卸顺序的倒放。

同一份 pos.csv 还是前端气流可视化的数据源：气流拓扑与热点卡片（flows/hotspots.json）由脚本从坐标文件同源生成，并带同源断言防止拓扑与器件位漂移——充气时端子喷出的那束青色粒子（第 13 章图）飞出的位置，就是 pos.csv 里那只端子的真实坐标。坐标文件驱动三处消费（贴装、网格、气流拓扑），一处生成、三处同源，这是“真值源”思想在结构-软件边界的延伸。

网格管线的完整产出是六个文件加一个清单（表 8.6），每个都有断言把守：面片数非零、isSolid 为真——空文件或破面网在生成当场就会让脚本失败，而不是等到前端渲染时才白屏。

8.10 3D 打印下单与水密自检

外壳走 JLC 3D 代打，与 PCBA 同一笔物流。材料三档的取舍直接抄自下单指南：PLA 默认推荐——结构强度够、最便宜，上壳约 34.4 cm³、下壳约 29.8 cm³；MJF 尼龙 PA12 是进阶选项，强度/韧性/耐温最佳，适合长期按压的卡扣与螺柱（自攻铜柱正是长期受力件），价高；树脂表面细但脆，不建议用于承载外壳。

下单前两道自检已写进脚本与指南。其一，壁厚：全壳最薄处 ≥1.2 mm，水密网格已验证——最小壁厚是

表 8.6: make_meshes.py 产出清单 (firmware/twin/meshes/)

文件	内容	去向
case_bottom.stl	下壳（含铜柱）	3D 爆炸视图底层
parts_f.stl	顶面器件阵复合体	爆炸视图（+12 mm 层）
pcb.stl	90×75×1.6 板体	爆炸视图基准层
parts_b.stl	底面器件阵（或空占位）	爆炸视图（-8 mm 层）
case_top.stl	上壳（通风栅）	爆炸视图顶层（+28 mm）
assembly.json	五部件清单/颜色/爆炸位移/包围盒	前端装配数据源

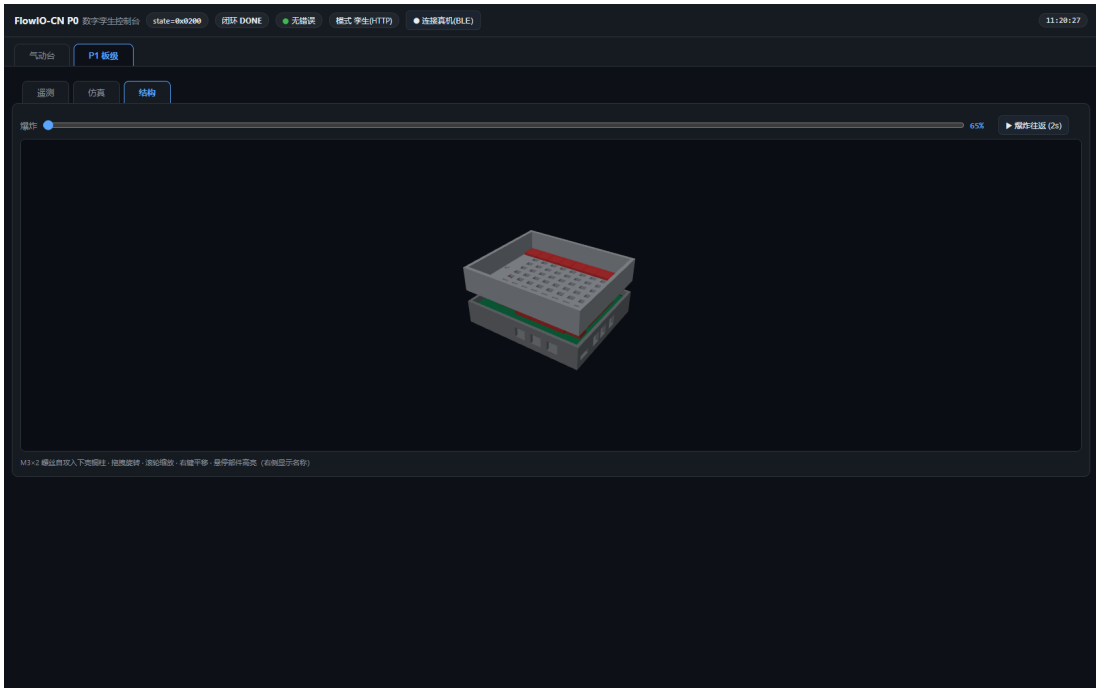


图 8.1: P1 装配的五部件结构视图：上壳（通风栅）、顶面器件阵、PCB、下壳（铜柱）——由 make_meshes.py 从 pos.csv 与外壳 STL 同源生成

FDM 打印不断料的底线。其二，**网格质量**：STL 格式本质是三角形面片的无序集合，“看起来像个壳”与“数学上是个封闭实体”是两回事——不封闭的模型在切片器里表现为随机的破面与补洞。`check_case.py` 对两只 STL 断言 `isSolid()`（水密），对两只 STEP 断言实体数、体积与 `isValid()`：

Listing 8.3: `check_case.py`：水密与实体有效性自检（全文仅数行）

```
for name in ("case-bottom", "case-top"):
    m = Mesh.Mesh(); m.read(name + ".stl")
    print("CHK", name, "STLfaces", len(m.Facets),
          "watertight", m.isSolid())
    sh = Part.Shape(); sh.read(name + ".step")
    print("CHK", name, "STEPsolids", len(sh.Solids),
          "vol_cm3", round(sh.Volume/1000, 1), "valid", sh.isValid())
```

摆放方向是打印质量的最后变量：上壳的通风栅面朝上平放可让栅格孔不生支撑。JLC 下单页默认可能自动转向——指南的对策是勾选“摆件需人工确认”，或在备注栏写“保持模型原始方向”；层高 0.1–0.2 mm 均可。

外壳与 PCB 一样执行**目视检查铁律**：建模完成后三次渲染截图逐区人工过目（收尾规格书 §6 的输出要求），配合 `make_meshes.py` 里逐网格的 `check_mesh` 打印（面片数 + `isSolid` 断言）。检查清单包括：四铜柱与安装孔对位、九个侧开孔与端子带的位置、通风栅避让天线区、顶盖螺丝孔与铜柱同轴。第 6 章“自动布线器静默删掉天线禁布区”的教训在这里同样适用——布尔运算是脚本自动执行的，每一次重跑都可能引入它不报告的几何退化，人眼逐区过图是最后一道不可替代的闸。

本章小结

外壳不是 PCB 的包装，是装配关系的代码化：参数区把壁厚、腔高、铜柱、开孔全部收敛为可重跑的常量，板改参数改，壳随板动；HD 安装孔从 (77,62) 移到 (68,65) 的故事说明无网络器件不受 DRC 保护、真值必须显式传递；坐标映射三锚点证伪、水密自检与天线净空的“刻意不做”，与孪生网格的同源生成一起，让结构与电路、软件共享同一套“可验证才交付”的纪律。至此，电路（5、6 章）、制造（7 章）、结构（本章）三块硬件拼图合龙——余下的章节属于软件。

第九章 固件：ESP-IDF 分层架构

固件是整个项目里最“怕等”的部分：PCB 在产线上飞（第 7 章），执行器还没到货，凭什么先写固件？答案在第 4 章已经埋下——孪生先行。但孪生能跑固件的前提，是固件本身长得分层：核心逻辑不依赖任何一块真硬件。本章讲 FLOWIO-CN 固件的四层架构：一份 `pn_core` 如何同时活在三个世界（ESP32-S3 真机、主机 gcc 测试、QEMU 仿真），CLI 十三条命令如何兼任调试入口与验收工具，TCA9548A 复用器如何用“函数注入”保持纯逻辑，以及串口与 BLE 如何汇入同一个命令分发器。

9.1 四层架构与两个“没有”

固件仓库（`firmware/`）的目录结构本身就是架构宣言：

Listing 9.1: `firmware/` 目录结构（README 摘录）

```
components/  
  pn_core/      纯逻辑层（动作/闭环/协议）——不依赖任何平台，主机可测  
  pn_hal_esp32/ ESP32-S3 硬件层（LEDC 阀/泵 + RMT 舵机 + I2C 传感器）  
main/          app_main: 10ms 控制任务 + 串口 CLI  
tests/         主机单元测试（gcc/cmake/ninja，无需 ESP-IDF）
```

自上而下四层（表 9.1）。分层的判据只有一条：`pn_core` 目录里的 C 文件只 `include` 自己的头文件与标准库，一个 ESP-IDF 头都不出现。这不是洁癖，而是两个“没有”的工程前提——没有开发板也能改逻辑（主机 gcc 直接编译测试），没有仿真器外设也能冒烟（QEMU 不仿真 RMT/I2C，逻辑层照跑）。

表 9.1: 固件四层架构（`firmware/README.md`）

层	代码	职责
应用层	<code>main.c</code>	串口 CLI 主循环 + 10ms 控制任务 + 呼吸演示
协议层	<code>proto.c</code>	5 字节帧 [0xA5][cmd][ports][pwm][crc8] 的流式解析与构造
动作层	<code>actions.c / closedloop.c</code>	端口三胞胎/方向阀/泵、状态字、节能、超压保护、压力闭环
HAL	<code>hal_if.h</code> 契约	<code>pn_hal_esp32</code> 实现（目标机）/ <code>mock_hal</code> （测试）双实现

HAL 契约小得出奇——四个函数指针就把“什么是一块 FlowIO”定义完了：`valve_write`（阀/泵占空比）、`pump_write`、`sensor_read`（kPa）、`now_ms`（时基）。真机实现把它们接到 LEDC/RMT/I2C；测试的 `mock_hal` 把它们接到内存里的假寄存器。动作层的一切语义——阀开多久了（`pn_valve_open_ms`）、节能降额是否到期——都从这个 4 函数契约里长出来，因此换一个 HAL 就换一个世界，而逻辑一行不改。

9.2 主机 33 测试：逻辑层的证据链

`firmware/tests/` 是一套零依赖极简框架（`test_util.h` 只有两个宏：`CHECK` 与 `RUN_TEST`），用 `cmake + Ninja` 在主机上编译，不需要装 ESP-IDF。当前 33 个测试的分布：动作层 10（`test_actions.c`）、闭环 7（`test_closedloop.c`）、协议 4（`test_proto.c`）、TCA9548A 8（`tca_tests.c`）、BLE 载荷 4（`ble_tests.c`）。实跑输出：

Listing 9.2: 主机单元测试实跑（gcc/cmake/ninja, 2026-10 当前状态）

```
$ cd firmware/tests && cmake --build build-test && ./build-test/pn_tests.exe
=== pn_core host unit tests ===
[ RUN ] t_inflation_sequence    (动作层: I 的阀位组合与状态字)
[ RUN ] t_overpressure_failsafe (安全: 超压硬规则全关)
[ RUN ] t_crc_known_vector      (协议: CRC-8/0x07 定值防手滑)
[ RUN ] t_leading_garbage_resync (协议: 垃圾字节后重同步)
...
=== 33 tests, 0 failed ===
```

数字背后有一段演化：README 里写的”21 tests 全绿”是 S3 时代的状态；S4 加 TCA9548A 驱动补 8 个、S5 加 BLE 载荷编解码补 4 个，长到 33。测试策略是**改逻辑必跑**——任何 pn_core 的修改，先在主机上跑这 33 个断言，再谈烧录。这份”主机可测”的纪律最终成为验收工单 A105 的软件侧证据（第 13 章）。

9.3 动作层：阀位组合与状态字

动作层（actions.c）把气路知识编码成五个组合动作，每个动作就是一张阀位真值表（表 9.2）。其中两处细节来自对 370 迷你泵气路的理解：抽真空走**排气位**（泵吸/充同侧，负压路径经排气位对端口侧抽取）；释放是**完全被动的**（泵停、进气关、端口开、排气通大气，不用泵抽）。

表 9.2: 五个组合动作的阀位组合（actions.c 语义）

动作	进气阀	排气阀	端口阀	泵
充气 I	开	关	开	起（pwm=0 自替 255）
抽气 V	关	开	开	起
释放 R	关	开	开	停（被动放气）
保压 S	关	关	关	停（隔离保压）
诊断保压 H	关	关	开	停（单一密封容积）

H 值得单独说明：它与 S 的唯一区别是端口阀保持通——泵侧密封后汇流管与下游执行器连成一个密封容积，端口侧泄漏因此可被汇流管传感器观测。这正是第 4 章 v2.1 物理修正（端口独立节点）与出厂筛选工况的固件侧对应物。

状态字是全系统的通用语言（CLI 的 T、BLE 遥测帧、上位机徽章都消费它）：bit0-4 = 端口 1-5 开，bit5 = 进气阀，bit6 = 排气阀，bit7 = 泵，bit9 = 传感器在线，bit15 = 错误；协议层另预留设备 ID 字段位（多机同步，对标 FlowIO JS API）。valve_drive() 是唯一写状态字的地方——写占空比的同时登记 s_open_since 时间戳，pn_valve_open_ms 由此提供”开了多久”，闭环超时与节能降额都依赖它。

安全有三条，全是**硬规则**：上电 pn_init 先把所有阀与泵写零（常闭阀 + duty=0，最坏情况也只是不动作）；超压保护 pn_check_overpressure 任何上层不可关闭，触发即 pn_stop_action(ALL) 全关；真机上 main.c 另加泵连续运行 120s 护栏（防干转/过热，超时强制停泵）。第三条原则写给未来的智能层：TinyML 之类永远是并行观察者，不进控制路径——后来泄漏检测整体下线（第 4 章），控制路径果然一行未动。

9.4 CLI：十三条命令的调试与验收两用

CLI 分发器 pn_core/cli.c 从 main.c 抽出后就是双宿主共用——真机串口、虚拟设备、孪生 DLL 走同一个 pn_cli_process_line()。表 9.3 是完整命令表：前九条对标 FlowIO 动作语义，后四条是本项目自己的运维入口。

三条历史注脚。其一，L 命令仍在表外活着：TinyML 泄漏检测 2026-10-02 下线后，L 改为应答 leak=off (TinyML retired 2026-10-02)——命令号保留不 404，旧客户端与 BLE 脚本不至于拿到?。这是”接口先冻结、

表 9.3: 串口 CLI 命令表 (cli.c + main.c, 115200-8N1)

命令	示例	语义
I <ports> <pwm>	I 1 255	充气 (ports 为位掩码, 1-5 位)
V <ports> <pwm>	V 1 200	抽气
R <ports>	R 1	释放 (被动放气)
S <ports>	S 1	停止/隔离保压
H <ports>	H 1	诊断保压 (端口保持通)
O/C <ports>	O 7	只开/关端口阀
G <ports> <kPa> <s>	G 1 40 0	闭环充气到目标压力
X		闭环复位 (完成门锁解除)
F		三步自检 (见下)
P		读全部传感器 (kPa)
T		打印状态字与错误码
W		I2C 全总线扫描 (bring-up 诊断)
M <gpio> <usec>	M 17 2500	舵机信号诊断 (LEDC 时基发脉冲)

语义可演化”的小型示范。其二，F 自检三步（传感器探测 → 3 端口阀 + 进/排气阀咔哒 → 泵 1.5 s 后汇流管 ΔP , > 2 kPa 判 PASS 否则 FAIL/LEAK）是回板当天 CLI 的第一件事，也构成 QEMU 冒烟的判定项。其三，cli.c 里没有一行 printf 直发应答——所有应答走 cli_out(): 先 vprintf 到串口，再把同一行喂给可注入的 resp_sink。这个看似多余的挂钩，正是第 10 章 BLE resp 特征的接缝：CLI 应答行原样变成 GATT 通知。

9.5 TCA9548A：函数注入保住的纯逻辑

双压力传感器共用固定地址 0x58 (XGZP6897D-C 选型码即地址，第 4 章)，必须经 TCA9548A 八通道复用器分时选通。麻烦在于：选通是 I2C 写，而 pn_core 不许碰 I2C。解法是注入——纯逻辑层只定义函数指针类型 tca_i2c_write_fn 与编码规则，真机在初始化时把硬件写函数绑进来：

Listing 9.3: tca9548.c 的编码与注入 (摘自源码)

```
static tca_i2c_write_fn s_fn; /* 真机注入; NULL=主机 mock 模式 */

uint8_t tca_encode(uint8_t ch)
{
    return (ch < TCA_CH_MAX) ? (uint8_t)(1u << ch) : 0;
}

void tca_bind(tca_i2c_write_fn fn) { s_fn = fn; }

tca_err_t tca_select(uint8_t ch)
{
    if (ch == TCA_MANIFOLD) { /* 汇流管直连: 写 0x00 断开全部下游
                               * (旁路 encode——0xFF 编码本为非法 0) */
        if (s_fn) return (s_fn(TCA_ADDR, 0x00) == 0) ? TCA_OK : TCA_ERR_BUS;
        s_mock_cur = TCA_MANIFOLD;
        return TCA_OK;
    }
    uint8_t code = tca_encode(ch);
    if (code == 0) return TCA_ERR_CHAN;
    if (s_fn) return (s_fn(TCA_ADDR, code) == 0) ? TCA_OK : TCA_ERR_BUS;
    s_mock_cur = ch; /* mock 模式: 仅记录当前通道 */
}
```

```

    return TCA_OK;
}

```

一个指针，两种世界。`s_fn == NULL`时整套逻辑跑在 mock 总线表上——每通道可挂最多 4 个模拟设备地址（`tca_mock_attach`注入 `Pa×10` 读数），`tca_scan` 数地址、`tca_read_pa` 读首个传感器、空通道返回 `TCA_ERR_BUS`——这正是 8 个主机测试（`tca_tests.c`）所断言的行为：编码边界（`tca_encode(0)==0x01`、`tca_encode(5)==0`）、通道越界、断线错误码、同通道双地址去重。真机侧 `main.c` 一行完成注入：

Listing 9.4: `main.c` 的绑定（摘自源码，S4）

```

pn_init(pn_hal_esp32_init(), PN_CFG_GENERAL);
tca_bind(tca_hal_write); /* pn_core TCA9548A 逻辑层绑定真机 I2C 写（QEMU 下恒 -1，无害） */

```

真机实现 `tca_hal_write` 有一次小的工程判断：任务书示例给的是 legacy API `i2c_master_write_to_device`，但它与 `hal_esp32.c` 既有的 new-driver 总线句柄互斥——于是改为复用既有 `s_tca_dev` 句柄走 `i2c_master_transmit`，“用既有 `i2c master` 句柄”的意图不变。所有真机 I2C 出总线都收敛到这一个函数，QEMU 或器件缺失时返回 -1，逻辑层把它翻译成 `TCA_ERR_BUS`，优雅降级而非挂死。传感器读取路径（`xgzp_read_kpa`）同样是“先 `tca_hw_select` 切通道，再对 0x58 从寄存器 0x04 连读 5 字节，24 位补码和除以 2^{21} 乘量程”——读法与换算是第 4 章表 4.5 三处手册修正的直接落地。bring-up 期的 W 命令（全总线扫描：主总线全段 + CH0/CH1 下游段）则把这套硬件拓扑变成一条可粘贴进验收工单 A206 的日志。

9.6 pn_cmd_feed: 串口与 BLE 的同一入口

S5 之前的命令入口只有串口 `fgets`；BLE 来了之后，最偷懒的做法是在 GATT 回调里再写一个解析器——那意味着两份文法、两份 bug。实际做法是给所有命令入口修一条共用河道 `pn_cmd_feed(buf, len)`：字节流进来，先识别是 0xA5 二进制帧还是 ASCII 明文行，帧翻译成等价 CLI 行（翻译表见第 10 章表 10.2），行聚齐后统一进 `pn_cli_process_line()`。跨多次 write 分片的行也能拼上——BLE 的 MTU 分段对上层完全透明。

Listing 9.5: `main.c` 的 `pn_cmd_feed` 核心：帧/行双模式与线程串行化（摘自源码）

```

void pn_cmd_feed(const uint8_t *buf, size_t len)
{
    char cooked[24];
    for (size_t i = 0; i < len; ++i) {
        uint8_t b = buf[i];

        /* 二进制帧模式：行缓冲空时见 0xA5 进入，收满 5 字节校验后翻译 */
        if (s_in_frame || (b == PN_PROTO_MAGIC && s_line_n == 0 && s_frame_n == 0)) {
            s_in_frame = true;
            s_frame[s_frame_n++] = b;
            if (s_frame_n < sizeof(s_frame)) continue;
            s_frame_n = 0; s_in_frame = false;
            if (s_frame[4] == pn_proto_crc8(s_frame, 4)) { /* 坏帧静默丢弃 */
                frame_to_line(s_frame, cooked, sizeof(cooked));
                xSemaphoreTake(s_cmd_mtx, portMAX_DELAY);
                dispatch_line(cooked);
                xSemaphoreGive(s_cmd_mtx);
            }
            continue;
        }
    }
}

```

```

    if (b == '\n' || b == '\r') {
        if (s_line_n) {
            s_line[s_line_n] = 0; s_line_n = 0;
            xSemaphoreTake(s_cmd_mtx, portMAX_DELAY);
            dispatch_line(s_line);
            xSemaphoreGive(s_cmd_mtx);
        }
        continue;
    }
    if (s_line_n < sizeof(s_line) - 1) s_line[s_line_n++] = (char)b;
    else s_line_n = 0; /* 超长行整行丢弃 (BLE.md: 命令行 63B) */
}
}

```

三处细节决定了它扛得住三个并发调用者（串口主循环、BLE cmd 任务、演示任务）。其一，互斥锁 `s_cmd_mtx` 把分发临界区缩到单行——三个线程谁先拿到谁先执行，状态字不会写一半。其二，帧模式只在行缓冲为空时由 `0xA5` 触发：半截命令 `I 1` 后跟一个恰好等于 `0xA5` 的字节，不会把状态机拖进帧模式；坏 CRC 的帧静默丢弃，与 `proto.c` 流式解析器同一策略（协议层的解析器另有“垃圾紧贴真帧”的缓冲内重同步，深度有界不递归）。其三，超长行整行丢弃而不是截断——截断会产生一条语义全变的合法命令，丢弃最多让调用方重试。

喂入端还有两个小机关。真机 `stdin` 需要 UART 驱动 + VFS 桥接（默认 `console` 只配输出，`fgets` 根本收不到字节——2026-09-23 真机首次点亮的实证；QEMU 的 UART 模型走轮询读，须跳过以免驱动中断差异）。另一个是 `PN_BOOT_DEMO`：充电宝独立供电时没有串口对端，上电 30s 内未按住 `BOOT` 键就自动跑呼吸演示循环（`I 6s` → `S 3s` → `V 6s` → `S 3s`）——演示任务自己也是 `pn_cmd_feed` 的第三个调用者，吃自己的狗粮。

9.7 10ms 控制节拍：七件事与一盏灯

`control_task` 以 10ms 周期轮转，每拍做七件事：闭环 `pn_inflate_to_tick()`（完成/超时/传感器错三态打印）、泵 120s 护栏、阀保持节能 `pn_optimize_power`（参数改读 BLE Config 服务的 RAM 镜像，见第 10 章）、超压保护、舵机 50Hz 脉冲流刷新、WS2812 状态灯节拍、BLE 遥测组帧（每 10 拍一次，10Hz）。闭环本体（`closedloop.c`）是个干净的非阻塞状态机：G 启动后不再占用主循环，每拍读传感器、比较目标、检查超时；完成门锁 `s_completed` 要求显式 X 复位才能再次启动；到达目标时先取进气阀开启时长再停止动作——停止会清时间表，顺序反了 `elapsed_ms` 就永远是零。启动时若目标端口已在充气（状态字匹配）则沿用当前动作不重启，与 FlowIO `inflateP` 同款语义。

WS2812 状态灯（GPIO48，板载 1 颗地址型 RGB）是这拍节拍里最“产品”的一件事。设计把模式设置与节拍驱动分开：`board_led_set(mode)` 只改模式变量，呼吸/闪烁动画由每 10ms 的 `board_led_tick_10ms()` 推进相位，且只在 RGB 变化时才碰 RMT 总线。五模式映射见表 9.4。

表 9.4: WS2812 状态灯模式映射（`hal_esp32.c`）

模式	名称	视觉
0	IDLE	呼吸蓝：2s 三角波 0 → 255 → 0（相位 10ms 步进）
1	RUNNING	常绿
2	HOLD	常青（绿 + 蓝）
3	ERR	红闪 2Hz（占空 50%）
4	OTA	紫闪 1Hz（红 128 + 蓝 255）

S4 阶段的实际接线是简化映射：`board_led_set(pn_get_state() ? 1 : 0)`——任一执行器动作即 RUN-

NING 绿，否则 IDLE 呼吸蓝；HOLD/ERR/OTA 三档留给后续（ERR 现由状态字 bit15 承担信息职责）。这盏灯背后有一条 RMT 通道预算的教训链：ESP32-S3 共 4 个 RMT TX 通道，舵机吃 3、状态灯占第 4；每通道 `mem_block_symbols` 取 48——先是照抄示例写 64，结果级联相邻通道内存、每只舵机吃掉 2 个 TX 通道，第三只直接 `no free tx channels`；改 32 又被驱动拒绝（“must be even and at least 48”）；48 恰是单通道 RAM 且够用。同源的第二条教训：发送队列深度初值 1 与 20ms 发送节奏竞争，新帧到达时上一帧未发完、队列满静默丢弃，舵机信号大部分时间停发——白线被内部上拉顶到 2.7V、LED 永闪的“灵异”根因；队列深 4 后稳态只用 1-2，彻底消除竞争。这两条都记在 `hal_esp32.c` 注释里，连同 2026-09-23/24 的实证日期。

9.8 QEMU 冒烟：没有射频与外设的全固件测试

主机测试覆盖 `pn_core`，QEMU 冒烟覆盖整只固件：真 `app_main`、真 FreeRTOS 调度、真启动流程，只缺射频与 RMT/I2C 外设。`qemu_smoke.sh` 的流程是 `merge_bin` 合并三段镜像 → 填充 16MB → QEMU 启动（-m 8M + `ssi_psram,is_octal=true` 才能匹配 N16R8 的 Octal PSRAM）→ 延迟喂 CLI 命令 → 九项 `grep` 判定：启动横幅、PSRAM 8MB 初始化、F 自检三步、I 1 255/H 1/S 1 应答、L 路由。九项全过才算冒烟通过（工单 A105）。

两个坑写进了脚本头注释。其一，启动期间 UART 提前收到字节会触发 QEMU 中断竞态崩溃（`esp_task_wdt_init` 前后窗口）——命令延迟 10s 再发（启动约 2s 完成，余量充足），仍偶发则重试至多 5 次。其二，QEMU 的 `esp32s3` 模型不仿真 RMT/I2C/BT 控制器——固件用 `efuse` 芯片版本探测（QEMU 报 v0.0，真实 S3 无此版本）：舵机/传感器初始化跳过、WS2812 跳过、BLE 跳过，[HAL] `QEMU detected` 打印在案，CLI 与逻辑层照常。这个“探测 + 优雅降级”的模式在真机上也兜底：器件缺失时闭环照常报传感器错，而不是挂死在驱动里。

本章小结

分层是本章唯一的主题词：`pn_core` 零平台依赖换来主机 33 测试与 QEMU 冒烟两道防线；HAL 四函数契约定义了“什么是 FlowIO”；TCA9548A 用函数注入让复用器逻辑既活在 `mock` 总线表上、又活在真机 I2C 上；`pn_cmd_feed` 把串口与 BLE 收敛为同一入口，帧与行、三个线程、两种宿主共享同一份文法。CLI 十三条命令既是开发期的手，也是验收期的尺——下一章把其中无线的那一半（BLE 四服务契约）讲完，第 13 章的硬件工单将逐条回收这里埋下的验证点。

第十章 BLE 无线：契约先行

无线链路是最容易“各写各的”的地方：固件作者想的是 GATT 表怎么注册，前端作者想的是 API 怎么调用，两边对载荷字节序的理解差一位，链路就通了但数据全错。FLOWIO-CN 的对策是**契约先行**：先有 `firmware/twin/BLE.md`——UUID 怎么派生、服务怎么组织、每个字节什么含义、回调里允许做什么不允许做什么，全部写成文档并标注“契约冻结”；固件实现（`ble_twin.c`）、载荷编解码（`ble_frame.c`）、浏览器端（`webapp/js/ble.js`）都以它为唯一真源，改字段先改文档。本章逐层展开这份契约：UUID 命名空间、四服务表、两种载荷的字节流，以及“回调不阻塞协议栈”的队列解耦设计。

10.1 架构：命令不新开解析器

BLE 接入的第一条设计决策写在 `BLE.md §1` 第一句：**命令路径不新开解析器**。浏览器经 Web Bluetooth 写入 `cmd` 特征的字节，与真机串口 `fgets(stdin)` 收到的字节，汇入同一个 `pn_cmd_feed()`（第 9 章 §9.6）——行缓冲聚合、0xA5 帧翻译、`pn_cli_process_line()` 分发，一条不差。由此得到三宿主行为一致：串口、BLE、孪生 DLL 对同一条命令的应答逐字节相同。遥测方向对称：10 ms 控制节拍每 100 ms 打包一个 20 字节状态帧，`notify_en` 特征做应用级开关，CCCD 订阅是第二道必要条件。

10.2 UUID 命名空间与四服务

自定 128 bit 基址写死于契约里（改即破坏兼容）：`f10a5c00-0000-4b1e-9c2d-8e3a1b0f0000`。派生规则极简：把基址末 4 个 hex 位替换为顺序编号后缀（0001/0002/...）。一个基址、九个后缀，定义了全部自定服务与特征（表 10.1）；`DeviceInfo` 则复用 SIG 标准 UUID（服务 0x180A），只有板卡版本号无标准号可用，占自定 0009。

表 10.1: BLE 四服务表（BLE.md §2-§3；基址...8e3a1b0f****）

服务	特征	属性	语义
DeviceInfo (0x180A)	fw_version (0x2A26)	read	ASCII，如 0.5.0-s5
	serial_no (0x2A25)	read	BT MAC 12 位 hex
	board_rev (0009)	read	ASCII，如 P1-N16R8
Command (0001)	cmd (0002)	write	命令帧（§10.3），单次 ≤ MTU-3
	resp (0003)	read+notify	最近一条 CLI 应答行（ASCII 去换行）
Telemetry (0004)	state (0005)	read+notify	20B 状态帧，10 Hz notify
	notify_en (0006)	read+write	u8 应用级遥测开关（非 0 才发）
Config (0007)	pwm_params (0008)	read+write	4B: [0]=pump_max [1]=hold_duty [2:4]=hold_ms LE

两处设计取舍值得展开。**resp** 特征是 CLI 应答行（如 `inflate=0、state=0x0221 err=0`）的镜像——第 9 章讲过的 `cli_out()` 应答挂钩（`pn_cli_set_resp_sink`）在这里落地：每条新应答存入 64 字节环形缓冲并在已订阅时 `notify`；未产生应答时 `read` 返回 `boot`。BLE 由此拿到的不是一套新协议，而是串口体验的完整平移。**Config** 服务解决的是“编译期宏没有运行时接口”的欠账：P0 的保压降额参数 `PN_HOLD_DEFAULT_DUTY/..._DELAY_MS` 原是编译期常量，S5 用 4 字节 RAM 镜像补上——`read` 返回镜像（初值 = 宏默认 255/170/500），`write` 立即生效（10 ms 节拍的 `pn_optimize_power` 改读 `ble_twin_hold_duty()`），掉电不保存（NVS 持久化留 P1）。契约对“未完成”同样诚实：`pump_max_pwm` 字段预留未接线。

UUID 在代码里有一个必踩的字节序坑：NimBLE 把 128 bit UUID 存成小端字节数组——`value[0]` 是 UUID 最低位字节（即后缀的低字节）。`ble_twin.c` 的宏 `PN_BLE_UUID_BASE_LE(sfx)` 把后缀拼在数组头部、基址逐

字节倒序排列，九个 UUID 常量由此展开；文档 §2 表与代码宏一一对应，”表里哪个后缀”与”代码哪个常量”不会走失。

10.3 载荷一：0xA5 帧骑在 write 上

cmd 特征接受的命令帧就是 pn_core/proto.h 的 v1 二进制帧——BLE 没有发明新格式，只是把第 9 章协议层的 5 字节帧驮在 GATT write 上：

Listing 10.1: cmd 写载荷（BLE.md §4.1：5B，小端无关）

```

字节 0      1      2      3      4
+-----+-----+-----+-----+-----+
| 0xA5 | cmd |ports| pwm | crc8 |
+-----+-----+-----+-----+

```

例：充气 端口1 PWM255 -> A5 2B 01 FF F8
 ('+'=0x2B; crc8(A5 2B 01 FF)=0xF8, 多项式 0x07 初值 0x00)

CRC 错的帧静默丢弃——与 proto.c 流式解析器同一策略：ASCII 明文行（如 I 1 255\n）同样被 pn_cmd_feed() 接受，两种编码可混用。帧内 cmd 字节是 ASCII 动作码，pn_cmd_feed 把它翻译成等价 CLI 行再分发（表 10.2）。注意这张表是**有损子集**：九条帧化命令覆盖高频动作，而 H/G/F 这些无帧码的命令走 ASCII 行直发——浏览器端的逆映射（frameForLine）遇到它们返回 null，自动降级为明文。

表 10.2: 0xA5 帧 cmd 字节到 CLI 行的翻译表（BLE.md §3.2）

帧内 cmd 字节	宏	等价 CLI 行	语义
'+'	PN_CMD_INFLATE	I <ports> <pwm>	充气
'-'	PN_CMD_VACUUM	V <ports> <pwm>	抽气
'^'	PN_CMD_RELEASE	R <ports>	释放
'!'	PN_CMD_STOP	S <ports>	停止/保压
'o' / 'c'	PN_CMD_OPEN/CLOSE	O/C <ports>	开/关端口阀
'?'	PN_CMD_QUERY	P	读全部传感器（pwm 字段 = 传感器号）
'S'	PN_CMD_STATE	T	回状态字
'R'	PN_CMD_RESET	X	闭环复位

10.4 载荷二：20 字节 state 帧与小端纪律

遥测方向的核心是 20 字节状态帧（BLE.md §4.2），全字段小端：

Listing 10.2: state 通知载荷（BLE.md §4.2：20B，全部小端 LE）

```

偏移 0      2      12 14      20
+-----+-----+-----+-----+
| u16| 5 x i16 |u16| 6B 保留=0 |
+-----+-----+-----+-----+
state 压力x10 tick

```

例：state=0x0221, p0=+12.3kPa(123), p1=-53.3kPa(-533), tick=0x01F4:
 21 02 | 7B 00 | EB FD | 00 00 | 00 00 | 00 00 | F4 01 | 00 00 00 00 00 00
 ~~~~~  
 -533 = 0xFDEB, 小端字节序 EB FD (tests/ble\_tests.c 同向量)

三个字段的语义都经过推敲。**压力用 i16 存 kPa×10**：物理量程  $\pm 100 \text{ kPa} \times 10 = \pm 1000$ ，16 位有符号数绰绰有余，整数传输免浮点；读失败的 slot 填哨兵 0x8000（-32768）——物理量程内的合法值永远到不了它，不会混淆；P0 现挂 2 只传感器，slot2-4 为 P1 预留恒 0。**tick 是 ms/100 mod 65536**：即 100ms 序号，接收端用它做丢帧检测与时序对齐——10Hz 流里 tick 不连续就知道丢了 notify。**6 字节保留恒 0**：给未来扩展开位，同时兼任帧校验——解包方检查保留字节非 0 即判非契约帧。

打包纯逻辑放在 pn\_core/ble\_frame.c（不进 HAL，主机可测——ble\_tests.c 的 4 个测试就以 ± 压力、哨兵、保留字节违例为向量）：

**Listing 10.3:** ble\_frame.c 的 state\_pack / state\_unpack（摘自源码）

```
int ble_state_pack(uint8_t out[PN_BLE_STATE_LEN], uint16_t state_word,
                  const int16_t pa10[PN_BLE_PRESS_SLOTS], uint16_t tick)
{
    if (!out || !pa10) return -1;
    put_u16_le(out + 0, state_word);
    for (int i = 0; i < PN_BLE_PRESS_SLOTS; ++i)
        put_u16_le(out + 2 + 2 * i, (uint16_t)pa10[i]);
    put_u16_le(out + 12, tick);
    memset(out + 14, 0, 6); /* 保留字节恒 0（契约 §4.2） */
    return PN_BLE_STATE_LEN;
}

int ble_state_unpack(const uint8_t *in, size_t len, uint16_t *state_word,
                    int16_t pa10[PN_BLE_PRESS_SLOTS], uint16_t *tick)
{
    if (!in || len < PN_BLE_STATE_LEN) return -1;
    for (int i = 14; i < PN_BLE_STATE_LEN; ++i)
        if (in[i] != 0) return -1; /* 保留字节非 0 = 非 contract 帧 */
    /* ……逐字段 get_u16_le 读出（略）…… */
    return 0;
}
```

浏览器端是同一契约的第二种实现（webapp/js/ble.js）——C 与 JavaScript 对着同一份文档各写一遍，正是契约存在的意义。CRC 与解包各取一例对照：

**Listing 10.4:** ble.js 的 CRC-8 与 20B 帧解析（摘自源码，与 proto.c/ble\_frame.c 同契约）

```
function crc8(bytes) { /* MSB-first, 与 pn_core/proto.c 同实现 */
    let crc = 0;
    for (let i = 0; i < bytes.length; ++i) {
        crc ^= bytes[i];
        for (let b = 0; b < 8; ++b)
            crc = (crc & 0x80) ? ((crc << 1) ^ 0x07) & 0xFF : (crc << 1) & 0xFF;
    }
    return crc;
}

export function parseState20(buf) {
    const dv = buf instanceof DataView ? buf
        : new DataView(buf instanceof ArrayBuffer ? buf : (buf && buf.buffer) || buf);
    if (dv.byteLength < 20) return null;
    for (let i = 14; i < 20; ++i)
```

```

    if (dv.getUint8(i) !== 0) return null; // 保留字节必须全 0
const state = dv.getUint16(0, true); // true = 小端
const pressures = [];
for (let i = 0; i < 5; ++i) {
    const raw = dv.getInt16(2 + i * 2, true);
    pressures.push(raw === -32768 ? 0 : raw / 10); // 0x8000=无效 -> 0
}
return { state, pressures, tick: dv.getUint16(12, true) };
}

```

三处实现差异各有理由：JS 的 `crc8` 少了 `&0xFF` 掩码会溢出成浮点（JavaScript 位运算 32 位，`<<1` 后须钳回单字节）；哨兵 `0x8000` 在浏览器侧归零而非报错——前端把“传感器失联”显示为 0 比显示 `-3276.8 kPa` 更不吓人；校验失败静默丢帧（`return null`）与固件“坏帧静默丢弃”对称。

## 10.5 线程模型：回调不阻塞协议栈

NimBLE 的 GATT 访问回调跑在协议栈自己的 host 任务里。契约 §6 把红线画得很清楚：回调里只许排队列，不许干活。cmd 特征的 write 回调把字节拷进 `cmd_msg_t`（长度 + 缓冲）投 FreeRTOS 队列（深 8），独立的 `cmd_task` 出队调 `pn_cmd_feed()`——命令解析与阀动作的耗时（最坏是自检的秒级 delay）完全不占 BLE 栈时序。队列满时回调返回 `BLE_ATT_ERR_PREPARE_QUEUE_FULL`（CCC 忙语义近似），丢弃有日志：

Listing 10.5: ble\_twin.c 的队列解耦（摘自源码）

```

static int acc_cmd(uint16_t conn, uint16_t attr,
                  struct ble_gatt_access_ctxt *ctxt, void *arg)
{
    if (ctxt->op == BLE_GATT_ACCESS_OP_WRITE_CHR &&
        (uintptr_t)arg == PN_BLE_UUID_SUFFIX_CHR_CMD) {
        cmd_msg_t m = { 0 }; uint16_t copied = 0;
        int rc = ble_hs_mbuf_to_flat(ctxt->om, m.buf, PN_BLE_CMD_MAX, &copied);
        if (rc != 0) return rc;
        m.len = copied;
        if (!s_cmd_q || xQueueSend(s_cmd_q, &m, 0) != pdTRUE) {
            printf("[BLE] cmd_queue_full, dropped\n");
            return BLE_ATT_ERR_PREPARE_QUEUE_FULL;
        }
        return 0;
    }
    /* .....resp 的 READ 分支：持 s_lock 拷出最近应答行（略）..... */
}

static void cmd_task(void *arg) /* 与串口同一 pn_cmd_feed 入口 */
{
    cmd_msg_t m;
    for (;;)
        if (xQueueReceive(s_cmd_q, &m, portMAX_DELAY) == pdTRUE)
            pn_cmd_feed(m.buf, m.len);
}

```

反向的三个数据流同样各有归属：`resp` 由 CLI 应答挂钩（`resp_sink`）写入——持 `s_lock` 存行、已订阅才组 `mbuf` 发 `notify`，锁外发送；`state` 由既有 `10ms control_task` 驱动，`ble_twin_tick_10ms()` 内 10 分频到 100ms，传感器读数转 `Pax10`（负压四舍五入方向单独处理），`tick` 取 `esp_timer` 毫秒时钟派生；共享小状态（`resp` 行 / `pwm` 参数 / `state` 帧）统一由 `s_lock` 串行化。GAP 侧行为从简：广播名 `FLOWIO-P1-XXXX`（`XXXX` = BT MAC 末 2 字节），`AdvData` 装 `Flags` + 128 bit 服务 UUID 0001、名字放 `ScanRsp`（31 字节装不下完整名）；单连接，断开自动重新广播；配对 `just-works`（无 IO、`bonding` 开、MITM 关、LE Secure Connections 开），密钥 NVS 持久化断电不丢——`bring-up` 期特征不做加密强制，真机验证后再评估收紧（`BLE_ERR_INSUFFICIENT_AUTHEN`）。

## 10.6 真机验证三步与无 BT 降级

契约 §7 给出六步 BRINGUP 清单，前三步对应验收工单 A209（第 13 章）：一，nRF Connect 或 Web Bluetooth 扫描，应见 `FLOWIO-P1-XXXX`，连上后四服务齐全，`0x180A` 可读出 `fw/serial/board_rev`；二，订阅 `resp`，写 `A5 2B 01 FF <crc>`——板上泵启动、`resp` 收到 `inflate=0`；三，`notify_en` 写 01 后 `state` 以 10Hz 到达，负压抽气时压力字节呈 `EB FD` 型小端负数。后三步验证 `Config` 生效（写 `FF C8 F4 01` 后保压 500ms 阀电压应下降）、断电 `bond` 保留、加密收紧评估。载荷本身则不需要等真机——`ble_tests.c` 的向量先验构成软件侧工单 A106：帧打包、负压小端、哨兵值、保留字节违例四类断言在主机上先行全绿。

无 BT 环境是契约 §8 的“双保险”：编译期 `#ifdef CONFIG_BT_ENABLED`（BT 关闭编 `stub`，链接零成本），运行时 `pn_hal_esp32_is_qemu()`（QEMU 的 `esp32s3` 无 BT 控制器仿真）——命中任一即打印 `[BLE] QEMU ...skip` 跳过初始化，`ble_twin_tick_10ms()` 空转，串口 CLI 与遥测填充逻辑不受影响。

浏览器端的连接管理（工单 A210）同样有降级路径：`requestDevice` 以 `namePrefix: "FLOWIO-P1"` 过滤，连上后先写 `notify_en=1` 再订阅 `state` 与 `resp`；状态帧经 `parseState20` 解析后由 `bleToStore` 合成与孪生 `/api/state` 同构的对象（状态字拆位、压力数组、合成阀电流 0.36A 常量驱动栅极辉光）——写一次的前端在孪生/真机两种模式间切换（第 11 章）。取消扫描、连接失败、断连三种情形统一走 `cleanup()` 安静回落：徽章转灰、HTTP 轮询自动恢复、流光回底光——不弹窗，是这套前端的性格。

## 本章小结

BLE 链路的可信度来自三份“同一”：命令入口与串口同一（`pn_cmd_feed`，三宿主同文法）；载荷与主机测试同一向量（`ble_frame.c` 纯逻辑，`0x8000` 哨兵与保留字节校验先验固化）；浏览器与固件对着同一契约各写一遍（CRC 与 20B 帧双实现互证）。工程红线只有一条——回调不阻塞协议栈，队列解耦让秒级命令与 10Hz 遥测互不踩脚。至此固件与无线链路闭环，下一章回到浏览器，看这些字节如何变成“长在产品上”的电流与气流。

# 第十一章 上位机与孪生前端 v2：产品即主角

v1.2 的孪生前端 (gui.html) 是一块合格的仪表盘：五支柱、曲线、按钮阵。但 2026-10-02 的用户裁定推翻了它的重心——仪表盘只是辅助，数字孪生的核心视图是产品本身及其爆炸视图上的气流、电流走向；审美对标 Apple 官网的 Liquid Glass。这不是美术需求，而是产品判断：白牌客户打开页面第一眼要认出”这是一台能买走的机器”，数据长在机器上才有说服力。本章讲 v2 重构 (设计规格书 2026-10-02-twin-frontend-v2-design.md)：布局倒转的信息架构、与 PCB 同源的流拓扑、驱动一切电流气流的板级孪生 board\_model.py——包括一次”关断续流  $\tau$  修正”的审查故事，以及仿真实验室浮层。

## 11.1 布局倒转：从仪表盘到舞台

v2 的信息架构是一次彻底倒转 (图 11.1)：旧版”图表居中、3D 缩在角落”，新版全屏 3D 产品场景是唯一主角，仪表功能全部退居边缘——顶栏细玻璃条 (产品名 + 模式徽章”孪生/真机 BLE” + 连接按钮)；左缘抽屉”控制” (8 阀 + 泵的 Apple 风分段控件)；右缘抽屉”遥测” (默认收起的 sparkline 小卡)；底部中央玻璃胶囊”运输条” (播放/暂停/单步/速度/录制/爆炸滑杆)；最底一行状态 (stale 徽章、未标定黄徽、时间与录制状态、仿真实验室入口)。

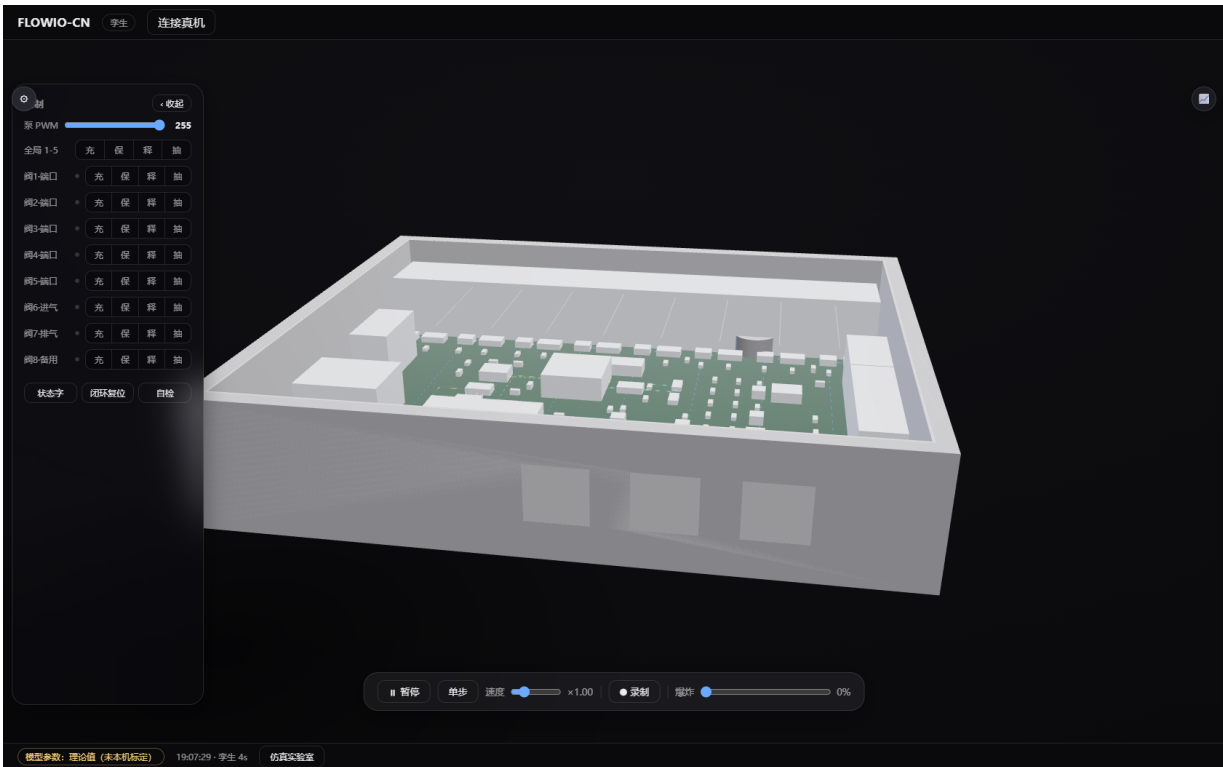


图 11.1: v2 首屏：全屏 3D 产品场景 + 左控制抽屉 + 底部运输条玻璃胶囊；状态行常驻”模型参数：理论值 (未本机标定)”黄徽章

首屏被规定为装配叙事：五部件 (上壳/器件阵/PCB/下壳等) 从四散位飞入合体，约 1.5s，全局缓动曲线 cubic-bezier(0.22,1,0.36,1) (”出舱”手感)；合体后轻微呼吸悬浮 (振幅 0.3mm、周期 4s) 宣告”这是活的”。爆炸视图不再是一个按钮，而是运输条里的大滑杆 + ”展开/合拢”电影模式 (2s 往返缓动)，语义与第 8 章 S3 网格管线的 explode\*k 同源。视觉系统的设计令牌写成表 (玻璃面板 rgba(255,255,255,.06) + backdrop-filter: blur(24px) saturate(180%) + 1px 半透明边 + 20px 圆角；主文本透明度.92 / 次文本.55；强调色只有两档——交互电蓝 #6aa9ff 与流路信号色)，并立了三条克制红线：一屏主色不超过两档；禁止网格

状仪表盘布局；曲线图去网格线只留细轴线。数字一律等宽（`tabular-nums`），遥测刷新时数字不跳动。

工程形态坚持**零构建**：纯 ES Modules，无 React/Vite，`server.py` 直服（`/ → webapp/index.html`，`/webapp/*` 静态资源，`/classic` 保留旧 `gui` 一个过渡版本）。七个小模块各司其职：`main.js` 应用壳（纯对象 `store` + 订阅）、`scene.js` 装配/爆炸/材质、`flows.js` 流光与粒子、`telemetry.js` 双源轮询、`panels.js` 抽屉与热点卡、`simlab.js` 仿真浮层、`ble.js` 真机模式（第 10 章）。Three.js 从 `r128` 升 `r16x`（`vendor` 单次下载），ECharts 按需加载只供展开小图。

## 11.2 流拓扑同源: make\_flows.py 与 flows.json

“电流长在产品上”的第一道关卡是**路径不能手画**——手画的路径与 PCB 对不上，专业客户一眼看穿。v2 的做法是把流拓扑变成网格管线（第 8 章）的旁路：`make_flows.py` 读 `fab/flowio-p1-pos.csv` 器件坐标 + 引脚表拓扑，生成 `flows.json` 与 `hotspots.json`。坐标系与 S3 的 `make_meshes.py` 完全一致（壳坐标： $x = \text{PosX} + 2.9$ 、 $y = -\text{PosY} + 2.9$ 、顶面器件  $z = 4.0\text{mm}$ ），器件焊盘间用曼哈顿折线模拟走线感。防手滑是硬的：任一 `ref` 在 `pos.csv` 查不到就报错列出全部缺失，**不许静默跳过**；`hotspots` 断言顶面器件；验收测试再断言每条电路路径起止点 = 器件坐标  $\pm 0.1\text{mm}$ 。

产出的 `flows.json` 是纯数据：12 条电路路径（DC/USB 双输入  $\rightarrow$  SS34  $\rightarrow$  5V 轨  $\rightarrow$  buck  $\rightarrow$  3V3 轨，加 8 条栅极支路 `U1 $\rightarrow$ R39 $\rightarrow$ Q3 $\rightarrow$ J10 $\rightarrow$` ）+ 8 条气流弧线（各端子出发的二次贝塞尔，端子外延 25 mm——初版 9 mm 假设在目视检查中被否决废弃）。每条路径带 `live` 字段——遥测数据的取值路径：

Listing 11.1: flows.json 的电路与气流条目（摘录）

```
{
  "id": "gate1",
  "color": "#6aa9ff",
  "points": [[30.9, 19.4, 4.0], [7.0, 19.4, 4.0], [7.0, 63.4, 4.0], ...],
  "desc": "ESP32→栅极电阻→MOSFET→端子 1",
  "live": "valves[0].i_A",
  "refs": ["U1", "R39", "Q3", "J10"]}

{
  "id": "port1",
  "ref": "J10",
  "curve": "quadratic",
  "start": [9.4, 71.4, 4.0],
  "ctrl": [9.4, 85.4, 4.0],
  "end": [9.4, 96.4, 4.0],
  "desc": "端子→执行器",
  "live": "valve_duty[1]"}
```

## 11.3 电流辉光与气流粒子：数据怎么“长”上去

`flows.js` 的渲染分两套。电流是 `THREE.Line` + 自定义 `shader` 的行进虚线：片段着色器以 `fract(vT*40 - uTime*uSpeed)` 生成虚线亮带，透明度  $\alpha = (0.08 + \text{dash} \times 0.92) \times uGain$ ，叠加混合出辉光。驱动它的就是 `live` 值的归一——栅极支路 `valves[n].i_A/0.36`、电源路径 `load_a/3.0`：

Listing 11.2: flows.js 的增益与速度映射（摘自源码）：uGain 随电流、uSpeed 随负载

```
const gain = st.connected ? 0.25 + 0.75 * L.live01 : 0.08; // 断连=8%底光
L.uniforms.uGain.value += (gain - L.uniforms.uGain.value) * Math.min(1, dt * 6);
L.uniforms.uTime.value = t;
L.uniforms.uSpeed.value = 0.6 + 2.2 * L.live01; // 满载流速x4.7
```

即增益  $uGain \in [0.25, 1.0]$  随归一电流线性、流速随负载再加速——零负载是静止微光，满载是快速强光，阀一开其栅极支路当拍“通电动起来”（图 11.2）。增益用指数平滑过渡（`dt*6`），遥测的 200 ms 台阶被抹成连续呼吸，不闪跳。

气流是粒子系统：每端口 50 粒沿贝塞尔曲线参数  $t$  循环推进（8 端口共 400 粒，守住  $\leq 500$  的性能预算），透明度与速度都  $\propto$  PWM duty。两处物理映射值得记：其一，**方向反转于真空**——`update()` 里取 `ports_p` 中第一

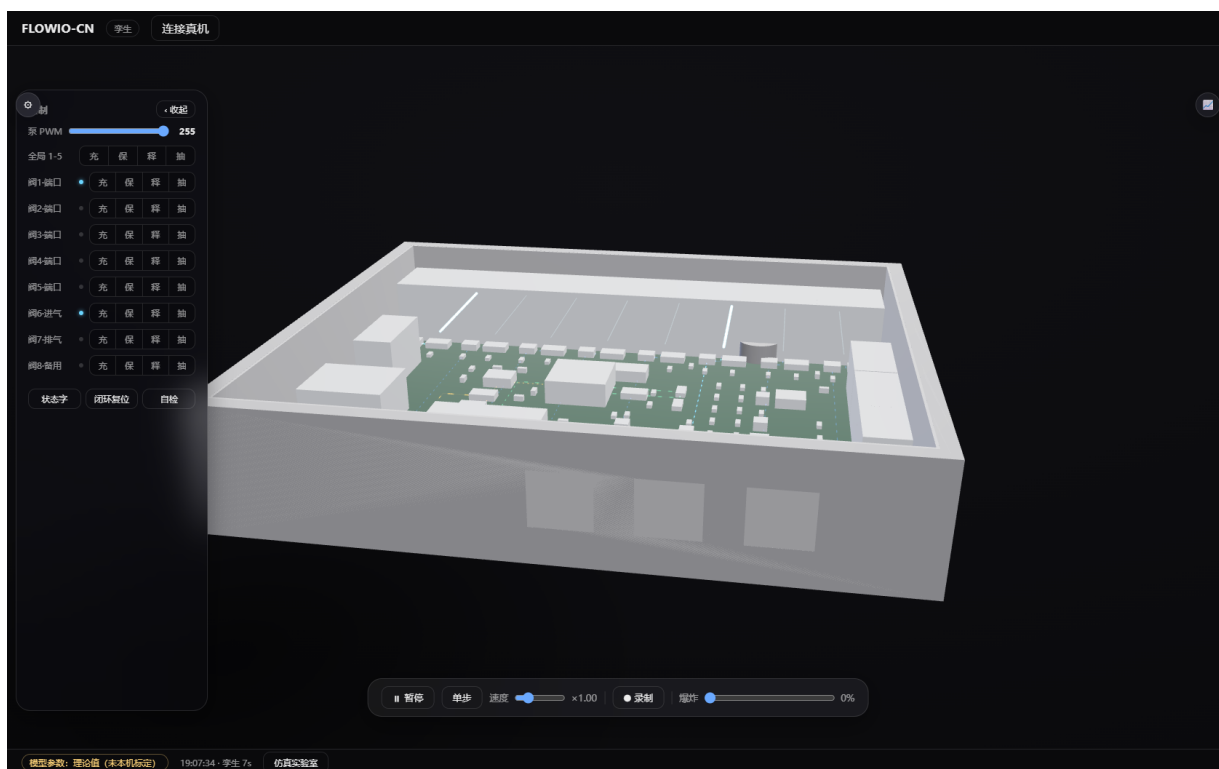


图 11.2: 充气态 (I 全充): 栅极支路电流辉光 + 端口气流粒子喷出; 遥测抽屉/热点卡给出实时值

个绝对值超阈值的压力符号, 负压时速度取负 ( $t$  递减, 粒子回流)、颜色从正压青 #6ad4ff 切到真空琥珀 #ffb454 (图 11.3, 与第 13 章 A102 的差分截图同源); 其二, **爆炸跟随**——流组的每帧位移直接复制 PCB 部件位移 (端子属于 PCB), 爆炸滑杆拉开时流线端点随部件平移拉伸, 形成“连线不断”的爆炸图观感。静止端口留 8% 底光提示通道存在; 断连时全部回落底光, 粒子淡出——安静, 无弹窗。

热点卡补上“点按看器件”的闭环: Raycaster 点击器件 (部件级复用 S3; 器件级用 hotspots.json 的 ref  $\rightarrow$  盒体表), 右下滑入玻璃卡显示 2-4 个实时值 (如 buck: 输出电压/效率/结温; 阀道: 电流/功率)——live 字段路径直接指向 /api/board/state 遥测 (board\_model.py 的契约), 点击即取, 无需新接口。

## 11.4 板级孪生：解析式 RL 与一次续流 $\tau$ 审查

所有电流辉光的数值都来自 board\_model.py (第 4 章 §4.8 讲过它的参数成色管理)。这里讲它的另一面: 作为前端数据源的模型, 如何保证算出来的数与电路仿真同物理——一段真实的审查故事。

阀线圈电流用一阶 RL 解析式而非数值积分。开通侧直接:  $\tau_{\text{on}} = L/(r_{\text{coil}} + r_{\text{ds}}) \approx 1.78 \text{ ms}$ ,  $i = I_{\infty} + (i_0 - I_{\infty})e^{-t/\tau_{\text{on}}}$ , 稳态  $I_{\infty} = 5 \text{ V}/14.04 \Omega \approx 0.356 \text{ A}$ ——100 ms 步长下  $dt/\tau \approx 56$ , 电流完全建立, 解析式既准又免积分。

关断侧最初 (实施计划草案) 写成开通的镜像: 衰减到 0,  $\tau$  沿用  $L/(r_{\text{coil}} + r_{\text{ds}})$ 。审查纠正了它: MOS 已断开, 线圈电流走的是 SS14 续流回路, 回路里没有  $r_{\text{ds}}$ 、只有线圈电阻与二极管钳位电压——正确模型是

Listing 11.3: board\_model.py 的关断续流 (摘自源码):  $\tau$  换回路, 负稳态 + 过零钳位

```
tau_on = p["l_coil"] / (p["r_coil"] + p["rds"]) # 激励回路: L/(r_coil+rds)
tau_off = p["l_coil"] / p["r_coil"] # 续流回路: L/r_coil (SS14 钳位)
i_inf_off = -p["vf_fw"] / p["r_coil"] # 续流负稳态 (过零截止)
...
if on:
    self.i[k] = i_inf + (i0 - i_inf) * math.exp(-dt / tau_on)
else:
```

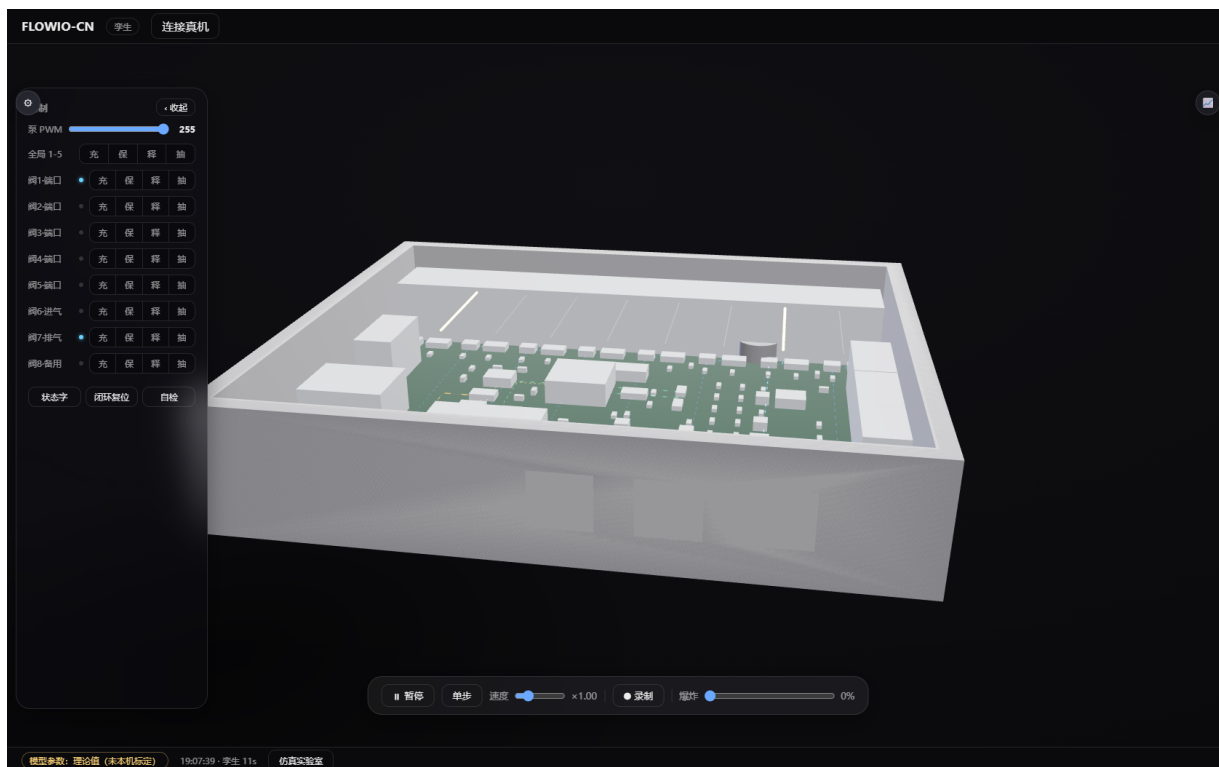


图 11.3: 抽真空态 (V): 气流方向反转、颜色转琥珀, 压力示数转负——方向与颜色由压力符号驱动

```
i = i_inf_off + (i0 - i_inf_off) * math.exp(-dt / tau_off)
self.i[k] = i if i > 0.0 else 0.0 # 二极管反向截止钳位
```

差别看似细微 ( $\tau_{\text{off}} = 1.79 \text{ ms}$  与  $\tau_{\text{on}} = 1.78 \text{ ms}$  几乎同值), 形状却不同: 续流回路的目标值是负稳态  $-V_F/r_{\text{coil}}$ , 电流以更陡的斜率冲向零并在约  $4.9 \text{ ms}$  处被二极管反向截止钳在零——镜像指数衰减则拖着渐近尾巴永不归零。哪个对? 续流二极管存在就不容商量。测试把它钉死: `test_board_model.py` 断言关阀  $10 \text{ ms}$  后电流已钳 0 ( $0 \leq i < 0.35 \times 0.5$ )。注释同时声明防线的另一半: `vf_fw=0.35` 与 `sim_engine` 续流 ODE 的 `VF_FW` 同源——两个实现解同一个物理量, 必须共享同一组常数, 否则板级模型与电路仿真各漂各的, 前端热点卡与仿真实验室对不上数, 孪生就自己拆自己的台。

同一审查还抓出两处“双源漂移”。其一, `telemetry()` 快照曾直接返回 buck 标称输出  $3.269 \text{ V}$ , 而历史曲线按负载调整率公式算出  $3.2638 \text{ V}$ ——同一页面上“当前值”与“曲线末端”差  $5 \text{ mV}$ 。修正: 快照与 `step()` 用同一式子重算 ( $v_{33} = v_{\text{out}} - r_{\text{load}} \cdot I_{\text{logic}}$ ), 标称值另存 `v_nom` 字段。其二, buck 效率与损耗曾是 @ $3 \text{ A}$  工况的常量快照 ( $\{706, 162, 436, 85\} \text{ mW}$ ), 与实时负载无关; 修正为 CCM 损耗分解动态计算 (开关/绕线/二极管/开关损耗四项, 公式与 `sim_engine._buck` 同源),  $v_5$  跌  $\rightarrow$  占空比  $D$  升  $\rightarrow$  损耗随之变化, 测试断言空载与满载的损耗向量不相等。三条修正合起来是一个方法论: 凡是同一物理量有两个公式, 就让它们同源; 凡是同源, 就用测试钉住。

## 11.5 双源轮询与安静降级

数据管道 `telemetry.js` 以  $200 \text{ ms}$  节拍交替轮询两个源: 偶拍 `/api/board/state?since=` (电气遥测 + 增量历史, `since` 游标去重边界点)、奇拍 `/api/state` (气动: 阀 `duty`/压力/状态字)。结果双管道分发: `setState` 进 `store` (抽屉 `sparkline` 消费)、`flows.update` 进流光 (粒子消费)。前端自持两级缓冲——`hist` 是服务端权威历史的镜像 (`v5/v33/load`, 60 点 `sparkline` 窗口  $\approx 24 \text{ s}$ , 展开小图 600 点  $\approx 4 \text{ min}$ ), `snap` 是无服务端历史的快照环 (三结温最大值、效率)。停轮询的三种条件: 页面隐藏、孪生时间暂停、**BLE 真机接管** (`store.bleActive`, 第 10 章); 连续两次失败才置 `stale` (单次抖动不算), `stale` 后主场景流光淡出为底光、状态行亮”遥测冻结”徽章——不弹窗, 不渲染过期值。

## 11.6 仿真实验室：四电路重算浮层

仿真实验室是 v1.2 四电路仿真的 v2 皮肤：从底部升起的全屏玻璃接管浮层，四个分段——Buck 电源 (TPS54331 → 3.3 V/3A)、双源二极管或 (SS34×2)、阀驱动 (AO3400A 低边 + RL)、I<sup>2</sup>C 上拉 (TCA9548A 总线 RC)。它不常驻、不抢主场景。后端是 `sim_engine.py` 的参数化行为级仿真（无 ngspice 环境下的自研状态机，纯 `stdlib`）：每电路一张参数域表——参数名、上下限、默认值（默认值与 BOM 一致，如 buck 的  $C_{out}=100+10+3\mu\text{F}$  合成 113），越域抛 `ParamError` 并指名道姓（前端 400 红条提示）；返回指标（带通过/告警判定徽章）与降采样波形数组，前端 canvas 自绘细线 + 细轴线 + 稀疏刻度 (§11.1 的克制红线)。接口复用 v1.2 端点零改动：GET `/api/board/sim/presets`（参数域表 → 动态表单）、POST `/api/board/sim`（重算）。参数域的意义在验收侧有一次实战：I<sup>2</sup>C 上拉 4.7 kΩ×115 pF 的上升时间实测告警案例，即用该浮层扫参数定位（第 13 章工单 A103）。

验收动线与旧版一样是“测试 + 目视”双轨：`test_webapp.js` 44 断言换新 DOM 锚点（场景 canvas、抽屉开关、热点卡、sim 浮层、运输条），外加 `debug` 浮层的 60 s fps 采样（预算 ≥45）；而装配动画、爆炸跟随流线、阀开栅极亮起、真空回流反向、断连态七项走**强制目视检查**（截图 + 机内视觉逐项审）——首屏、充气、真空三张图（图 11.1–11.3）正是 26 项目视记录的直接复用（工单 A101/A102，第 13 章）。旧 `gui.html` 打“经典版”标保留在 `/classic`（P1 面板独有的泄漏注入/物理注入/scheduler 编排等调试入口迁移完成前不退线），v2 验收通过后再定下线。

### 本章小结

v2 的本质是把孪生前端从“看数据”改成“看产品”：布局倒转让 3D 场景当主角，流拓扑由 `make_flows.py` 从 `pos.csv` 同源生成（不许手画、不许静默跳过），电流辉光的增益与流速、气流粒子的方向与颜色全部由遥测数据驱动，断连与降级一律安静。数值侧的底气来自 `board_model.py`：RL 解析式经过关断续流  $\tau$  的审查修正、快照与历史同式重算、损耗动态化——同一物理量同源、同源被测试钉死。孪生侧至此闭环；下一章给外部开发者一条 5 分钟上手的路：Python SDK。

## 第十二章 Python SDK：把协议钉死

白牌厂商拿到一台 FlowIO 兼容模块，愿意花在上手上的时间大约是五分钟。sdk/python/（包名 flowio\_sdk）就是为这五分钟准备的：pip install pyserial 之后一条命令点亮执行器，FlowIO("COM3").inflater(1, 180) 之类的高层 API 背后是 5 字节 0xA5 帧。但本章真正的重点不是 API 表面，而是它示范的协议防漂移手艺：SDK 与固件 proto.c 用同一组测试向量双向钉死，其中一次“计划草案把 CRC 多项式写错”的真实纠错，值得每个要写跨端协议的人读一遍。

### 12.1 0x8C 事故未遂：计划草案不是真源

SDK 的实施方案（board-twin 计划 Task 6）里有一段示意代码，其中 CRC 这样写：

**Listing 12.1:** 计划草案的 CRC（错误示范——照抄会让 SDK 全军覆没）

```
CRC8_POLY = 0x8C      # 与 proto.c 一致 (Dallas/Maxim 反向)
def crc8(data: bytes, crc: int = 0) -> int:
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = ((crc >> 1) ^ CRC8_POLY) if (crc & 1) else (crc >> 1)
    return crc
```

注释信誓旦旦“与 proto.c 一致”，但它是 LSB 先行的反射多项式（0x07 的位反转）；而固件 proto.c 的 pn\_proto\_crc8 是 MSB 先行、多项式 0x07 初值 0x00 的 CRC-8/ATM。两条算法对同一帧算出的 CRC 不同——以充气帧（A5 2B 01 C8）为例：正确 CRC 是 0x7D，草案算法算出 0xB1。更糟的是错误的后果形态：固件对坏 CRC 帧静默丢弃（第 9 章），SDK 每条命令都石沉大海，没有任何报错指向 CRC——这种 bug 在“链路看起来通了”的假象里能消耗掉一个下午。

草案的错不止一处：命令字节写的是 CLI 字母（‘I’=0x49 等），而 proto.h 的动作码是 ‘+’/‘-’/‘!’/‘/’...；UUID 基址、pwm 默认值也与最终契约有出入。救了这一切的是计划里自己写的一句话：“命令字节以 proto.c 实表为准——实现前先 grep 该表”。实现时真的去 grep 了，于是 CRC 算法、命令表、帧长一并按真源重写。教训可以写成一条通用规则：计划文档里的代码是草案，真源在编译过的固件里；跨端协议的每个数值都要能指回一个可执行的出处。

还有一个巧合值得记录：向量表里“停止”帧 A5 21 01 00 的正确 CRC 恰好等于 0x8C——错误多项式的常数恰好是某条正确帧的 CRC 值。这意味着肉眼对照一两帧的字节串发现不了算法用错（对得上这帧、对不上其余帧），独立的第三方检查值因此必不可少。

### 12.2 crc8 与 encode：全函数与两枚锚点

交付版 protocol.py 的两个核心函数全文如下——注释即契约：

**Listing 12.2:** flowio\_sdk/protocol.py 的 crc8 与 encode（全文摘录）

```
def crc8(data: bytes) -> int:
    """CRC-8/ATM (poly=0x07, init=0x00, MSB 先行, 无反射/无异或输出)。

与 proto.c 的 pn_proto_crc8 位级一致；自检锚点 crc8(b"123456789")==0xF4。



"""
    crc = 0x00


```

```

for byte in data:
    crc ^= byte
    for _ in range(8):
        crc = ((crc << 1) ^ 0x07) & 0xFF if (crc & 0x80) else (crc << 1) & 0xFF
return crc

def encode(cmd, ports: int = 0, pwm: int = 0) -> bytes:
    """构造 5 字节帧（等价 proto.c 的 pn_proto_build）。

    参数:
    cmd: 单字节 str（如 '+' 或命令字节 int（如 CMD_INFLATE）
    ports: 端口位掩码 0x00..0xFF（有效位 0x1F，超出仅透传不校验）
    pwm: 占空比 0..255（QUERY 帧语义为传感器号）

    """
    frame = bytearray(FRAME_LEN)
    frame[0] = PN_PROTO_MAGIC
    frame[1] = _cmd_to_byte(cmd)
    frame[2] = _to_byte(ports, "ports")
    frame[3] = _to_byte(pwm, "pwm")
    frame[4] = crc8(bytes(frame[1:4])) # 覆盖前 4 字节（proto.c ST_CRC 分支同款）
    return bytes(frame)

```

多项式正确性靠两枚独立锚点钉死，都不依赖固件代码本身：`crc8(b"123456789")==0xF4` 是 CRC-8/ATM 的国际标准检查值（多项式、初值、移位方向、输出异或四要素中任何一项错都过不了这一关）；`crc8(b"\xA5")==0x72` 则来自固件 `test_proto.c` 的 `t_crc_known_vector`——单字节定值，防有人手滑改多项式。参数侧的纪律是“宁可拒发也不静默截断”：`_to_byte` 把 `ports/pwm` 收紧到 0..255，越界抛 `ValueError`（`encode("+", 0, 256)` 直接报错，而不是截成 255 悄悄发一个满占空比——那是气动执行器上危险的“善意”）。

## 12.3 vectors.json: 与固件同向量的回归网

帧级正确性靠 `tests/vectors.json` 的黄金字节（表 12.1）：前 5 组的输入参数逐一对齐固件 `test_proto.c` 中 `pn_proto_build` 的调用参数——固件测试怎么构帧，SDK 就怎么构，两侧字节串必须逐字节相同；后 3 组补边界（全端口 + 最大占空比、全端口 `pwm=0`、混合端口 + 非零 `pwm` 的字段复用）。

表 12.1: SDK 协议测试向量（`vectors.json` 摘编；对应工单 A104）

| 向量名                               | 来源                                       | 参数           | 帧字节                     |
|-----------------------------------|------------------------------------------|--------------|-------------------------|
| <code>inflate_fw_roundtrip</code> | 固件 <code>t_frame_roundtrip</code>        | '+', 1, 200  | a52b01c87d              |
| <code>stop_fw</code>              | 固件 <code>t_bad_crc_dropped</code>        | '!', 1, 0    | a52101008c              |
| <code>state_fw</code>             | 固件 <code>t_bad_crc_dropped</code>        | 'S', 0, 0    | a553000028              |
| <code>query_fw_resync</code>      | 固件 <code>t_leading_garbage_resync</code> | '?', 31, 1   | a53f1f0184              |
| <code>reset_fw</code>             | 固件 <code>t_crc_known_vector</code>       | 'R', 0, 0    | a552000043              |
| <code>inflate_max_boundary</code> | 边界                                       | '+', 31, 255 | a52b1fff79              |
| <code>vacuum_zero_boundary</code> | 边界（ <code>pwm=0</code> 自替 255）           | '-', 31, 0   | a52d1f00f7              |
| <code>release_field_reuse</code>  | 字段复用                                     | '^', 10, 42  | a55e0a2aed              |
| CRC 锚点（独立第三方）                     |                                          |              | "123456789"→f4, 0xA5→72 |

`test_protocol.py`（`stdlib unittest`，无 `pytest` 依赖）织成四层网：`encode` 对 8 组向量逐字节比对黄金帧；同 8 组 `decode` 往返（`cmd/ports/pwm` 三元组还原）；`encode`→`decode` 直通往返；错误路径——坏 CRC 用固件同款破坏法（`frame[4] ^= 0xFF`）、坏魔数（重算 CRC 排除干扰专测魔数）、坏长度、参数越界分别断言抛

`ProtocolError`。注意两侧对坏帧的哲学差异是刻意的：固件面对字节流要容错重同步（静默丢弃 + 重同步）；SDK 面对单帧要帮程序员抓 bug（显式抛错、指名道姓）。

## 12.4 client.py: 高层 API 与传输接缝

`FlowIO` 类把帧协议包成语义动作（表 12.2）。端口参数有两条路：位掩码常量（`PORT1.. PORT5, ALL_PORTS`）或 `port_mask(1,3)` 式助手（端口号 1..5 之外即抛错）。

表 12.2: `FlowIO` 高层 API（`client.py`；命令字节抄自 `proto.h` 动作码）

| 方法                                  | 帧       | 语义（对应 <code>actions.c</code> ）                |
|-------------------------------------|---------|-----------------------------------------------|
| <code>inflate(port, pwm)</code>     | '+'     | 开进气阀 + 端口阀 + 泵起（ <code>pwm=0</code> 固件自替 255） |
| <code>vacuum(pwm, ports)</code>     | '-'     | 开排气位 + 端口阀 + 泵起                               |
| <code>hold(port)</code>             | '!'     | 停止/保压（全阀关 + 泵停）                               |
| <code>release(port)</code>          | '^'     | 释放（被动放气， <code>pwm</code> 忽略）                 |
| <code>open_ports/close_ports</code> | 'o'/'c' | 只开/关端口阀                                       |
| <code>query(sensor)</code>          | '?'     | 读压力（ <code>pwm</code> 字段复用为传感器号），读回一行         |
| <code>state()</code>                | 'S'     | 读状态字（读回一行，超时得空串）                              |
| <code>reset()</code>                | 'R'     | 闭环复位                                          |

传输层是一条窄接缝：任何实现 `send(bytes)` 与 `readline() -> str` 两个方法的对象都能从构造器注入（`FlowIO(transport=...)`）——测试用假传输离线跑，未来的 BLE 或孪生 HTTP 通道也从同一接缝接入而不动 `client.py` 一行。串口实现 `SerialTransport` 有三处细节：`pyserial` 是可选依赖（延迟到实例化才 `import`，缺失时报“请先执行 `pip install pyserial`”而不是裸 `ImportError`）；打开串口时 `dtr/rts` 显式拉低——否则打开瞬间会复位 ESP32（`serial_monitor.py` 同款经验）；`list_ports()` 双路探测（`pyserial` 自带枚举优先，Windows 下退化为扫 `COM1..COM255`）。回包语义同样诚实：`0xA5` 帧协议的回包 handler 在固件侧尚未接线（S4 骨架先行），故默认不读回，仅 `state()/query()` 这类天生要回包的行动作默认读一行（超时得空串，不致命）。SDK 本体 `stdlib-only`——不装 `pyserial` 也能 `encode/decode` 离线工作。

## 12.5 hello\_glove: 一个来回的冒烟示例

示例全文（`examples/hello_glove.py`）——充、保、释、抽一个来回，每步打印发出的帧字节；无参数运行时列出可用串口后退出，本身就是可用性设计的一部分：

Listing 12.3: `hello_glove.py` 全文（`examples/`）

```
"""hello_glove.py——FlowIO上位机最小示例（充→保→释→抽一个来回）。

用法:: python hello_glove.py COM3 # 指定串口
python hello_glove.py # 无参数时列出可用串口后退出
"""

from __future__ import annotations
import sys, time
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1])) # sdk/python 入包路径
from flowio_sdk import FlowIO, encode # noqa: E402

def main() -> int:
```

```

if len(sys.argv) < 2:
    from flowio_sdk import SerialTransport
    ports = SerialTransport.list_ports()
    print("用法: python hello_glove.py <COM口> 如: python hello_glove.py COM3")
    print("可用串口:", ports if ports else "(未检测到; 请安装 pyserial 或检查连接)")
    return 1

port = sys.argv[1]
cycles = int(sys.argv[2]) if len(sys.argv) > 2 else 1

with FlowIO(port) as io:
    print(f"[hello_glove]_{port}_已连接, {cycles}_个循环: 充→保→释, 最后抽一次")
    for i in range(cycles):
        print(f"--cycle_{i+1}/{cycles}")
        print(f"inflate(1,180)->", encode("+", 1, 180).hex())
        io.inflate(1, 180)      # 端口 1 充气, 180/255 占空比
        time.sleep(2.0)
        print(f"hold(1)----->", encode("!", 1, 0).hex())
        io.hold(1)              # 保压 (停泵关阀)
        time.sleep(1.0)
        print(f"release(1)----->", encode("^", 1, 0).hex())
        io.release(1)          # 释放 (被动放气)
        time.sleep(1.0)
        print(f"vacuum(180)----->", encode("-", 0x1F, 180).hex())
        io.vacuum(180)          # 全端口抽气一次
        reply = io.state()      # 读一次状态字
        print(f"state()----->", repr(reply) or "(无回包——固件 0xA5_handler 接线后即有)")
        print("[hello_glove]_完成")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

打印帧字节 (encode("+", 1, 180).hex() -> a52b01c87d) 不是装饰: 用户拿它对照 vectors.json 或 nRF Connect 的写日志, 就完成了第 13 章工单 A104 的“向量可见”证据链: 串口/孪生双模式则与前端一致——固件 pn\_cmd\_feed 双编码兼容 (第 9 章) 意味着 SDK 的帧发到串口、BLE cmd 特征或孪生 DLL 是同一串字节。

## 12.6 接口面：SDK 的边界即产品的边界

开放给二次开发者的接口面要刻意收窄, SDK 目前公开的是: 高层动作方法 + encode/decode/crc8 (想自己组帧的人用) + port\_mask 与端口常量 + ProtocolError。两个单位约定写进文档就不再改: 压力一律 kPa×10 整数传输 (BLE 帧如此, 回包文本如 sensor0=12.30 kPa); 状态字位定义与第 9 章表一致 (bit0–4 端口、bit5/6 方向阀、bit7 泵、bit9 传感器在线、bit15 错误)。串口是 SDK 的第一通道 (115200-8N1); BLE 通道浏览器端已有 ble.js (第 10 章), Python 侧经 transport 接缝留给 P1——契约 (UUID、载荷) 都已冻结, 接入只是工作量而非设计题。

## 本章小结

SDK 的价值不在几百行代码，而在它把协议变成了带防漂移锚点的公共财产：CRC-8/ATM 被标准检查值 0xF4 与单字节定值 0x72 双重钉死；8 组黄金帧向量与固件测试同参数、同字节；错误路径两侧分工——固件容错重同步、SDK 拒收即抛错。那次“0x8C 草案”的未遂事故是全项目方法论的一次微型演习：计划不是真源，grep 过的、编译过的、被向量测过的才是。至此，固件（第 9 章）、无线（第 10 章）、前端（第 11 章）与 SDK 四条对外通道全部落地，第 13 章开始逐工单验收。

# 第十三章 验收实录：数字孪生与实物

验收在本书里不是附录，而是一等公民的写作方式——每一节对应一张验收工单，验收产生的截图、波形与实测值即书稿图源。本章分上下两篇。上篇是已经完成的一半：板还在嘉立创产线上，但数字孪生的全套产品（前端、固件逻辑、BLE 契约、SDK）已经可以逐项验收，六张软件侧工单 A101–A106 于 2026-10-02 全部判定 pass，本章按”操作步骤 → 判定 → 证据”的固定体例逐节实录。下篇是等待硬件的一半：二十四张硬件工单（A201–A224）与六张标定工单（A301–A306）已生成模板，回板后按 15–30 分钟一张的节奏执行，产物直接回流本章与第 14 章。上篇证明”软件这边没有欠账”，下篇规定”硬件那边怎么还账”。

## 13.1 上篇：数字孪生验收（已完成）

软件侧验收的总战场是 `firmware/twin/` 的回归矩阵：前端全链 `test_webapp.js` 44 断言、经典版三套 `test_gui/test_gui_p1/test_e2e`（28/40/43）、API 两套 `test_api.sh/test_api_board.sh`（50 项 + 板级全 OK）、Python 侧 `test_sim_engine.py/test_board_model.py/test_flows.py`、固件主机 33 测试与 QEMU 冒烟、SDK `test_protocol.py` 九用例。2026-10-02 交付日全绿（HANDOFF”前端 v2 交付”节）。六张工单各自从这张矩阵里取证，表 13.1 先给总览，以下逐节实录。

表 13.1: 软件侧六张工单验收一览（2026-10-02 全部 pass）

| 工单   | 验什么           | 证据形态              | 详见           |
|------|---------------|-------------------|--------------|
| A101 | 装配叙事 + 爆炸滑杆跟随 | 截图 + 渲染态断言（7+4 项） | 本节；13.1.7    |
| A102 | I/V/S 三态差分联动  | 差分截图 + 断言（13 项）   | 本节；图 13.2    |
| A103 | 仿真实验室四电路重算    | 截图 + 断言（7 项）      | 本节；图 13.3    |
| A104 | SDK 协议向量双向    | 测试输出（8 组向量往返）     | 第 12 章表 12.1 |
| A105 | 固件两道防线        | 33 测试 + 九项 grep   | 第 9 章 §9.2   |
| A106 | BLE 载荷向量先验    | 4 向量主机测试          | 第 10 章 §10.6 |

### 13.1.1 A101 装配叙事与爆炸跟随

操作步骤：启动 `server.py` 后打开 `http://127.0.0.1:8000/`，首屏即装配叙事——五部件（上壳/器件阵/PCB/下壳等）从四散位飞入合体约 1.5s，随后进入呼吸悬浮；拖动运输条爆炸滑杆至 50%，记录分解态截图。自动化侧由 `test_webapp.js` 经调试钩 `window.__t2` 读取渲染态：加载块断言五部件在场且收敛位移 <3 mm，爆炸块断言 `setExplode(1)` 后部件位移 >10 mm、系数平滑跟随  $k \approx 1$ 、流组每帧复制 PCB 位移（流线端点绑定不断线）。

判定：pass。装配叙事收敛、爆炸位移量级、流组跟随、滑杆标签联动四项断言全绿；目视侧首屏记录 `v2_01_intro.png`（同款图见第 11 章图 11.1）。证据：`firmware/twin/shots/v2_01_intro.png`、`v2_02_explode50.png` 归档于交付提交 `ec3cefa`，拷贝入 `book/figures/`（本章图 13.1）。

### 13.1.2 A102 开阀电流亮起、真空反向与静默差分

操作步骤：在左控制抽屉发三条命令做差分——I 1 255（1 号口全充）、S 1（确定性关阀）、V 1 255（1 号口抽真空），各等遥测管道（200ms 节拍）回流一拍后截图。三态的关键差异：充气态 1 号栅极支路电流辉光增强（`uGain>0.5`）、行进虚线加速（`uSpeed>1.5`）、端子喷出青色粒子（正压色 #6ad4ff、方向 +1）；关阀态辉光回落底光（`uGain<0.35`）、粒子淡出（`op<0.05`）；真空态粒子相位递减（ $t$  逆向行进）、颜色转琥珀 #ffb454——方向与颜色都由压力符号驱动（第 11 章 §11.3）。

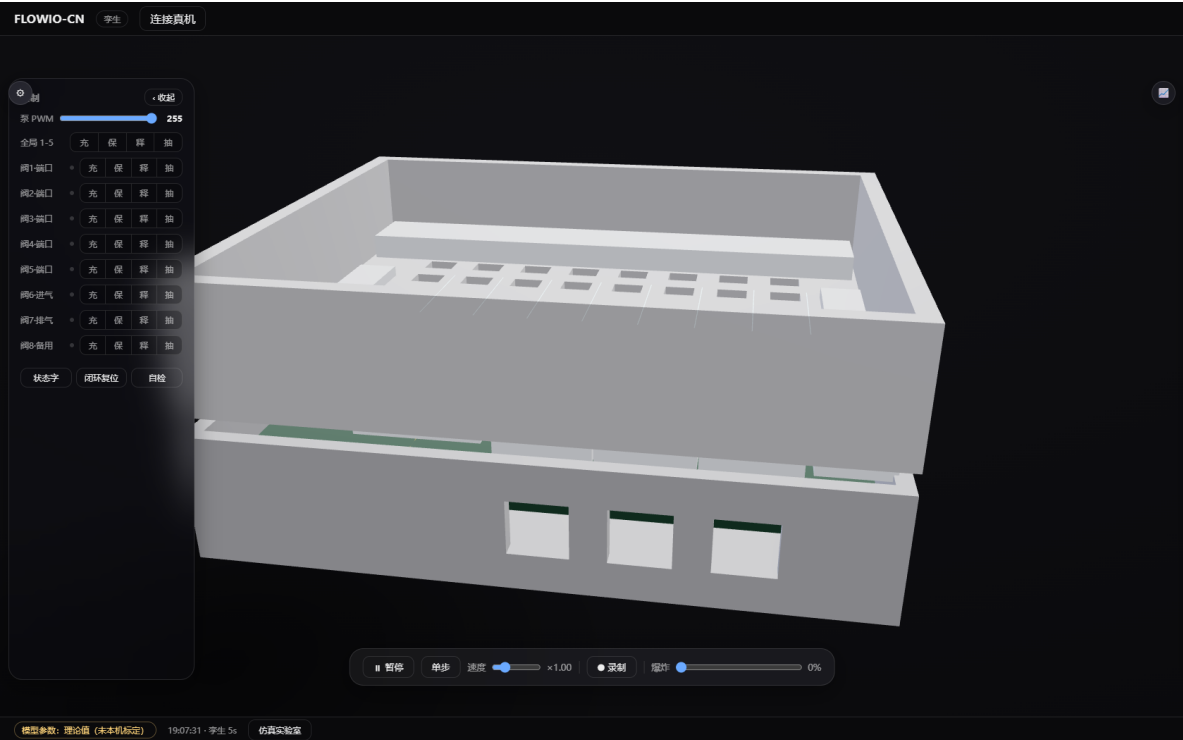


图 13.1: A101 爆炸 50%: 五部件沿轴向分解，电流线与气流弧随 PCB 部件平移拉伸——“端点绑定”连线不断”，左滑杆百分比标签实时联动

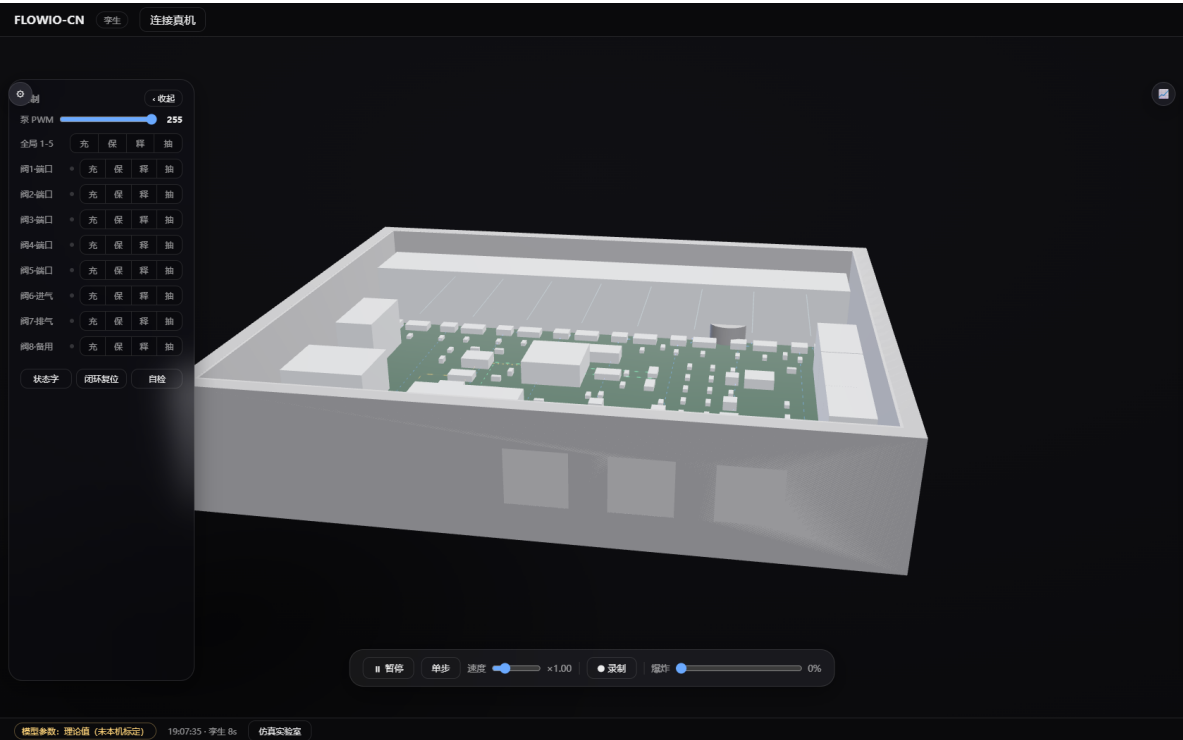


图 13.2: A102 静默态 (s 1 后): 栅极支路回落 8% 底光、粒子淡出——“通道存在”的提示仍在，但不再假装有气流

充气态与真空态截图（v2\_03\_inflate.png/v2\_05\_vacuum.png）与第 11 章图 11.2、图 11.3 同源，此处不重复收录；本节补入第三张差分锚点——静默态。判定：pass，三态差分对照表 13.2（断言全文见 §13.1.7 矩阵第 12–24 行）。其余端口静默也在断言内：开 1 号阀时 port2 粒子透明度  $\approx 0$ ——联动是选择性的，不是全场发光的假动画。

表 13.2: A102 三态差分判定表（期望即 spec §3 语义，实测取自渲染态断言）

| 状态      | 栅极电流辉光            | 端口气流粒子        | 判定   |
|---------|-------------------|---------------|------|
| I 1 255 | 亮起 (uGain>0.5)    | 喷出，青色，t 递增    | pass |
| S 1     | 回落底光 (uGain<0.35) | 淡出 (op<0.05)  | pass |
| V 1 255 | 亮起 (同充气)          | 琥珀色，t 递减 (反向) | pass |

13.1.3 A103 仿真实验室四电路重算

操作步骤：状态行点开” 仿真实验室” 浮层，依次选择 Buck 电源 / 双源二极管或 / 阀驱动 / I<sup>2</sup>C 上拉四个电路，保持 BOM 默认参数各重算一次；再把 I<sup>2</sup>C 上拉的负载电容从默认值改动并越域提交一次，观察 400 红条与修正后的收敛。判定：pass——四电路指标表与波形 canvas 均按参数即时重算（断言” 四电路选择卡””presets 动态表单（buck 参数  $\geq 6$  输入）””自动重算指标表 + 波形 canvas”），越域请求返回 400 且红条指名道姓含 vin，修正后红条收敛。

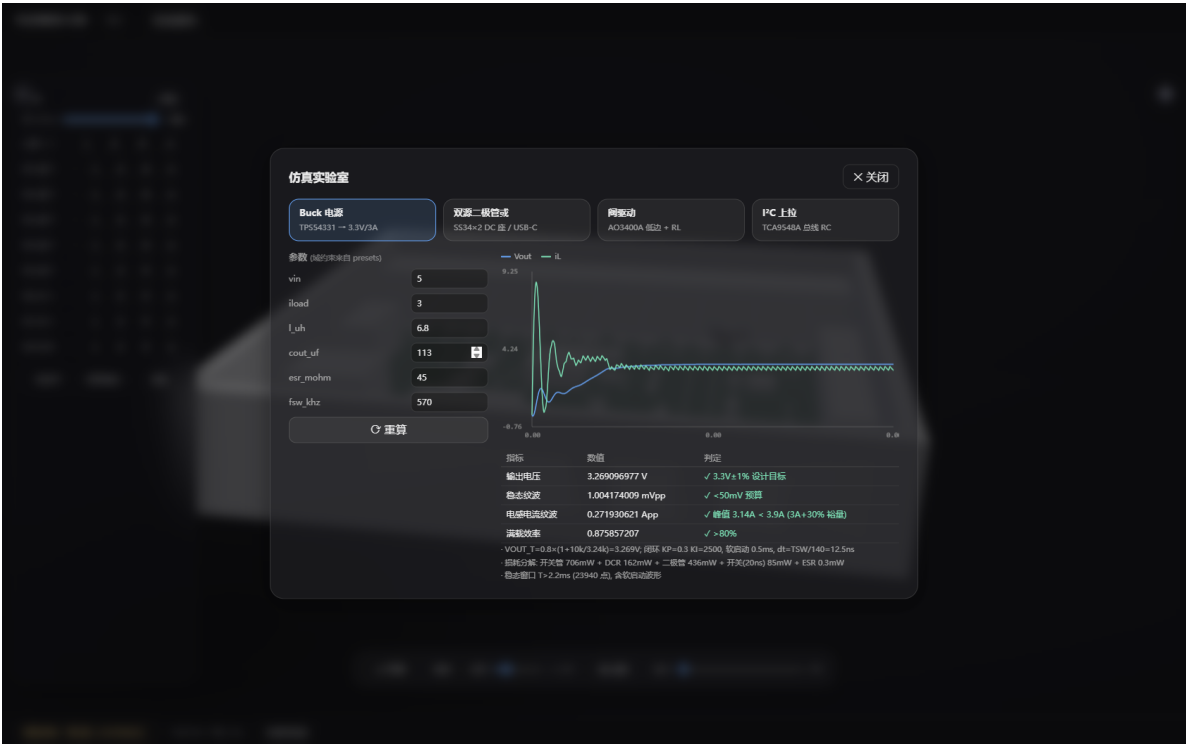


图 13.3: A103 仿真浮层：全屏玻璃接管，四电路分段选择、参数域驱动的动态表单与指标/波形重算——四电路的物理与达标判据见第 4 章 §4.9

这张浮层在验收史上留过一个实战案例：I<sup>2</sup>C 上拉 4.7k $\Omega$ ×115pF 的上升时间告警（第 11 章 §11.6），就是用它扫参数定位的——告警不是终点，浮层给出” 改哪个参数能救” 的动线。证据：v2\_08\_simlab.png；软件侧由 test\_webapp.js 仿真块 7 断言覆盖（矩阵第 36–42 行）。

13.1.4 A104 SDK 协议向量双向 + hello\_glove 冒烟

操作步骤:python -m unittest test\_protocol(sdk/python/tests/)跑协议回归;python examples/hello\_glove.py (孪生模式) 跑”充 → 保 → 释 → 抽”一个来回的最小冒烟。判定: pass——vectors.json 的 8 组黄金帧向量与固件 test\_proto.c 同参数构帧、逐字节相同; 同 8 组 decode 往返还原 cmd/ports/pwm 三元组; 坏 CRC/坏魔数/坏长度/参数越界四条错误路径各自由 ProtocolError 接住。向量表全文与”CRC 多项式写错”的纠错故事见第 12 章表 12.1 (§12.3)。

值得强调的是这组工单的取证形态: 它没有截图, 证据是测试输出本身——两侧对同一向量表双向编解码, 任何一端漂移都会立刻红。hello\_glove 冒烟补的则是”五分钟上手”承诺的可兑现性 (第 12 章 §12.5)。

13.1.5 A105 固件 QEMU 冒烟 + 主机 33 测试

操作步骤: cmake + Ninja 主机编译后跑 firmware/tests/ 全量; bash qemu\_smoke.sh 跑整只固件冒烟。判定: pass——33 个主机测试全绿 (动作层 10 / 闭环 7 / 协议 4 / TCA 8 / BLE 载荷 4), QEMU 冒烟九项 grep 判定全过: 启动横幅、PSRAM 8MB、F 自检三步、I 1 255/H 1/S 1 应答、L 路由。两道防线的分工与实跑输出见第 9 章 §9.2 与 §9.8。

这是六张工单里唯一”没有浏览器”的一张, 却是对回板影响最大的一张: 逻辑层在烧录前就带着 33 条断言的战绩上路, BRINGUP 里遇到的硬件问题才可能被定位到硬件, 而不是与软件 bug 混作一团。

13.1.6 A106 BLE 契约载荷向量 (向量先验)

操作步骤: 跑 ble\_tests.c 四个主机测试 (帧打包、负压小端、哨兵值、保留字节违例)。判定: pass——契约 twin/BLE.md §4.1 命令帧 (0xA5 前缀 + CRC-8/ATM) 与 §4.2 state 20B 小端载荷以向量形式先验冻结: 负压 -533Pa 打包为 EB FD 小端、压力无效哨兵 0x8000、保留字节违例拒收。真机侧的 nRF Connect 扫描/写/订阅三步不在此单范围——那是回板后 A209 的活 (第 10 章 §10.6)。

”向量先验”是这张工单的方法论价值: 无线链路的正确性不必等天线, 字节级契约可以在主机上先钉死; 等真机到场时, A209 要验的只剩射频与连接管理本身。

13.1.7 test\_webapp.js: 44 断言矩阵

六张工单里四张 (A101/A102/A103 及 v2 全链) 共享同一台取证机器——test\_webapp.js。它的做法值得单独记一节: playwright 无头浏览器加载 v2 页面, 经 window.\_\_t2={scene,flows,panels,simlab} 调试钩直接读渲染态 (shader uniform、粒子相位、部件位移), 而非截图比像素; 控制命令用 fetch 拦截采集 (POST 截获、GET 透传保遥测存活), 保证”发命令”与”看回流”在同一条真实管道上; 热点卡用相机投影 U3 中心坐标做真鼠标点击 (raycaster 实证), 不用 DOM 伪事件。表 13.3 是全部 44 条断言的矩阵, 按功能块分十一组。

表 13.3: test\_webapp.js 44 断言矩阵 (2026-10-02 全绿; 分组标题行非断言)

| #         | 断言 (含判定基准)                       |
|-----------|----------------------------------|
| 加载与首屏 (7) |                                  |
| 1         | 5 部件 STL 装配在场 (__t2.scene.parts) |
| 2         | 装配叙事收敛 (部件位移 <3mm)               |
| 3         | 热点器件 6 个 (hotspots.json)         |
| 4         | 电流线 12 条 (4 电源 + 8 栅极)           |
| 5         | 气流端口 8 组 × 50 粒                  |
| 6         | 状态行: 未标定黄徽常驻 / stale 隐藏          |
| 7         | 遥测抽屉默认收起 (spec §2)               |

表 13.3（续）

| #              | 断言（含判定基准）                       |
|----------------|---------------------------------|
| <b>爆炸滑杆（4）</b> |                                 |
| 8              | setExplode(1) → 部件位移 >10 mm     |
| 9              | 爆炸系数平滑跟随 ( $k \approx 1$ )      |
| 10             | 流组每帧复制 pcb 位移（端点绑定不断线）          |
| 11             | 滑杆 input → 百分比标签联动（50%）         |
| <b>充气联动（7）</b> |                                 |
| 12             | gate1 live01= 阀电流归一（>0.5）       |
| 13             | gate1 uGain>0.5（电流辉光）           |
| 14             | gate1 uSpeed>1.5（行进虚线加速）        |
| 15             | port1 粒子可见（op>0.3）              |
| 16             | 其余端口静默（port2 op≈0）              |
| 17             | 正压配色青 0x6ad4ff / 方向 +1          |
| 18             | 粒子沿曲线行进（300 ms 内位移）             |
| <b>关阀回落（2）</b> |                                 |
| 19             | S 1 → gate1 回落底光（uGain<0.35）    |
| 20             | S 1 → port1 粒子淡出（op<0.05）       |
| <b>真空反向（4）</b> |                                 |
| 21             | 真空压力符号 → dir= -1                |
| 22             | 真空配色琥珀 0xffb454                 |
| 23             | 真空粒子可见（op>0.3）                  |
| 24             | 粒子相位递减（t 逆向行进）                  |
| <b>遥测抽屉（3）</b> |                                 |
| 25             | 遥测浮钮可见（默认收起）→ 点击展开              |
| 26             | 遥测抽屉 5 张 sparkline 卡            |
| 27             | 遥测/控制抽屉收起 ↔ 展开往返                |
| <b>控制命令（4）</b> |                                 |
| 28             | 阀 1 充 → POST /api/cmd “I 1 255” |
| 29             | PWM 滑杆并入命令（全局抽 → “V 31 200”）    |
| 30             | 阀 3 释 → “S 4”（确定性关阀掩码）          |
| 31             | 脚部按钮状态字 → “T”                   |
| <b>热点卡（4）</b>  |                                 |
| 32             | U3 点击 → 热点卡出现（ref+ 器件）          |
| 33             | 热点卡 live 值全部填充（非 —）             |
| 34             | 热点卡迷你横条渲染                       |
| 35             | 热点卡关闭按钮收卡                       |
| <b>仿真浮层（4）</b> |                                 |
| 36             | 浮层升起（非 hidden）                  |
| 37             | 四电路选择卡                          |
| 38             | presets 动态表单（buck 参数）           |

表 13.3（续）

|          |                                    |
|----------|------------------------------------|
| #        | 断言（含判定基准）                          |
| 39       | 自动重算指标表 + 波形 canvas                |
| 越域与恢复（3） |                                    |
| 40       | 参数越域 → 400 红条含 vin                 |
| 41       | 修正参数后指标回归（红条收敛）                    |
| 42       | ESC 关闭浮层                           |
| 收尾（2）    |                                    |
| 43       | /classic 旧 gui 挂载（5 端口卡片 + 气路 SVG） |
| 44       | 全程无页面 JS 错误                        |

矩阵之外，回归网还有三层：旧三套（test\_gui 28/28、test\_gui\_p1 40/40、test\_e2e 43/43）改挂 /classic 保证过渡版不坏；test\_api.sh 50/50 守住 API v1.2 端点零改动；性能用?debug 浮层 60s rAF 采样，实测 56.8 fps（预算 ≥45，headless 受 vsync 钳制，真机更高）。

13.1.8 26 项目视审计：从截图到像素证据

断言读的是渲染态，但“看起来对不对”断言管不了——装配动画的手感、玻璃材质的层次、粒子在亮壳上的对比度，最终都要过人眼这一关。v2 交付按 F3 模式做强制目视：十张截图 v2\_01-v2\_10（首屏/爆炸 50%/充气/静默/真空/热点卡/遥测抽屉/仿真浮层/断连态/恢复态），先机内像素审计——直方图、色簇、帧间差分，外加“浏览器投影走廊”校准（同一元素在浏览器坐标系与截图像素坐标系之间的映射带），共 26 项检查全数通过；再交 AI 视觉复核装配材质、UI 布局与热点卡内容，全部对上。断言、像素、AI 三层证据互不替代：断言防逻辑错、像素防渲染错、AI 复核防语义错。

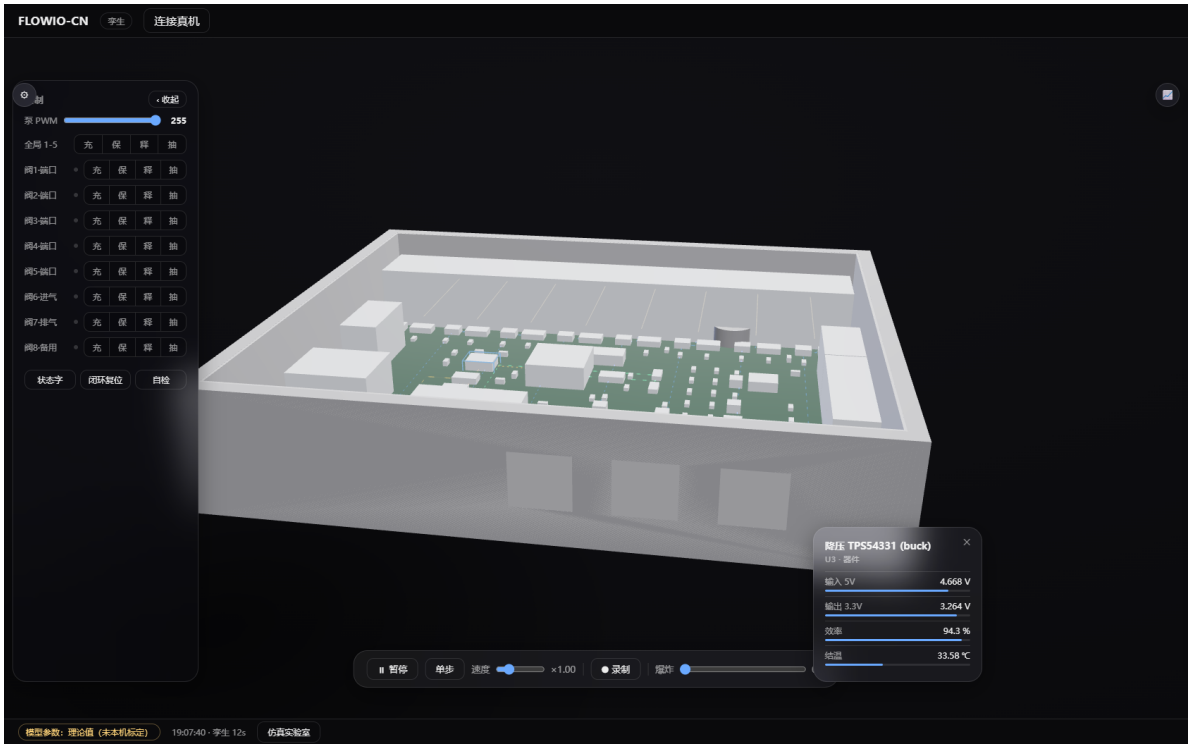


图 13.4：目视审计之热点卡：raycaster 点击 U3（buck 变换器）弹出玻璃卡，实时值全部填充（无一空位），迷你横条渲染正常

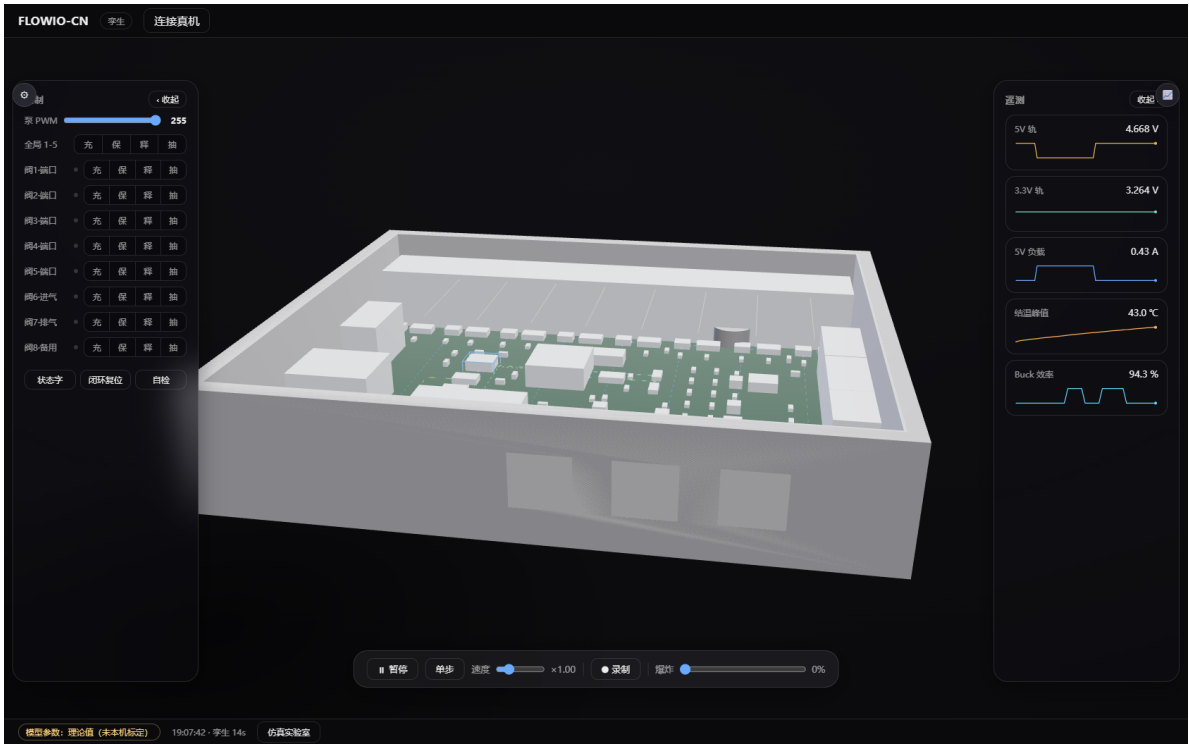


图 13.5: 目视审计之遥测抽屉：右缘展开 5 张 sparkline 玻璃卡，数字等宽（tabular-nums）刷新不跳动

断连态是目视审计里最有设计含量的一项（图 13.6）：停掉遥测源后，连续两次失败才置 `stale`（单次抖动不算），主场景流光淡出为底光、粒子淡出、状态行亮”遥测冻结”徽章——不弹窗、不渲染过期值（第 11 章 §11.5）。恢复后底光重新亮起、徽章消退。这一”安静的降级”哲学：坏消息用光的暗下来讲，比用弹窗讲更符合一台机器的体面。

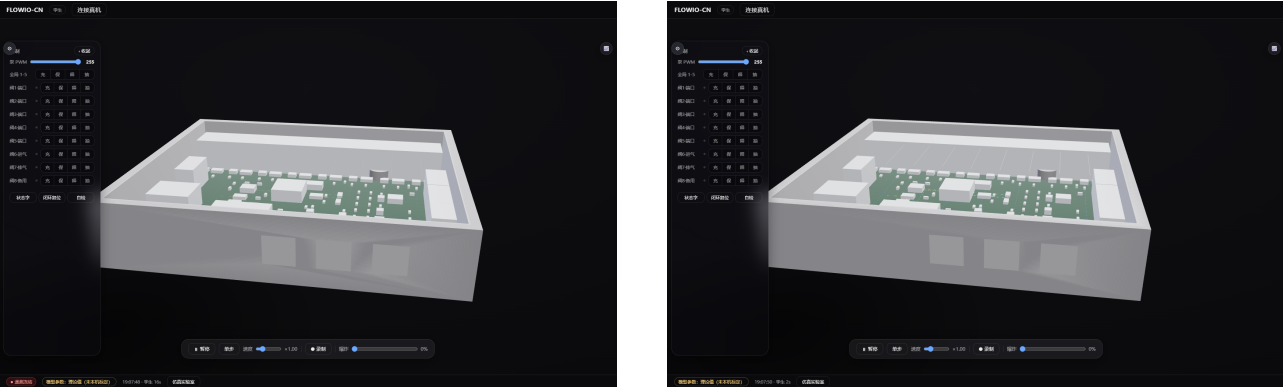


图 13.6: 目视审计之断连态（左：v2\_09\_stale，遥测冻结，流光回落底光；右：v2\_10\_recovered，恢复后重新点亮）

审计也留下一条已知事项并维持裁定：气流粒子 `depthTest:false` + 高渲染序，保证”通道存在、气流喷出”在默认视角（管路被壳遮挡）下始终可见——代价是亮壳表面上叠加粒子的对比度偏克制。像素审计走廊内 38px 青簇可检出，判定不改（spec §3.2 底光语义）。把这类”审过但保留”的记录写进书，与 `pass/fail` 一样是验收产物。

13.1.9 验收工单机制：一张工单，两份产物

上篇六张工单能”验收完即写完书”，靠的是机制本身。设计规格（2026-10-02-acceptance-book-design.md）把 BRINGUP 的 30 个 checkbox 与书稿 12 章的素材需求合流成 36 张验收工单：每张是一个 YAML 文件，放在

book/tickets/ 下、以工单号命名（A###.yaml）——头部记录元数据（相位 sw/hw/cal、对应 BRINGUP 章节、汇入书稿章、状态、证据要求），--- 之后是 Markdown 正文（操作步骤、实测值表、判定、素材管线指令）。一张工单完成等于两份产物落袋：BRINGUP 对应 checkbox 勾掉，书稿对应小节的图与数据就位。**fail** 的工单同样入书——“踩坑实录”专栏比 pass 更有教学价值。

管理工具 ticket.py（在 book/tools/ 下，纯标准库实现，YAML 头按“一行一字段”手写格式解析、不依赖 PyYAML）用三个子命令覆盖全生命周期：

Listing 13.1: ticket.py 用法（自项目根）

```
python book/tools/ticket.py new A225 "新验收项" hw 9+ --book-ch 11 # 生成模板
python book/tools/ticket.py report # 进度一屏
python book/tools/ticket.py set_status A205 pass # 更新状态
```

report 的实时输出即 36 张工单的总账（摘头尾）：

Listing 13.2: ticket.py report 输出（2026-10-02，摘录）

```
ID  phase status  title
-----
A101 sw      pass   孪生 v2 首屏装配叙事 + 爆炸滑杆
A102 sw      pass   电流/气流联动：I/V/S 命令差分截图
...  ...      ...    (A103--A106 软件侧全 pass)
A201 hw      pending 首电三测（万用表）
...  ...      ...    (硬件 24 张、标定 6 张全 pending)
A306 cal     pending 孪生对拍：闭环精度 + 数字孪生精度报告
-----
合计 36 张 (sw 6 / hw 24 / cal 6) : pass 6 • pending 30 • fail 0 • waived 0
```

软件侧六张（pass）即本篇 A101–A106 各小节与 §13.1.7、§13.1.8 的实录来源；其余三十张的下篇总表见 §13.2.2。状态机四态（pending/pass/fail/waived）中 fail 不算失败的项目、算进书资格，waived 留给“理性放弃并记录理由”的项——三者都会在书里留下正文。

## 13.2 下篇：实物验收（工单驱动，待回板）

下篇的三十张工单在等一块板：嘉立创制板 5 片、贴装 2 片（主力 + 备用，第 7 章下单实录）。这一篇先立好三样东西——执行节奏（§13.2.1）、工单总表（§13.2.2）、首电安全门（§13.2.3）——回板之日即可开工，边验收边把本章下篇从“总表”长成与上篇同体量的实录。

### 13.2.1 回板后工作流：每张工单 15–30 分钟

工单的粒度按“一个工作台上的一段专注时间”设计：执行 10–20 分钟，归档与写书 5–10 分钟。固定动线九步：

Listing 13.3: 单张工单动线（spec 2026-10-02-acceptance-book-design.md §4）

```
领工单 (ticket.py report 选 pending 头一张)
-> 照 firmware/BRINGUP.md 对应章节执行验收 (截图/拍照/录数)
-> 产物拷入 book/figures/ (图) 与 book/data/ (CSV)
-> 工单正文填实测值表 + 判定 (pass/fail + 容差)
-> ticket.py set_status <ID> <pass|fail|waived>
-> BRINGUP.md 对应 checkbox 勾掉
```

```
-> 书稿对应章 .tex 增图/表/叙述 (fail 走踩坑实录专栏)
-> xelatex 编译过 -> git commit
```

这个节奏的可贵之处在**不积压**：验收产物当天变成书稿，书稿编译错误当天暴露；三十张工单全部走完时，第 13 章与第 14 章的实物素材同步齐稿——验收完成日即书稿主体完成日，这是”验收 × 专著一体化”设计的全部赌注。

13.2.2 硬件 24 张 + 标定 6 张工单总表

表 13.4 由 book/tickets/A2\*.yaml 与 A3\*.yaml 的元数据生成，列出回板后待执行的三十张工单。硬件侧按 BRINGUP 九章动线排序：先通电视频流（A201–A204），再外设逐个点亮（A205–A208），无线与上位机（A209–A210），然后是六章校准（A211–A215，产物改 BOARD\_PARAMS 使黄徽转绿）、泵三实验与闭环（A216–A219），最后整机四项（A220–A224）。标定侧六张（A301–A306）是硬件数据的**二次消费**：实测曲线回流孪生后的对拍叠图，产物汇入第 14 章精度报告。

表 13.4: 硬件（hw）24 张 + 标定（cal）6 张工单总表（2026-10-02 生成，均 pending）

| ID   | 工单内容                                   | BRINGUP 节状态 |         |
|------|----------------------------------------|-------------|---------|
| A201 | 首电三测（万用表）                              | 1           | pending |
| A202 | CH340 烧录/自动复位                          | 1           | pending |
| A203 | 74HCT245 电平迁移（示波器）                     | 2           | pending |
| A204 | MOS 触发栅极/Vds 波形（示波器）                   | 2           | pending |
| A205 | WS2812 状态灯                             | 4           | pending |
| A206 | TCA9548A 五通道扫描                         | 3           | pending |
| A207 | 8 阀全功能（听声 + 电流）                        | 5           | pending |
| A208 | I <sup>2</sup> C tr 实测（100/400 kHz 裁决） | 6           | pending |
| A209 | BLE 三步（扫描/写/订阅）                        | 8           | pending |
| A210 | gui 真机模式（Web Bluetooth）                | 8           | pending |
| A211 | 电气校准 ① 阀线圈 R/L（万用表 + LCR）              | 7           | pending |
| A212 | 电气校准 ② AO3400 RDS（示波器反推）               | 7           | pending |
| A213 | 电气校准 ③ buck ESR/Cout（纹波反推）             | 7           | pending |
| A214 | 电气校准 ④ 负载调整率/效率（4 线法）                  | 7           | pending |
| A215 | 电气校准 ⑤⑥ $\theta_{ja}$ 红外 + ESP32 电流钳   | 7           | pending |
| A216 | 泵实验 A 充气死点（calibrate_pump.py）          | 9           | pending |
| A217 | 泵实验 B 真空死点（calibrate_pump.py）          | 9           | pending |
| A218 | 泵实验 C 升压曲线（calibrate_pump.py）          | 9           | pending |
| A219 | 闭环充气到目标 kPa                            | 9           | pending |
| A220 | 外壳装配（M3 自攻 + 端子开孔对位）                   | 9+          | pending |
| A221 | 整机气密（保压 60 s）                          | 9+          | pending |
| A222 | 长跑温升                                   | 9+          | pending |
| A223 | 8 阀轮巡                                  | 9+          | pending |
| A224 | 满载纹波                                   | 9+          | pending |
| A301 | 孪生对拍：阀驱动电流曲线（BOARD_PARAMS 改后叠图）        | 7           | pending |
| A302 | 孪生对拍：电源效率/纹波（4 线法实测叠图）                 | 7           | pending |
| A303 | 孪生对拍：热学温升（ $\theta_{ja}$ 修正后叠图）        | 7           | pending |
| A304 | 孪生对拍：充气升压曲线（实验 C CSV 叠图）               | 9           | pending |

表 13.4（续）

| ID   | 工单内容                     | BRINGUP 节状态 |         |
|------|--------------------------|-------------|---------|
| A305 | 孪生对拍：真空死点曲线（实验 B CSV 叠图） | 9           | pending |
| A306 | 孪生对拍：闭环精度 + 数字孪生精度报告     | 9           | pending |

表里埋着几处跨章的伏笔，回板时会逐一回收：A203/A204 的示波器波形对照第 5 章电平迁移与低边驱动的拓扑选择；A208 用 `tr` 实测裁决 100/400 kHz（第 5 章 I<sup>2</sup>C 上拉告警的实物侧答案）；A211–A215 的六项校准常数逐一直接写入 `board_model.py` 的 `BOARD_PARAMS`，UI“未标定”黄徽章（图 11.1 右下状态行）转绿的那一截图，就是 A215 的收官证据；A216–A218 的泵三实验脚本 `calibrate_pump.py` 已备好，输出“直接可抄的 `#define` 修改建议”（第 4 章 §4.11）。

### 13.2.3 首电安全三测预告（A201）：万用表的唯一保留环节

三十张工单大多以“软件读数”取证（串口日志、遥测截图、示波器波形），唯有一张从上个世纪活到现在——A201 首电三测，工具栏里只有一只万用表。理由很硬：软件无法保护你免于第一次上电的短路。烧录、遥测、BLE 全部成立的前提是 5V/3V3 轨活着，而“轨活着”在首电那一刻只能用表说话。三测动线：

1. **轨阻抗**：不上电，万用表二极管/电阻档测 5V 与 3V3 轨对地阻抗，排短路——贴装板的第一道闸，第 7 章收货验收的延伸；
2. **限量首电**：经限流（电源限流挡或 USB 电流表）首电，观察静态电流应为 mA 级而非安级——安级即回第 1 步找短路；
3. **轨电压**：表笔确认 5V 与 3V3 电压在容差内（buck 空载输出对照第 4 章 §4.8 的标称值），方可进入 A202 烧录。

三测的实测读数与表笔照片按素材管线归档为 A201 证据。它是整条动线的第一张工单，也是“数字孪生验收（上篇）”与“实物验收（本篇）”的分界：从此以后，每一个数字都长在真板上。

## 本章小结

上篇用六张 `pass` 工单证明了“回板之前，软件无欠账”：装配叙事与爆炸跟随（A101）、三态差分联动（A102）、四电路重算（A103）、SDK 双向向量（A104）、固件两道防线（A105）、BLE 先验冻结（A106），背后是 44 断言矩阵、26 项目视审计与三层证据哲学（断言防逻辑错、像素防渲染错、AI 复核防语义错）。下篇用三十张 `pending` 工单规定了回板之后怎么还账：15–30 分钟一张的九步动线、按 BRINGUP 九章排序的总表、以及一只万用表守着的首电安全门。上下两篇合起来，才是“验收”的完整定义——已完成的可复核，未完成的可执行。

## 第十四章 产品化：标定、精度与路线

### 14.1 Phase CAL 六项对拍

A301–A306：改 BOARD\_PARAMS 与 PHYSICS 常数后，孪生曲线与实测曲线叠图——阀电流、电源效率、温升、充气升压、真空死点、闭环精度六个维度。

### 14.2 数字孪生精度报告

对拍偏差的量化呈现：产品级的”模型可信度”文档，是 FLOWIO-CN 区别于普通开源套件的核心卖点。

### 14.3 路线图

P2 方向：更多执行器通道、无线同步、批量产线标定工装；以及从单板到整机的供应链收敛。

## 参考文献

- [1] Xiaoyu Cai et al. “Implementation of a Virtual Reality Based Digital-Twin Robotic Minimally Invasive Surgery Simulator”. In: *Bioengineering* 10.11 (2023), p. 1302. DOI: [10.3390/bioengineering10111302](https://doi.org/10.3390/bioengineering10111302).
- [2] Ozgun Kilic Afsar et al. “OmniFiber: Integrated Fluidic Fiber Actuators for Weaving Movement based Interactions into the ‘Fabric of Everyday Life’”. In: *Proceedings of the 34th ACM Symposium on User Interface Software and Technology (UIST '21)*. 2021, pp. 1010–1026. DOI: [10.1145/3472749.3474802](https://doi.org/10.1145/3472749.3474802).
- [3] Qinglin Qi et al. “Enabling technologies and tools for digital twin”. In: *Journal of Manufacturing Systems* 58 (2021), pp. 3–21. DOI: [10.1016/j.jmsy.2019.10.001](https://doi.org/10.1016/j.jmsy.2019.10.001).
- [4] Lei Ren et al. “Digital Twin Robotic System With Continuous Learning for Grasp Detection in Variable Scenes”. In: *IEEE Transactions on Industrial Electronics* 71.7 (2024), pp. 7650–7660. DOI: [10.1109/TIE.2023.3299049](https://doi.org/10.1109/TIE.2023.3299049).
- [5] Mario Sansone et al. “Understanding Shape Memory Alloy Torsional Actuators: From the Conceptual to the Preliminary Design”. In: *Actuators* 11.3 (2022), p. 81. DOI: [10.3390/act11030081](https://doi.org/10.3390/act11030081).
- [6] Ali Shtarbanov. “FlowIO Development Platform—the Pneumatic ‘Raspberry Pi’ for Soft Robotics”. In: *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems (CHI EA '21)*. 2021, pp. 1–6. DOI: [10.1145/3411763.3451513](https://doi.org/10.1145/3411763.3451513).
- [7] Ali Shtarbanov. “Modular Development Platforms and Creative Ecosystems: Design & Deployment for Wide Impact Across Fields”. PhD thesis. Cambridge, MA: Massachusetts Institute of Technology, 2025.
- [8] Omkar Singh and Anjan Kumar Ray. “A physical-virtual based digital twin robotic hand”. In: *International Journal on Interactive Design and Manufacturing (IJIDeM)* 18 (2024), pp. 4829–4839. DOI: [10.1007/s12008-024-01773-7](https://doi.org/10.1007/s12008-024-01773-7).
- [9] Andrew A. Stanley et al. “Lumped-Parameter Response Time Models for Pneumatic Circuit Dynamics”. In: *ASME Journal of Dynamic Systems, Measurement, and Control* 143.5 (2021), p. 051001. DOI: [10.1115/1.4049009](https://doi.org/10.1115/1.4049009).
- [10] Fei Tao et al. “Digital Twin in Industry: State-of-the-Art”. In: *IEEE Transactions on Industrial Informatics* 15.4 (2019), pp. 2405–2415. DOI: [10.1109/TII.2018.2873186](https://doi.org/10.1109/TII.2018.2873186).
- [11] Fei Tao et al. “Digital Twin Modeling”. In: *Journal of Manufacturing Systems* 64 (2022), pp. 372–389. DOI: [10.1016/j.jmsy.2022.06.015](https://doi.org/10.1016/j.jmsy.2022.06.015).
- [12] Matheus S. Xavier, Andrew J. Fleming, and Yuen K. Yong. “Model-Based Nonlinear Feedback Controllers for Pressure Control of Soft Pneumatic Actuators Using On/Off Valves”. In: *Frontiers in Robotics and AI* 9 (2022), p. 818187. DOI: [10.3389/frobt.2022.818187](https://doi.org/10.3389/frobt.2022.818187).
- [13] Matheus S. Xavier, Andrew J. Fleming, and Yuen K. Yong. “Modelling and Simulation of Pneumatic Sources for Soft Robotic Applications”. In: *2020 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*. 2020, pp. 916–921. DOI: [10.1109/AIM43001.2020.9158802](https://doi.org/10.1109/AIM43001.2020.9158802).
- [14] Disheng Xie et al. “A Portable Soft Robotic Glove with Fully Functional Thumb Assistance for Complex Dexterous Fine Motor Skills”. In: *Advanced Science* (2026), e76275. DOI: [10.1002/advs.76275](https://doi.org/10.1002/advs.76275).
- [15] Lining Yao et al. “PneUI: Pneumatically Actuated Soft Composite Materials for Shape Changing Interfaces”. In: *Proceedings of the 26th ACM Symposium on User Interface Software and Technology (UIST '13)*. 2013, pp. 13–22. DOI: [10.1145/2501988.2502037](https://doi.org/10.1145/2501988.2502037).
- [16] Ayse Feyza Yilmaz, Gökhan Ince, and Ozgur Atalay. “Fabric-Based Wearable Robotic Exoskeleton Gloves: Advancements and Challenges”. In: *Advanced Materials Technologies* 11 (2026). DOI: [10.1002/admt.202502282](https://doi.org/10.1002/admt.202502282).